Ph.D. Thesis Proposal

Uncertainty-Aware Spatial Perception for Generalizable State Estimation

Yuheng Qiu

September 2025

Department of Mechanical Engineering
College of Engineering
Carnegie Mellon University
Pittsburgh, Pennsylvania

Thesis Committee:

Sebastian Scherer	Carnegie Mellon University *(Chair)*
Wenshan Wang	Carnegie Mellon University
Aaron Johnson	Carnegie Mellon University
Kostas Daniilidis	University of Pennsylvania

Submitted in partial fulfillment of the requirements for the degree of Doctor of Philosophy in Robotics.

Abstract

Robust spatial perception must work reliably across any environment and any condition, much as mammals navigate seamlessly from bright open terrain to dark confined spaces, through occlusions, crowds, and sudden disturbances, because their brains maintain uncertainty understanding over visual and inertial signals and use it to govern spatial reasoning and every downstream decision. Current robotic perception systems fall short of this standard because they rely on hand-tuned covariance matrices and fixed noise parameters that do not transfer across sensors, platforms, or operating conditions. This dissertation develops *learned, metrics-aware uncertainty* as the unifying principle that replaces brittle hand-tuned models with data-driven covariance estimation, enabling generalizable state estimation across diverse environments and robotic platforms.

The thesis builds uncertainty-aware spatial perception in four progressive parts. Part I addresses the most information-rich modality first: vision. We propose MAC-VO, which learns metrics-aware 2D matching covariance and propagates it into full 3D covariance for correspondence selection and residual weighting in stereo visual odometry, and QuantMAC-VO, which compresses this pipeline for real-time edge deployment via sim-to-real calibration and kurtosis-based hybrid quantization. However, vision alone degrades under occlusion, blur, and aggressive motion, so Part II establishes a complementary inertial foundation. AirIMU jointly learns IMU noise correction and covariance propagation through differentiable preintegration, and AirIO exploits body-frame IMU representation for observability-aware inertial odometry under agile dynamics.

With both modalities now producing metrics-aware covariance, Part III fuses them for robust multi-modal estimation and cross-platform adaptation. MAC-I^2 uses learned covariance from vision and inertia for tuning-free visual-inertial initialization and online calibration; MACVIO extends this to cross-modal uncertainty correlation learning for adaptive fusion; and MAC-IO uses EKF-based structured self-supervision to adapt inertial models on real data across platforms. Finally, Part IV scales uncertainty-aware estimation to a complete spatial perception system. MACSLAM integrates learned multi-frame covariance into global factor-graph optimization for robust data association, loop closure, and multi-session mapping.

The framework has been validated across a wide range of real-world deployments, including nighttime off-road driving, agile AI-powered drone navigation, and space robotics tasks such as the NASA Lunar Autonomy Challenge. Together, these contributions form a unified framework for uncertainty-aware spatial perception that is accurate, robust, and deployable for any robot, in any environment, with minimal manual tuning.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

Robust spatial perception must work reliably across any environment and any condition. Mammals achieve this by continuously estimating the reliability and uncertainty of visual and inertial cues, filtering distractions, down-weighting ambiguous evidence, and integrating trustworthy signals to maintain stable spatial reasoning for navigation and action. Current robotic perception systems fall short of this standard. They rely on hand-tuned covariance matrices and fixed noise parameters that do not transfer across sensors, platforms, or operating conditions. When two sensors are fused with miscalibrated weights, the estimator either over-trusts a degraded modality or ignores a reliable one, producing state estimates that are worse than either sensor alone. Deployment on a new robot therefore requires human retuning, and downstream tasks such as planning and control receive unreliable state estimates.

The bottleneck is not only model capacity or data scale, but also uncertainty representations that remain valid under distribution shift. In the environment, lighting changes, dynamic objects, textureless regions, and motion blur all alter measurement reliability. Across platforms, camera and IMU characteristics differ, while vibration, temperature, and mounting conditions change noise statistics over time. Most existing systems therefore fail when deployed outside their calibration regime, blocking the goal of spatial perception that works for any robot, anywhere, anytime, with minimal manual tuning.

This dissertation develops *learned, metrics-aware uncertainty* as a core mechanism and uses it to build a full, deployable spatial perception system. Rather than treating covariance as a fixed parameter, the framework learns metrics-aware covariance from data and integrates it across the full stack, from visual and inertial front-ends to cross-modal fusion and global SLAM optimization, so the system transfers across cameras, IMUs, platforms, and conditions. We propose an uncertainty-aware spatial perception system that enables robust state estimation, principled multi-modal fusion and calibration, and efficient deployment through uncertainty-guided compute allocation.

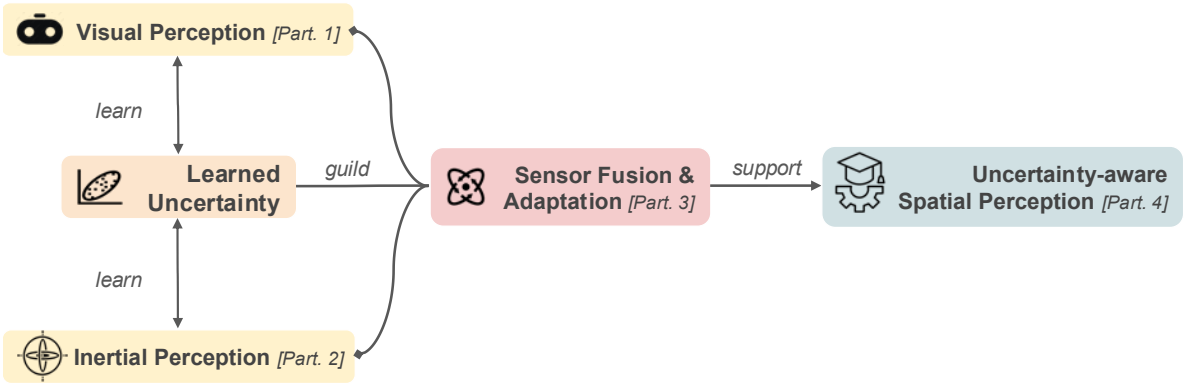

Figure 1.1: Overview of the proposed thesis framework across four parts: visual uncertainty learning, inertial uncertainty learning, cross-modal fusion and adaptation, and unified uncertainty-aware SLAM.

1.1 Thesis Statement

Thesis Statement
Spatial perception systems can achieve robust, adaptable operation across diverse environments and platforms with minimal additional human engineering by learning metrics-aware covariance.

- *Minimal additional human engineering* is defined as deployment to a new robot or environment without repeated manual retuning of noise parameters, fusion weights, and calibration settings.
- *Metrics-aware covariance* is defined as uncertainty estimates with physically meaningful scale that can be used directly to weight correspondences, residuals, and sensor factors in estimation.
- *Robust, adaptable operation* is defined as maintaining estimation accuracy under occlusion, lighting change, motion blur, and dynamic scenes while transferring across cameras, IMUs, and platforms with little manual intervention.

1.2 Thesis Organization

Part I: Vision Uncertainty and Learning-based Visual Odometry. The first challenge is to learn visual uncertainty that is both robust and efficient under real deployment constraints. Current visual odometry systems either rely on fixed geometric uncertainty models or produce scale-agnostic confidence estimates that fail under illumination shifts, texture variation, and camera-domain transfer. Chapter 3 presents **MAC-VO**, a learning-based stereo visual odometry system that jointly learns metrics-aware matching uncertainty alongside pose estimation. MAC-VO propagates 2D covariance into full 3D covariance with inter-axis correlations for correspondence selection and residual weighting. Chapter 4 presents **QuantMAC-VO**, which compresses this pipeline for real-time edge deployment via sim-to-real calibration and kurtosis-based

hybrid quantization, with negligible accuracy loss. Together, these chapters establish robust and deployable visual uncertainty modeling.

Part II: Inertial Uncertainty and Learning-based Inertial Odometry. The second challenge is to learn inertial uncertainty and representation so that inertial odometry remains reliable across IMU grades, platforms, and agile motion. Classical IMU preintegration uses static noise parameters, and many learning-based methods use representations that lose observability under aggressive dynamics. Chapter 5 presents **AirIMU**, which jointly learns IMU noise correction and uncertainty propagation through a differentiable preintegration pipeline, producing metrics-aware inertial covariance that generalizes across IMU grades and robotic platforms. Chapter 6 presents **AirIO**, which shows that preserving body-frame IMU representation improves observability under agile UAV dynamics, and fuses the learned inertial model with an uncertainty-aware EKF for learning-based inertial odometry without external sensors. Together, these chapters deliver transferable inertial perception with learned uncertainty.

Part III: Sensor Fusion and Adaptation with Metrics-Aware Uncertainty. (Partially Completed) The third challenge is to fuse visual and inertial uncertainty as a common currency for initialization, calibration, and cross-platform adaptation without per-platform retuning. Existing visual-inertial systems require manual balancing of modality trust and often degrade when moved to new platforms without supervised labels. Chapter 7 presents **MAC-I^2** (completed), which uses metrics-aware covariance from both vision and inertia for robust, tuning-free visual-inertial initialization and online calibration. Chapter 8 presents **MACVIO** (proposed), which extends this to cross-modal uncertainty correlation learning for adaptive fusion weights and sensor reliability handling. Chapter 9 presents **MAC-IO** (proposed), which uses EKF and IMU preintegration as structured self-supervision signals to adapt uncertainty-aware inertial models on real data, closing the loop between estimation structure and domain transfer. Together, these chapters aim to enable multi-modal perception systems that adapt to new sensors, platforms, and environments with minimal human tuning.

Part IV: Full Uncertainty-Aware Spatial Perception. (Proposed) The final challenge is to carry learned uncertainty into global SLAM so that data association, loop closure, and graph optimization remain consistent under distribution shift. Most SLAM back-ends still assume fixed diagonal noise and ignore correlations induced by shared learned features and temporal structure. Chapter 10 presents **MACSLAM** (proposed), which builds on a unified visual perception head that provides geometric features, visual place recognition, and semantic and language-grounded representations. MACSLAM integrates learned multi-frame measurement covariance into global factor-graph optimization, enabling uncertainty-aware data association, robust loop closure, and consistent multi-session mapping. This part brings the thesis full circle to the mammalian blueprint that motivated this work by unifying local estimation from Parts I–III with global consistency.

1.3 Proposed Contributions

This dissertation makes the following key contributions across four parts:

Part I: Vision Uncertainty and Learning-based Visual Odometry

- **C1. MAC-VO: Visual Perception Uncertainty Learning [Completed].** MAC-VO introduces a metrics-aware covariance learning framework for stereo visual odometry that predicts uncertainty directly from visual features rather than relying on fixed geometric models. Trained across diverse cameras and lighting, MAC-VO yields scale-consistent uncertainty that reflects correspondence reliability and improves localization and fusion.
- **C2. QuantMAC-VO: Efficient VO Deployment via Quantization [Completed].** QuantMAC-VO presents a full-stack quantization framework for learning-based VO that enables real-time deployment on edge hardware. With sim-to-real calibration and kurtosis-based hybrid quantization, it achieves up to 1.83× speedup on NVIDIA Jetson Thor with negligible accuracy degradation.

Part II: Inertial Uncertainty and Learning-based Inertial Odometry

- **C3. AirIMU: Inertial Uncertainty Learning [Completed].** AirIMU develops a data-driven framework for learning IMU preintegration uncertainty that captures platform- and motion-specific noise characteristics. Using differentiable integration and covariance propagation, AirIMU enables accurate uncertainty modeling across robotic platforms.
- **C4. AirIO: Uncertainty-Aware Inertial Odometry [Completed].** AirIO enhances inertial odometry by leveraging learned IMU uncertainty and improved observability, enabling robust operation in GPS-denied environments and reduced drift over long trajectories.

Part III: Sensor Fusion and Adaptation with Metrics-Aware Uncertainty

- **C5. MAC-I^2: Visual-Inertial Initialization and Calibration [Completed].** MAC-I^2 provides robust VI initialization and online calibration by leveraging metrics-aware uncertainty from both MAC-VO and AirIMU. It achieves tuning-free initialization across challenging environments including extreme exposure, occlusions, and dynamic objects.
- **C6. MACVIO: Cross-Modal Uncertainty Fusion [Proposed].** MACVIO learns cross-modal uncertainty correlations to enable adaptive fusion weights, sensor selection, and improved consistency (see Chapter 8).
- **C7. MAC-IO: Self-Supervised Inertial Adaptation [Proposed].** MAC-IO uses EKF and IMU preintegration as structured self-supervision to adapt uncertainty-aware inertial models on real data across platforms (see Chapter 9).

Part IV: Full Uncertainty-Aware Spatial Perception

- **C8. MACSLAM: Uncertainty-Aware SLAM [Proposed].** MACSLAM models multi-frame measurement covariance, cross-factor correlations, and calibration drift in a global factor graph to improve data association, loop closure, and optimization (see Chapter 10).

Chapter 2

Background and Related Work

This chapter establishes the technical foundations, formalizes the thesis problem, and briefly positions prior work. Sec. 2.1 introduces the notation, mathematical tools, and estimation frameworks used throughout the dissertation. Sec. 2.2 defines uncertainty-aware spatial perception as a learning-and-inference problem over visual and inertial measurements. Sec. 2.3 provides a concise related-work overview focused on the four parts of this thesis.

2.1 Notation and Preliminaries

2.1.1 Notation

Throughout this thesis, bold lowercase letters ($\mathbf{x}$) denote vectors, bold uppercase letters ($\mathbf{R}$) denote matrices, and calligraphic letters ($\mathcal{X}$) denote sets. We use $\mathbf{R} \in \mathrm{SO}(3)$ for rotation matrices and $\mathbf{T} \in \mathrm{SE}(3)$ for rigid-body transformations. The operator $[\cdot]_\times$ constructs the skew-symmetric matrix from a 3-vector. $\mathrm{Exp}(\cdot)$ and $\mathrm{Log}(\cdot)$ denote the exponential and logarithmic maps between Lie groups and their algebras. Superscripts and subscripts indicate reference frames: $\mathbf{T}_b^a$ transforms vectors from frame $\{b\}$ to frame $\{a\}$.

2.1.2 Factor Graph Optimization

Factor graph optimization [8] provides the unifying estimation framework for this thesis. A factor graph is a bipartite graph $\mathcal{G} = (\mathcal{X}, \mathcal{F}, \mathcal{E})$, where variable nodes $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ represent unknown states (poses, velocities, biases, landmarks) and factor nodes $\mathcal{F} = \{f_1, \ldots, f_m\}$ encode probabilistic constraints derived from sensor measurements. Maximum a posteriori (MAP) estimation seeks:

$$\mathcal{X}^* = \arg\min_{\mathcal{X}} \sum_k \|\mathbf{r}_k(\mathcal{X}_k)\|^2_{\mathbf{\Sigma}_k}, \tag{2.1}$$

where $\mathbf{r}_k$ is the residual of the k-th factor, $\mathbf{\Sigma}_k$ is its covariance matrix, and $\|\mathbf{r}\|^2_{\mathbf{\Sigma}} = \mathbf{r}^T\mathbf{\Sigma}^{-1}\mathbf{r}$ is the Mahalanobis norm. The covariance $\mathbf{\Sigma}_k$ determines how much the optimizer trusts each measurement; miscalibrated covariances lead to over- or under-weighting of factors, which is the central problem this thesis addresses.

2.1.3 Stereo Visual Odometry

Stereo visual odometry (VO) estimates frame-to-frame camera motion from stereo image sequences. Given calibrated stereo pairs, a VO front-end establishes feature correspondences and triangulates 3D points $\mathbf{p}_i \in \mathbb{R}^3$. The VO back-end then estimates the relative pose $\mathbf{T}_{c_k}^{c_{k-1}}$ by minimizing:

$$\mathbf{T}_{c_k}^{c_{k-1}*} = \arg\min_{\mathbf{T}} \sum_i \left\| \mathbf{p}_{k-1,i}^c - \mathbf{T}\mathbf{p}_{k,i}^c \right\|_{\mathbf{\Sigma}_i}^2, \tag{2.2}$$

where $\mathbf{\Sigma}_i$ is the covariance of the i-th correspondence and determines its influence on pose estimation. In practice, VO robustness depends critically on this weighting: ambiguous or mismatched correspondences should be down-weighted, while geometrically consistent correspondences should dominate. Traditional VO typically uses fixed diagonal covariance or disparity-only heuristics, which become unreliable under illumination change, low texture, motion blur, and dynamic objects. This limitation motivates learning metrics-aware covariance for correspondence selection and residual weighting in Part I of this thesis.

2.1.4 IMU Preintegration

The IMU measures linear acceleration $\mathbf{a}$ and angular velocity $\boldsymbol{\omega}$ at high frequency. IMU preintegration [102] computes relative rotation, velocity, and position increments between two keyframes without repeated re-integration when the linearization point changes. The preintegrated measurements from frame i to frame j are:

$$\begin{aligned}
\boldsymbol{\gamma}_{ij} &= \prod_{k=i}^{j-1} \mathrm{Exp}(\boldsymbol{\omega}_k \Delta t), \\
\boldsymbol{\beta}_{ij} &= \sum_{k=i}^{j-1} \mathbf{R}_{b_k}^{b_i} \mathbf{a}_k \Delta t, \\
\boldsymbol{\alpha}_{ij} &= \sum_{k=i}^{j-1} \left(\boldsymbol{\beta}_{ik} \Delta t + \tfrac{1}{2} \mathbf{R}_{b_k}^{b_i} \mathbf{a}_k \Delta t^2 \right),
\end{aligned} \tag{2.3}$$

where $\boldsymbol{\gamma}_{ij}$, $\boldsymbol{\beta}_{ij}$, and $\boldsymbol{\alpha}_{ij}$ denote the rotation, velocity, and position preintegration terms, respectively. The associated covariance $\mathbf{\Sigma}_{ij}$ is propagated through the linearized dynamics:

$$\mathbf{\Sigma}_{ik+1} = \mathbf{A}_k \mathbf{\Sigma}_{ik} \mathbf{A}_k^T + \mathbf{B}_k \mathrm{diag}(\mathbf{\Sigma}^{gyro}, \mathbf{\Sigma}^{acc}) \mathbf{B}_k^T, \tag{2.4}$$

where $\mathbf{A}_k$ and $\mathbf{B}_k$ are the state transition and noise Jacobian matrices. Traditional methods set $\mathbf{\Sigma}^{gyro}$ and $\mathbf{\Sigma}^{acc}$ to fixed values from datasheet specifications or Allan deviation analysis, which fail to capture motion-dependent noise characteristics.

2.2 Problem Definition

The goal of this thesis is to build a spatial perception system that estimates robot motion and map structure while explicitly modeling uncertainty that transfers across environments and platforms. Let the latent trajectory and map be $\mathcal{S} = \{\mathcal{X}_{1:T}, \mathcal{M}\}$, where $\mathcal{X}_{1:T}$ denotes the sequence of

robot states and $\mathcal{M}$ denotes map variables. Given synchronized visual and inertial measurements $\mathcal{Z}_{1:T} = \{\mathbf{z}_1, \ldots, \mathbf{z}_T\}$, the classical objective is:

$$p(\mathcal{S} \mid \mathcal{Z}_{1:T}). \tag{2.5}$$

In practice, this posterior is approximated by factor-graph optimization with measurement residuals and covariance weighting (see Sec. 2.1.2). The central limitation is that most systems use fixed or heuristically tuned covariance models that become miscalibrated under distribution shift. This thesis treats covariance as a learned, metrics-aware quantity and optimizes:

$$\min_{\theta, \mathcal{S}} \sum_{k} \|\mathbf{r}_k(\mathcal{S}; \mathcal{Z}_{1:T})\|^2_{\mathbf{\Sigma}_k(\mathcal{Z}_{1:T};\theta)}\,, \tag{2.6}$$

where θ denotes parameters of learned uncertainty models and $\mathbf{\Sigma}_k(\cdot;\theta)$ denotes data-conditioned covariance.

Under this formulation, the thesis addresses three linked objectives: accurate local estimation, robust cross-modal fusion, and globally consistent SLAM. The key requirement is transfer with minimal manual retuning across cameras, IMUs, and deployment domains.

2.3 Related Work

Visual uncertainty for odometry. Classical visual odometry and SLAM methods rely on geometric residuals with fixed or heuristic covariance weighting [6, 20, 33]. Learning-based methods improve matching and depth quality [12, 28, 41], but most uncertainty outputs are relative confidence scores without physically meaningful scale. The remaining gap is metrics-aware uncertainty that can be used directly in estimation and remains reliable under domain shift. This gap motivates Part I (Chapters 3 and 4).

Inertial uncertainty and representation. Traditional inertial odometry uses calibrated noise parameters and analytic propagation [102, 110], while learning-based IO improves motion estimation and bias handling [3, 103, 115]. However, prior work either assumes fixed uncertainty parameters or relies on representations that do not transfer well to high-dynamic platforms. The key gap is jointly learning inertial uncertainty and observability-preserving representation for robust transfer across IMU grades and platforms. This gap motivates Part II (Chapters 5 and 6).

Visual-inertial fusion and adaptation. Modern VIO and VI initialization systems are accurate but require hand-tuned weighting and calibration choices [6, 72, 134]. Existing online calibration and fusion frameworks improve robustness but still depend on careful parameter adjustment [24, 166]. The gap is a shared uncertainty representation that enables tuning-free fusion and cross-platform adaptation. This gap motivates Part III (Chapters 7, 8, and 9).

Global SLAM under learned uncertainty. Factor-graph SLAM back-ends usually assume fixed diagonal noise for local and loop-closure factors [8, 15, 17]. Although learned features improve front-end robustness, uncertainty is rarely propagated as structured multi-frame covariance into global optimization. The gap is uncertainty-aware global SLAM that models cross-frame correlation for stable data association and loop closure under shift. This gap motivates Part IV (Chapter 10).

Chapter 3

MAC-VO: Metrics-Aware Covariance for Stereo Visual Odometry

3.1 Introduction

Visual Odometry (VO) predicts the relative camera pose from image sequences and often serves as the front-end of Simultaneous Localization and Mapping (SLAM) systems. Over the past few decades, both geometric and learning-based methods have been developed with significant advances in generalizability and accuracy [12, 20–22]. However, VO remains a challenging problem in real-world scenarios, with multiple visually degraded scenarios such as low illumination, dynamic and texture-less scenes.

To improve the robustness in challenging scenes, geometric-based VO algorithms employ outlier filtering strategies [23] and weigh the optimization residuals by the covariance of the observed features [24]. However, how to effectively *select the reliable keypoints* and *model their covariance* becomes two significant challenges. Existing methods typically select the keypoints based on local intensity gradient with a manually defined threshold [25–27]. These approaches lead to errors and outliers because they don't model the structure or context information of the environment (e.g., features on repetitive patterns may not be ideal candidates despite high image gradients). Moreover, the covariance model is often empirically modeled using a constant parameter, which is sub-optimal. In addition, the parameters in the keypoint selection and covariance model need to be extensively tuned for different environments.

With advances in learning-based visual features, more algorithms utilize learned features [25] to optimize the camera pose. Confidence score [28] or confidence weights [12, 22] of these feature points are often obtained in an unsupervised manner. The learned confidence helps to track the features and model the reliability during the optimization. However, these confidence or uncertainty values are scale-agnostic, which means they don't reflect the actual estimation error in the 3D space. This scale-agnostic problem brings two limitations. Firstly, it makes the covariance inconsistent across different environments that vary in scale, such as indoors and outdoors. Secondly, it makes it harder to integrate multiple constraints from different modalities or sensors.

To overcome the above challenges, this chapter addresses the problem of *modeling metrics-aware covariance values for the 3D keypoints*. More specifically, this is achieved through two

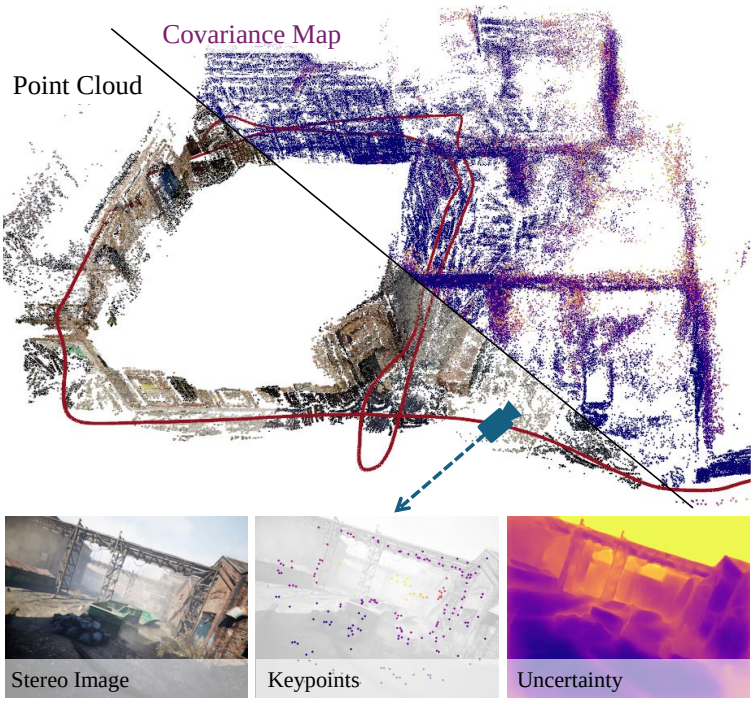

Figure 3.1: Without relying on multi-frame bundle adjustment, MAC-VO aggregately reconstructs the map based on the two-frame Pose graph optimization. We propose a *metrics-aware covariance* model for 3D keypoints based on learned matching uncertainty.

innovations. Firstly, we propose a learning-based model to quantify the 2D metrics-aware uncertainty of feature matching. Inspired by the FlowFormer [29] and GMA [30], we employ an iterative update model and motion aggregator to predict the uncertainty in 2D image space, which helps to filter unreliable features in the occluded region or low-illumination area. Secondly, based on the learned 2D uncertainty values, we model the covariance of the feature points in 3D space using a *metrics-aware 3D covariance model*. Compared to DROID-SLAM [12], which utilizes a scale-agnostic diagonal covariance matrix, our approach provides a more accurate representation by modeling the covariance of 3D feature points. This covariance model includes the inter-axes correlation of the 3D features. In the ablation study, we demonstrate the inter-frame consistency and the intra-frame consistency of our proposed covariance model.

We integrate the above two innovations into MAC-VO, a stereo VO that features superior keypoint selection and pose graph optimization based on the metrics-aware covariance model and achieves accurate tracking results in challenging cases compared with state-of-the-art VOs and even some SLAM systems without fine-tuning or multi-frame optimization. In summary, the main contributions of this chapter are:

- We present a learning-based 2D uncertainty network with metrics awareness, leveraging an iterative motion aggregator to capture the inconsistency of the feature matching. We use this metrics-aware uncertainty to evaluate the quality of features in order to select keypoints and guide the back-end optimization.
- This chapter introduces a novel metrics-aware 3D covariance model based on the 2D uncertainty of feature matching and depth estimation. The ablation study demonstrates the necessity of scale consistency and the off-diagonal terms in the pose graph optimization.

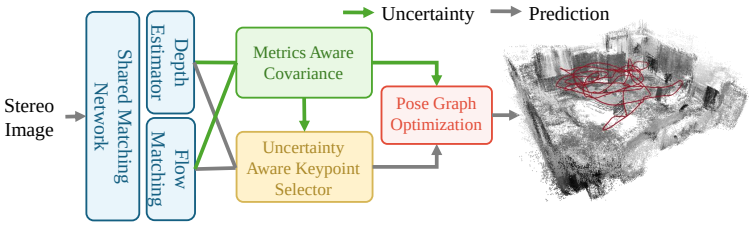

Figure 3.2: MAC-VO pipeline. It employs a shared matching network to jointly estimate depth, optical flow, and corresponding uncertainties. The learned uncertainty is utilized in the keypoint selector, the metrics-aware 3D covariance model, and the back-end optimization.

- We propose the MAC-VO, a stereo VO pipeline that estimates the camera pose and registers 3D features with metrics-aware covariance. In the experiments, MAC-VO outperforms existing VO algorithms and even some SLAM algorithms in challenging environments.

3.2 Related Works

Existing geometric-based methods optimize the camera pose based on geometric constraints like re-projection error [20, 31] or photometric error [32–34]. To more accurately model the uncertainty of the depth, Civera et al. [35] and Montiel[36] investigate the inverse depth parameterization. These methods often use constant parameters and simple heuristics to model the covariance matrix of these errors during the factor graph optimization. For multi-sensor fusion [6, 24] and semantic SLAM [37, 38], the covariance model plays a significant role in weighting the confidence of different sensors and modules. To capture sensor uncertainty, the covariance models are often tuned based on empirical prior. However, these simplified covariance models fail to capture the complexity of the challenging environments.

Recent advances in deep learning have transformed research in optical flow estimation [39, 40], feature matching [28, 41], depth estimation [42–44], and end-to-end camera pose estimation [21, 45]. Several methodologies have been developed to address the uncertainty in estimating depth, flow, and pose. Dexheimer et al. [46] proposed a learned depth covariance function that is applied in downstream tasks like 3D reconstruction. Nie et al. utilize a self-supervised learning method to jointly train depth and depth covariance of images in the wild [47]. ProbFlow [48] proposes using post-hoc confidence measures to assess the per-pixel reliability of flow.

Amidst these developments, hybrid learning-based SLAM systems [43, 49–53] are emerging to synergize the geometric constraint with the adaptability of learning-based methods. To improve the reliability of the feature tracking process, some methods introduce learning-based uncertainty measurements or confidence scores [28, 54–57] for pose optimization. DROID-SLAM [12] utilizes a differentiable bundle adjustment layer to implicitly tune the uncertainty model. This method employs a simplified diagonal covariance model for the bundle adjustment. These methods focus on the relative confidence between features, but ignore the scale consistency of the covariance model.

For keypoint selection, geometric-based VO relies on hand-crafted features [26, 27], which detect the edges and corner points. Recent advances in learning-based methods train feature extractors in a data-driven manner [25, 58]. However, these keypoints also prioritize edge and

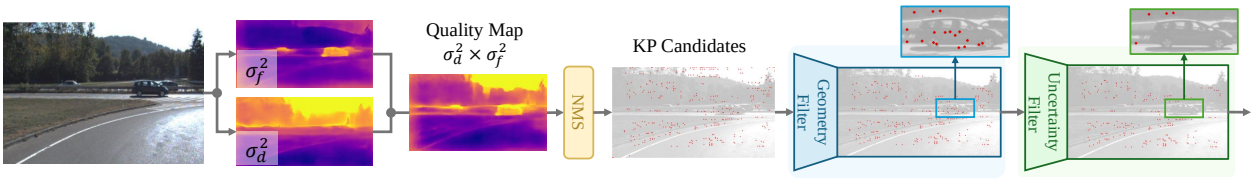

Figure 3.3: We include three filters: Non-minimum Suppression (NMS) filter, geometric filter, and uncertainty-based filter. In the KITTI dataset, the uncertainty filter implicitly filters out the unreliable feature matchings in the scene.

corner features due to data bias in the pre-training dataset. Recent works like D3VO [49] have shown that relying on edge and corner points can degrade state estimation, sometimes performing worse than random selectors [22]. The accuracy of learning-based feature matching and depth estimation algorithms is particularly compromised at object edges due to feature interpolation and the ambiguity in neural networks. In this chapter, we propose a keypoint selector based on learned uncertainty to filter out the unreliable features.

3.3 Method

As illustrated in Figure 3.2, MAC-VO outlines an effective integration of a learning-based front-end and a geometrically constrained back-end using the metrics-aware covariance model. In the front-end (Section 3.3.1), we train an uncertainty-aware matching network to model the corresponding uncertainty that stems from feature deficiencies. Utilizing the learned uncertainty, we develop a keypoint selector in Section 3.3.2 to choose reliable features. The metrics-aware 2D uncertainty is then propagated to the 3D space via the proposed covariance model in Sec. 3.3.3. In the back-end optimization (Sec. 3.3.4), we optimize the relative motion by minimizing the distance between registered keypoints weighted by the 3D covariance.

3.3.1 Network & Uncertainty Training

The objective of the proposed matching network is to estimate the flow $\hat{f} \in \mathbb{R}^2$ and the corresponding uncertainty $\hat{\Sigma}_f = \text{diag}(\sigma_u^2, \sigma_v^2)$ between two frames. We adopt the FlowFormer architecture [29] as the backbone for optical flow estimation, leveraging its transformer-based design. As shown in Figure 3.4, the recurrent decoder iteratively refines flow predictions by capturing ambiguities in feature correspondence within the cost volume's matching space, utilizing global context from the encoded cost memory.

To extend this framework and leverage both global motion cues and local features, we introduce a covariance decoder that predicts $\log \Delta\sigma$, the iterative updates of the uncertainty in log space. Operating the uncertainty update in log space facilitates additive updates, ensures positive variances, and stabilizes gradients. After iterative updates, the log-variance passes through an `exp` activation function to obtain the final uncertainty.

To supervise the uncertainty, we leverage the negative log-likelihood loss used in conformal prediction [59–61]:

$$L_{cov} = \sum_{i}^{N} \alpha_i \left((y - \hat{f}_i)^\top \hat{\Sigma}_f^{-1} (y - \hat{f}_i) + \log(\det \hat{\Sigma}_f) \right), \tag{3.1}$$

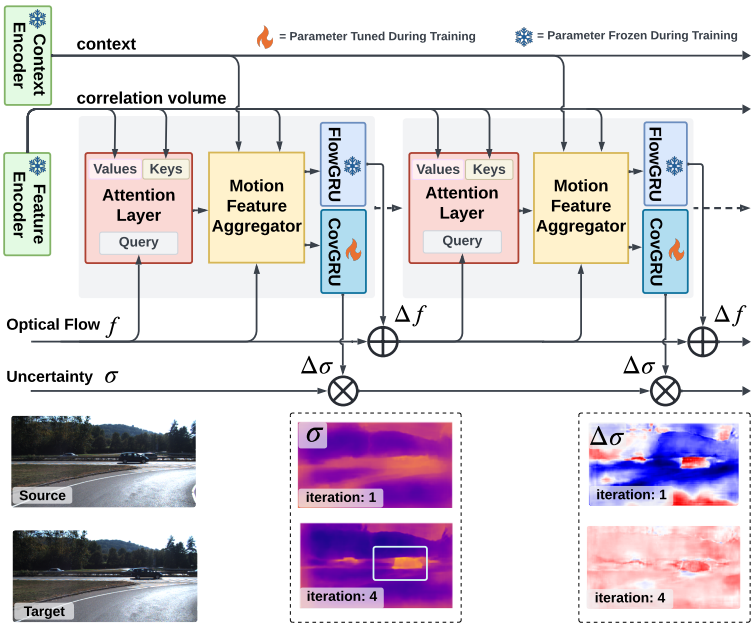

Figure 3.4: Top: Architecture of the uncertainty-aware matching network. **Bottom:** Each iteration, the model captures the inconsistency of the matching. For the $\Delta\sigma$, the red color indicates a positive $\Delta\sigma$ increasing the uncertainty, and blue means decreasing uncertainty.

where y is the ground truth optical flow, f_i denotes the i-th iteration of the network outputs, and α_i is the weight for each iteration and is set to decrease exponentially with a ratio of 0.8. During the training stage, we initialize the encoder network parameters with the pre-trained model by FlowFormer. We then train the covariance module on the synthetic dataset TartanAir [62], and evaluate the model in the TartanAirv2 sequences and other real-world datasets. Our experiments demonstrate that the model can generalize to real-world datasets without fine-tuning.

3.3.2 Uncertainty-based Keypoint Selection

Unlike the random selector used in DPVO[22] and the hand-crafted feature selector used in ORB-SLAM [26], we leverage the learned uncertainty estimation to filter out unreliable features. This is achieved by composing three filters: the *uncertainty filter*, *geometry filter*, and the *non-maximum suppression* (NMS). To prevent the clustering of keypoint candidates, we first apply the NMS filter, ensuring a more even spatial distribution of features. The geometry filter further removes keypoints located at the image boundaries and those outside the valid depth observation range. Finally, the uncertainty filter eliminates pixels with depth and flow uncertainty greater than 1.5 times the median uncertainty of the current frame. Illustrated in Figure 3.3, the uncertainty filter removes all keypoint candidates on the moving vehicle on a KITTI trajectory. Our proposed uncertainty estimation strategy also filters out keypoints on occluded objects, reflective surfaces, and feature-less areas, resulting in a more robust and reliable feature set.

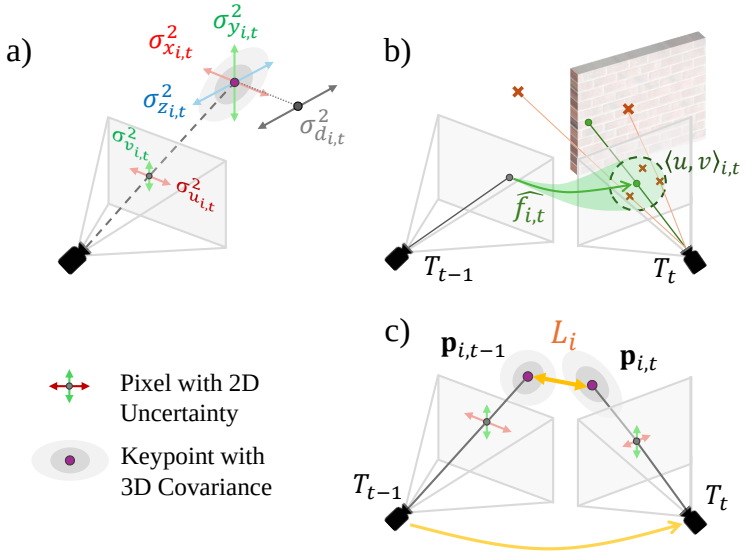

Figure 3.5: a) Uncertain estimation of depth due to the error in feature matching. **b)** Projecting depth and matching uncertainty from image plane to 3D space. **c)** Residual $\mathcal{L}_i$ for pose graph optimization.

3.3.3 Metrics-aware 3D Covariance Model

In the context of the camera projection geometry, the covariance of a 3D keypoint is determined by the uncertainty of the depth σ_d^2 and matching (σ_u^2, σ_v^2). To accurately model the covariance for 3D keypoints, it is critical to determine (1) the depth uncertainty of the matched points $\hat{\sigma}_d^2$ and (2) the off-diagonal covariance terms during the 2D-3D projection.

Project 2D Uncertainty to 3D Covariance Following the pinhole camera model with focal length $\mathrm{f}_x, \mathrm{f}_y$ and optical center $\mathrm{c}_x, \mathrm{c}_y$, the coordinate of the keypoint is calculated by: $\mathbf{x}_{i,t} = (\mathbf{u}_{i,t} - \mathrm{c}_x)\mathbf{d}_{i,t}/\mathrm{f}_x$, $\mathbf{y}_{i,t} = (\mathbf{v}_{i,t} - \mathrm{c}_y)\mathbf{d}_{i,t}/\mathrm{f}_y$ and $\mathbf{z}_{i,t} = \mathbf{d}_{i,t}$, as shown in Figure 3.5 (a). To accurately capture the uncertainties associated with these measurements, the main diagonal of the covariance matrix is formulated as:

$$
\begin{aligned}
\sigma_{x_{i,t}}^2 &= \left(\sigma_{u_{i,t}}^2\sigma_{d_{i,t}}^2 + \sigma_{u_{i,t}}^2 d_{i,t}^2 + (u_{i,t} - c_x)^2\sigma_{d_{i,t}}^2\right)/\mathrm{f}_x^2, \\
\sigma_{y_{i,t}}^2 &= \left(\sigma_{v_{i,t}}^2\sigma_{d_{i,t}}^2 + \sigma_{v_{i,t}}^2 d_{i,t}^2 + (v_{i,t} - c_y)^2\sigma_{d_{i,t}}^2\right)/\mathrm{f}_y^2, \\
\sigma_{z_{i,t}}^2 &= \sigma_{d_{i,t}}^2.
\end{aligned}
\tag{3.2}
$$

In this model, the projected coordinates are interdependent due to the common multiplier of depth $\mathbf{d}_{i,t}$. To precisely formulate the covariance of the 3D keypoints ${}^c\Sigma_{i,t}^p$ under camera coordinate, it is essential to include the off-diagonal covariance terms in ${}^c\Sigma_{i,t}^p$.

$$
{}^c\Sigma_{i,t}^p = \begin{bmatrix} \sigma_z^2 & \sigma_{xz_{i,t}} & \sigma_{yz_{i,t}} \\ \sigma_{xz_{i,t}} & \sigma_{x_{i,t}}^2 & \sigma_{xy_{i,t}} \\ \sigma_{yz_{i,t}} & \sigma_{xy_{i,t}} & \sigma_{y_{i,t}}^2 \end{bmatrix}
\begin{aligned}
\sigma_{xz} &= \sigma_d^2(u - \mathrm{c}_x)/\mathrm{f}_x \\
\sigma_{yz} &= \sigma_d^2(v - \mathrm{c}_y)/\mathrm{f}_y \\
\sigma_{xy} &= \frac{\sigma_d^2(u - \mathrm{c}_x)(v - \mathrm{c}_y)}{\mathrm{f}_x\mathrm{f}_y}.
\end{aligned}
\tag{3.3}
$$

Results in the ablation study also confirm the necessity of off-diagonal terms for accurate pose graph optimization. Detailed derivation for Eq. 3.2 and Eq. 3.3 is in Sec. **??**.

Uncertainty Correction after Keypoint Matching As shown in Figure 3.5 (b), the matched features are expected to fall in a probabilistic range centered at $[u_{i,t}, v_{i,t}]$ with the flow uncertainty $(\sigma^2_{u_{i,t}}, \sigma^2_{v_{i,t}})$. However, the depth observations of matched keypoints may vary due to scene geometry. For example, if a matched keypoint is located near the edge of an object, such as a wall, even a minor perturbation in feature matching can lead to substantial discrepancies in 3D keypoint registration. This, in turn, amplifies the uncertainty in the reconstructed 3D structure.

To address this problem, we correct the depth uncertainty $\hat{\sigma}^2_{d_{i,t}}$ of the matched feature point based on the depth feature of the local patch $D_{i,t}$ with the kernel size of 32. We approximate it with the weighted sum of the variances within the patch. The weights are determined by a 2D Gaussian kernel φ, which utilizes $\sigma^2_{u_{i,t}}$ and $\sigma^2_{v_{i,t}}$ to adjust the influence of each point within the patch: $\sigma^2_{d_{i,t}} = \sum_j \varphi_j (d_j - \mu_{D_{i,t}})^2$.

3.3.4 Pose Graph Optimization

We optimize the camera pose $T_t \in SE(3)$ at time t in the world frame by minimizing the distance of the matched 3D keypoints $p_{i,t-1}$ and ${}^c p_{i,t}$, where ${}^c p_{i,t}$ is in the camera frame. To reduce the initial error margin of the optimization, we initialize the camera pose using the relative motion estimated by the TartanVO [21].

The pose graph optimization is formulated as follows:

$$\begin{aligned} T^\star &= \arg\min_{T_t} \sum_i \| \mathbf{p}_{i,t-1} - T_t \, {}^c\mathbf{p}_{i,t} \|^2_{\Sigma_i}, \\ \Sigma_i &= \Sigma^p_{i,t-1} + R_t \, {}^c\Sigma^p_{i,t} \, R_t^\top. \end{aligned} \tag{3.4}$$

$\| \cdot \|_{\Sigma_i}$ represents the Mahalanobis distance with covariance matrix Σ_i. Unlike DROID-SLAM [12], which employs a diagonal covariance matrix, we model the correlation between axes to capture accurate inter-dependencies. We solve this problem using Levenberg-Marquardt optimization via *PyPose* [63, 64].

3.4 Experiment

Datasets & Baseline We evaluate the proposed model and baseline methods on a variety of public datasets, including synthetic dataset TartanAir v2 [62], real-world data from EuRoC [65], KITTI [66], as well as customized data collected from a Zed-X camera. These datasets cover a diverse range of hardware configurations, motion patterns, and environments. To demonstrate our method's robustness under challenging scenarios, we collected the TartanAir v2, a new set of difficult trajectories following the TartanAir [62] that includes frequent indoor-outdoor transition and low-illumination scenes as shown in Figure 3.6. To demonstrate the generalizability of our model, we use the same configuration and same matching model across all datasets.

Evaluation Metrics Since the proposed method does not contain loop closure or global bundle adjustment, our evaluation focuses on relative error. We use relative translation error

(t_{rel}, m/frame) and rotation error (r_{rel}, °/frame) as:

$$
\begin{aligned}
t_{\text{rel}} &= \frac{1}{N}\sum_{t=1}^{N}\left\|\mathbf{p}_{t+1}-\mathbf{p}_t-R_t\hat{R}_t^{\top}(\hat{\mathbf{p}}_{t+1}-\hat{\mathbf{p}}_t)\right\|_2, \\
r_{\text{rel}} &= \frac{180}{\pi}\frac{1}{N}\sum_{t=1}^{N}\left\|\log\left(\hat{R}_{t,t+1}^{\top}R_{t,t+1}\right)\right\|_2,
\end{aligned}
\tag{3.5}
$$

where $\mathbf{p}_t$ and R_t are ground truth position and rotation, $\hat{\mathbf{p}}_t$ and $\hat{R}_t$ is the estimated position and rotation. $R_{t,t+1} = R_t^{\top}R_{t+1}$ is the rotation from frame t to frame $t+1$.

3.4.1 Quantitative Analysis

Table 3.1: Performance comparison of different methods on the EuRoC Dataset. Only odd-ordered trajectories are shown due to page limit. Average is calculated over all EuRoC trajectories.

Trajectory	MH01		MH03		MH05		V102		V201		V203		Avg.	
	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
SLAM														
🎞 ORB-SLAM 3	0.0035	0.0450	0.0058	0.0603	0.0059	0.0526	0.0096	0.1757	0.0064	0.1615	0.033	0.9497	0.0092	0.1866
🎞 DROID-SLAM*	**0.0012**	**0.0159**	0.0034	0.2656	**0.0025**	**0.0193**	**0.0026**	**0.0417**	**0.0012**	**0.0289**	**0.0034**	**0.1033**	**0.0024**	0.0590
🎞 MASt3R-SLAM*	0.0143	0.5493	0.0493	0.8890	0.0254	0.5808	0.0354	1.3178	0.0117	0.6818	0.0430	2.3306	0.0277	1.0472
VO														
🎞 TartanVO*	0.0121	0.0560	0.0302	0.2791	0.0193	0.0604	0.0251	0.1244	0.0065	0.0920	0.0303	0.2986	0.0198	0.1270
🎞 TartanVO	0.0277	0.5122	0.0514	0.6635	0.0464	0.4797	0.0394	1.0420	0.0195	0.4684	0.0473	1.9657	0.0368	0.8346
🎞 iSLAM-VO	0.0042	0.0560	0.0076	0.2789	0.0070	0.0603	0.0066	0.1241	0.004	0.0920	0.0151	0.2984	0.0071	0.1269
🎞 DPVO*	0.0015	0.0207	0.0028	0.0273	0.0028	0.0243	0.0041	0.0496	0.0016	0.0342	0.0045	0.1205	0.0027	0.0437
🎞 **Ours**	0.0014	0.0214	**0.0023**	**0.0238**	**0.0025**	0.0216	0.0029	0.0434	**0.0012**	**0.0289**	0.0049	0.1284	**0.0024**	**0.0403**

🎞 Monocular method. 🎞 Stereo method. * Scale-aligned with ground truth.

EuRoC Dataset We assessed our model on the EuRoC [65] dataset, as detailed in Table 3.1, comparing it against baseline methods including visual odometry and state-of-the-art visual SLAM systems with loop-closure and global bundle adjustment. While our method exhibits compatible performance to DROID-SLAM on average t_{rel}, it outperforms all baselines in terms of r_{rel} by around 10%.

Table 3.2: Performance comparison on the TartanAir v2 Hard Dataset. ORB-SLAM3 and MASt3R-SLAM lose track on all sequences and are therefore not included. Average is calculated over all TartanAir v2 Hard trajectories.

Trajectory	H00		H01		H02		H03		H04		H05		H06		Avg.	
	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
SLAM																
🎞 DROID-SLAM*	.0485	.1174	**.0023**	**.0210**	.0190	.0821	**.0064**	.0300	**.0057**	**.0255**	.1463	.2357	.0310	**.0908**	.0370	**.0861**
VO																
🎞 TartanVO*	.1605	3.338	.2918	2.775	.2718	3.305	.2775	2.191	.2204	2.874	.1644	2.899	.2350	3.756	.2316	3.020
🎞 TartanVO-stereo	.1505	.4329	.0914	.7542	.0715	.3265	.0842	.4053	.0678	.6569	.0803	1.186	.0784	.9458	.0892	.6725
🎞 iSLAM-VO	.4235	2.630	.3070	3.018	.3252	2.189	.3622	2.435	.2576	2.899	.2574	3.755	.2099	3.145	.3061	2.867
🎞 DPVO*	.4984	.4937	.1738	.7112	.0539	.2539	.3847	2.703	.0481	.1869	.1891	1.430	.3365	2.943	.2406	1.246
🎞 **Ours**	**.0085**	**.1018**	.0344	.1450	**.0048**	**.0628**	.0150	.0778	.0092	.1414	**.0048**	**.0552**	**.0217**	.4160	**.0141**	.1429

🎞 Monocular method. 🎞 Stereo method. * Scale-aligned with ground truth.

Table 3.3: Performance comparison of different methods on the KITTI Dataset. Only even-numbered trajectories are shown due to page limit. Average is calculated over all KITTI odometry trajectories.

Trajectory	00		02		04		06		08		10		Avg.	
	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
SLAM														
[MM] ORB-SLAM 3	0.0252	0.0586	0.0438	0.0529	0.0274	0.0322	0.0228	0.0338	0.0271	0.0455	**0.0166**	0.0515	**0.0258**	0.0434
[MM] DROID-SLAM*	0.0198	0.0538	0.0250	0.0445	0.0255	0.0296	0.0199	0.0296	0.0275	0.0381	0.0309	0.0715	0.0900	0.0448
[M] MASt3R-SLAM*	0.8988	0.7154	1.0104	0.5959	1.2266	0.1124	1.2599	0.1289	0.7760	0.6075	0.6889	0.4709	0.9538	0.4202
VO														
[M] TartanVO*	0.2066	0.1055	0.1626	0.1105	0.1152	0.0789	0.2234	0.0816	0.1857	0.0823	0.1745	0.0907	0.2207	0.0886
[MM] TartanVO	0.0656	0.1026	0.0905	0.1197	0.1747	0.1158	0.0923	0.0968	0.0721	0.1063	0.0679	0.0969	0.1804	0.1147
[MM] iSLAM-VO	0.0577	0.1052	0.0686	0.1101	0.1356	0.0787	0.0837	0.0812	0.0510	0.082	0.0449	0.0905	0.0878	0.0883
[M] DPVO*	0.4542	**0.0495**	0.4209	**0.0381**	0.0348	**0.0219**	0.2393	**0.0250**	0.3051	**0.0347**	0.0661	**0.0386**	0.1951	**0.0329**
[MM] **Ours**	**0.0192**	0.0654	**0.0223**	0.0715	**0.0206**	0.0473	**0.0187**	0.0456	**0.0254**	0.0509	0.019	0.0569	0.0420	0.0645

[M] Monocular method. [MM] Stereo method. * Scale-aligned with ground truth.

TartanAir v2 Dataset TartanAir v2 presents significant challenges for visual odometry due to its rigorous motion and extreme illumination changes. The traditional method, ORB-SLAM, fails on most trajectories. Our approach improves 61.9% in t_{rel} compared to DROID-SLAM. Notably, on trajectory H00, which simulates the lunar surface shown in Figure 3.6, our model demonstrates a remarkable 82.4% decrease in t_{rel} and achieves the lowest r_{rel} among all baseline methods.

KITTI Dataset To further validate our robustness and consistency in outdoor, large-scale trajectories with dynamic objects, we evaluate our system on the KITTI [66] dataset. Our method, which relies solely on two-frame pose optimization, shows commendable performance, ranking behind ORB-SLAM3, a full visual SLAM system, on t_{rel}. Our system significantly outperforms other visual odometry approaches in t_{rel}, demonstrating a 53.3% reduction in relative translation error. The degraded performance of r_{rel} may be attributed to the lack of multi-frame optimization.

3.4.2 Qualitative Analysis and Ablation Study

In addition to quantitative evaluations, we conduct qualitative evaluations of our proposed system across multiple datasets including EuRoC, KITTI, and TartanAir v2, supplemented by manually collected data using a ZED-X stereo camera. MASt3R-SLAM struggles on many trajectories of KITTI and TartanAir, primarily due to incorrect loop closure detections from visual ambiguities. Our model, even without multi-frame optimization, achieves top-tier performance and exhibits fewer glitches than baseline methods. As demonstrated in the Figure 3.6, our method produces smoother trajectories and superior pose estimation precision. Figure 3.6 a) presents that our model correctly identifies the region occluded by the mounted platform and dynamic objects in the scene and assigns a high uncertainty score to these regions.

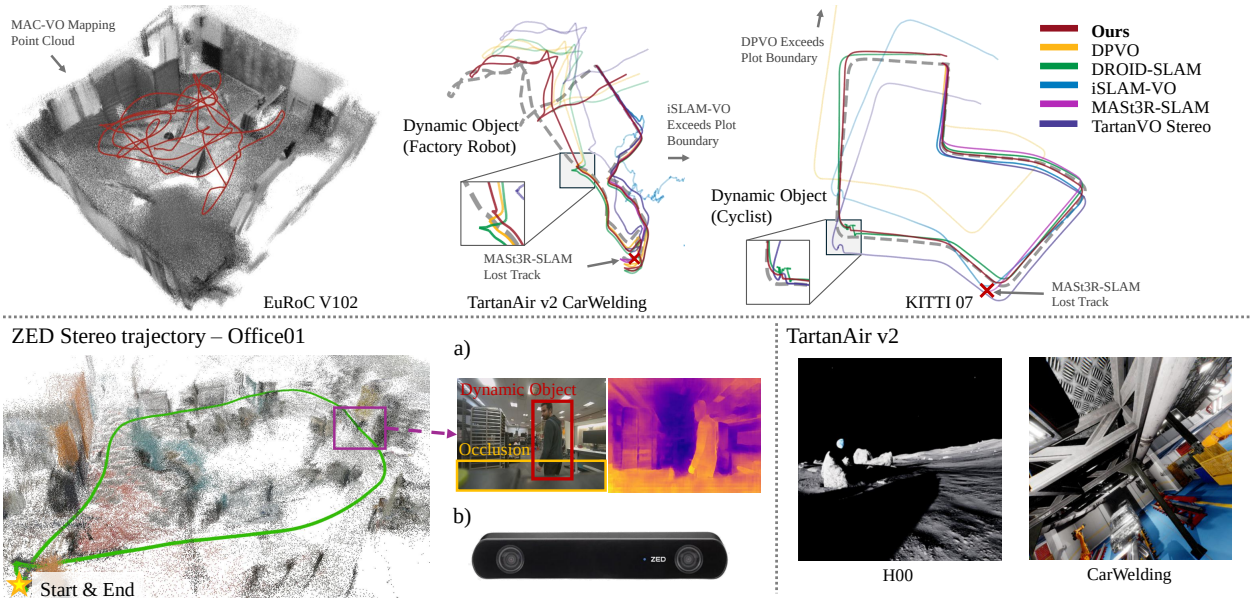

Figure 3.6: Top: Trajectories estimated by our model and baselines. We highlight the segments where dynamic objects interrupt the VO. **Bottom left:** We collect data in the office using our own payload with the ZED-X camera. The data contains multiple dynamic objects and visual occlusions. **Bottom right:** samples of TartanAir v2 test dataset, which simulates the exotic lunar environment.

Table 3.4: Ablation study.

Dataset	TartanAir v2 Hard		TartanAir v2 Easy	
	t_{rel}	r_{rel}	t_{rel}	r_{rel}
Module Ablation				
w/o CovKP & CovOpt	.0743	.5221	.0521	.2799
w/o CovOpt	.0679	.3776	.0511	.2367
w/o CovKP	.0188	.2347	.0066	.0808
Covariance Ablation				
DiagCov	.0461	.3023	.0277	.1548
Scale Agnostic	.0204	.2321	.0086	.0764
Ours	**.0141**	**.1429**	**.0051**	**.0670**

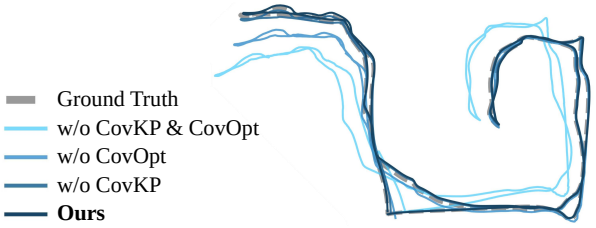

Figure 3.7: Results of various ablation setups on the H01 of TartanAirv2.

Ablation Study In Table 3.4, we first perform the ablation study on each module of MAC-VO including (a. *w/o CovKP & CovOPT*) with random keypoint selector and identity covariance matrix. (b. *w/o CovOpt*) replace the metrics-aware covariance model with the identity covariance model. (c. *w/o CovKP*) replace the proposed keypoint selector with a random keypoint selector.

Table 3.5: Time spent on each module of MAC-VO under different optimization with image resolution of 640×640.

	Raw	TRT*	MP† + TRT*	MAC-VO Fast‡
Modules (ms)				
Frontend Network	401.9	239.3	239.9	81.6
Optimization	53.4	57.5	-	-
Motion Model	5.2	5.1	5.3	5.2
Keypoint Selector	0.7	0.6	0.6	0.5
Covariance Model	0.6	0.7	0.7	0.7
Overall (fps)	2.15	3.25	3.96	**10.57**

* TRT: TensorRT framework - https://developer.nvidia.com/tensorrt

† MP: multi-processing the PGO in parallel with the matching network.

‡ MAC-VO Fast: utilizes half-precision number (`bf16`) and light-weight model.

To demonstrate the necessity of scale consistency and off-diagonal terms in our covariance model, we run the ablation study with different configurations: (I. *DiagCov*) remove off-diagonal terms in Eq. 3.3, (II. *Scale-agnostic*) normalize the covariance model by the average determinants of covariance matrices of each frame.

3.4.3 Runtime Analysis

The runtime analysis shown in Table 3.5 uses the platform with `AMD Ryzen 9 5950X` CPU and `NVIDIA 3090 Ti` GPU. We also introduce a fast mode (MAC-VO Fast) that utilizes half-precision number in the network inference to enhance efficiency. This mode also speeds up the memory decoder network by reducing the number of iterative updates from 12 to 4. The fast mode performs at 10.5 fps (frames per second) with 70% of the performance of the original MAC-VO.

3.5 Network and Training Details

In the uncertainty-aware matching network, we employ a flow decoder similar to Flowformer to obtain Δf. In each iteration, the flow f_x is updated by $f_x \leftarrow f_x + \delta f$. With the shared features, we finetune a new decoder with `ConvGRU` layers to estimate the uncertainty updates $\Delta\sigma$. In each iteration, the matching uncertainty is updated by $\sigma \leftarrow \sigma \times \Delta\sigma$.

For each pixel x, we extract the local cost-map patch q_x from the cost map, and the context feature t_x from the context encoder. We encode the q_x using a transformer FFN. Given the 2D position $p = x + f_x$, we encode it into positional embedding $\mathrm{PE}(p)$. We aggregate these features using CNN encoder ME to generate local motion features I_x. Utilizing the GMA module, the network's global motion features G_x are acquired from the current motion features $\mathrm{Att}_f(t_x)$ and I_x. Subsequently, the shared motion features m_x is obtained by concatenating the G_x, I_x and $\mathrm{Att}(t_x)$, where $\mathrm{Att}(t_x)$ is the attention of the context features. To estimate the flow update Δf and $\Delta\Sigma$, we use the ConvGRU $\Sigma_\pi(m_x)$:

Table 3.6: The GPU memory consumption and the number of parameters during testing.

Methods	GPU memory	Parameters
MAC-VO	4.20GB	19.2M
TartanVO-stereo	1.16GB	47.3M
DROID-SLAM	28.34GB	4.0M
DPVO	1.51GB	3.4M

$$
\begin{aligned}
I_x &= \mathrm{ME}(f_x, \mathrm{Concat}(\mathrm{FFN}(q_x), \mathrm{PE}(p))) \\
G_x &= \mathrm{GMA}(\mathrm{Att}(t_x), I_x) \\
m_x &= \mathrm{Concat}(\mathrm{Att}(t_x), I_x, G_x) \\
\Delta f &= \mathrm{FlowGRU}(m_x) \\
\Delta \Sigma &= \mathrm{CovGRU}(m_x)
\end{aligned}
\tag{3.6}
$$

We use `AdamW` optimizer with a learning rate of 12.5×10^{-5}. The model is trained on A100 GPU consuming 16 GB GPU memory.

3.5.1 GPU Memory & Parameters

As shown in Table 3.6, MAC-VO uses 4.20GB of GPU memory, which is 6.7 times smaller than DROID-SLAM. This is because we utilize sparse features for back-end optimization, reducing the requirements for GPU memory.

3.6 Conclusion and Discussion

This chapter presents MAC-VO, a learning-based stereo visual odometry method that outperforms most visual odometry and even SLAM algorithms on challenging datasets by jointly learning metrics-aware visual uncertainty alongside pose estimation. The key innovations are a metrics-aware 2D uncertainty network that captures matching inconsistencies and a 3D covariance model that propagates these uncertainties into pose graph optimization with full inter-axis correlations. The ablation study confirms the necessity of both scale consistency and off-diagonal terms for accurate pose estimation.

In the current formulation, MAC-VO focuses on two-frame pose optimization. Accuracy can be further improved with multi-frame optimization, bundle adjustment, and loop closure, which are addressed in the full SLAM system presented in Chapter 10. The metrics-aware covariance model also provides the foundation for multi-sensor fusion with IMUs, as developed in Part III of this thesis.

Chapter 4

QuantMAC-VO: Quantization for Learning-Based Visual Odometry with Metrics-Aware Covariance

4.1 Introduction

Recently, learning-based Visual Odometry (VO) [43, 67–70] has emerged as a robust paradigm for state estimation in robotics, outperforming traditional geometric methods in challenging dynamic and texture-poor environments [20, 71, 72]. However, this performance gain comes with heavy computational and memory demands, making such models prohibitive for deployment on resource-constrained edge platforms.

Model quantization [73, 74], particularly Post-Training Quantization (PTQ), has become the *de facto* standard for efficient deployment of deep neural networks [1, 75–77]. However, most PTQ methods rely on a small calibration dataset and implicitly assume that its activation statistics are representative of deployment conditions. This assumption breaks down for learning-based VO, where long-horizon motion dynamics, viewpoint changes, and scene-dependent geometry induce highly non-stationary and heavy-tailed activations. As a result, quantization parameters calibrated on limited sequences tend to overfit to specific environments and fail to generalize, leading to significant performance degradation when deployed on robots operating in previously unseen real-world scenarios.

In addition, learning-based VO networks exhibit pronounced heavy-tailed activations and spatially localized outliers as shown in Figure 4.3, which significantly hinder effective quantization. To address this, Hadamard rotation [78] has been proposed to disperse outliers across dimensions via an orthogonal transform, thereby improving quantization robustness. However, this strategy introduces new challenges in the context of learning-based odometry. VO architectures typically consist of many layers with relatively small feature dimensions, for which the computational and memory overhead of applying Hadamard rotations becomes non-negligible. When deployed on edge platforms, the added cost of these rotations can dominate inference time, and in some cases, even result in slower execution than the unquantized model.

To address these challenges, this chapter presents a full-stack quantization framework for

Figure 4.1: QuantMAC-VO deployed on the NVIDIA Jetson Thor platform, achieving real-time learning-based visual odometry with metrics-aware covariance on edge hardware.

learning-based VO with metrics-aware covariance, built upon MAC-VO (Chapter 3) and a foundation calibration dataset grounded in TartanAir [79]. Inspired by the strong sim-to-real transferability of visual foundation models and modern learning-based VO systems [70], we leverage a large-scale, diverse calibration source to improve quantization robustness across environments. To mitigate the computational overhead introduced by rotation-based quantization, we propose a kurtosis-based hybrid quantization strategy that selectively applies computationally expensive online rotations only to critical layers exhibiting spatial outliers, while employing efficient offline channel smoothing for channel-wise outliers. Beyond frontend acceleration, we further investigate quantization effects in the backend pose optimization. We find that applying the scaling factor to the Gauss-Newton optimizer is mathematically equivalent to scaling the coordinate of the pose estimation. Therefore, our quantization on the back-end optimizer maintains consistent performance. As shown in Figure 4.1, we deployed the proposed system on the NVIDIA Jetson Thor edge platform, demonstrating real-time performance with negligible accuracy loss on resource-constrained robotic hardware. The main contributions of this chapter are:

- We introduce a sim-to-real quantization calibration strategy enabling zero-shot generalization without re-calibration, and observe a scaling law whereby real-world performance improves monotonically with the diversity of synthetic calibration data.
- We propose a kurtosis-based hybrid quantization method that selectively applies Hadamard rotations to layers with high kurtosis, reducing system overhead while maintaining high quantization efficiency.
- We investigate backend optimization under quantization by introducing explicit scaling factors, and show that the resulting formulation is mathematically equivalent to solving pose estimation under a scaled coordinate system.
- We demonstrate that the proposed quantization framework accelerates MAC-VO by 1.83× on the NVIDIA Jetson Thor platform, achieving real-time performance with negligible accuracy degradation.

4.2 Related Work

4.2.1 Quantization for Neural Networks

Existing post-training quantization (PTQ) methods are categorized into weight-only and weight-activation quantization [80]. Weight-only methods are commonly used for LLMs, like GPTQ [77] and AWQ [81], which achieve high compression rates but offer limited inference acceleration. In contrast, weight-activation quantization methods [1, 82] quantize both weights and activations, improving memory usage and latency. The main challenge is activation outliers causing quantization errors. Techniques like SmoothQuant [1] shift quantization difficulty from activations to weights, while OmniQuant [83] optimizes performance by training quantization parameters. Recently, Quarot [78] introduced rotations to eliminate outliers; however, this solution is not applicable to MLLMs due to inherent modality differences.

4.2.2 Quantization for SLAM

Traditional geometric visual odometry systems primarily rely on minimizing reprojection or photometric errors [20, 33], typically employing fixed covariance heuristics that often falter in complex environmental dynamics. The advent of deep learning has fundamentally reshaped this landscape, delivering significant performance leaps in core components such as optical flow estimation [39, 40], dense depth prediction [42, 43, 84, 85], and learnable feature matching [28, 41].

Building upon these advancements, hybrid SLAM architectures have emerged to synergize the robustness of geometric constraints with the adaptability of data-driven priors [12, 49]. A critical innovation in these modern frameworks is the integration of learned uncertainty. Unlike earlier methods that utilized static empirical priors [35], recent approaches employ neural networks to predict pixel-wise confidence or depth covariance [46, 48], which serve as dynamic weights during factor graph optimization or differentiable bundle adjustment [12, 54]. Methods like QVIO [86, 87] focus on quantizing raw sensor measurements or residuals. However, many existing systems, such as DROID-SLAM [12], often resort to simplified diagonal covariance models, potentially neglecting scale consistency which is vital for rigorous sensor fusion. Based on this, SPAQ-DL-SLAM [88] compresses DROID-SLAM by applying pruning and quantization strictly to the neural modules. However, they leave the backend untouched in high precision. Furthermore, the strategy for keypoint selection has evolved. While classical descriptors [26] and early learned extractors [25] predominantly target high-frequency regions like edges and corners, recent studies suggest that these areas are prone to interpolation artifacts and neural ambiguity, potentially degrading state estimation [22, 49]. Consequently, state-of-the-art frameworks (e.g., MAC-VO, Chapter 3) utilize learned uncertainty maps to actively filter unreliable features, ensuring that only high-quality constraints enter the backend optimization. The increasing reliance on these sophisticated, heavy-weight uncertainty estimation networks underscores the necessity for efficient quantization strategies to facilitate their deployment on edge devices.

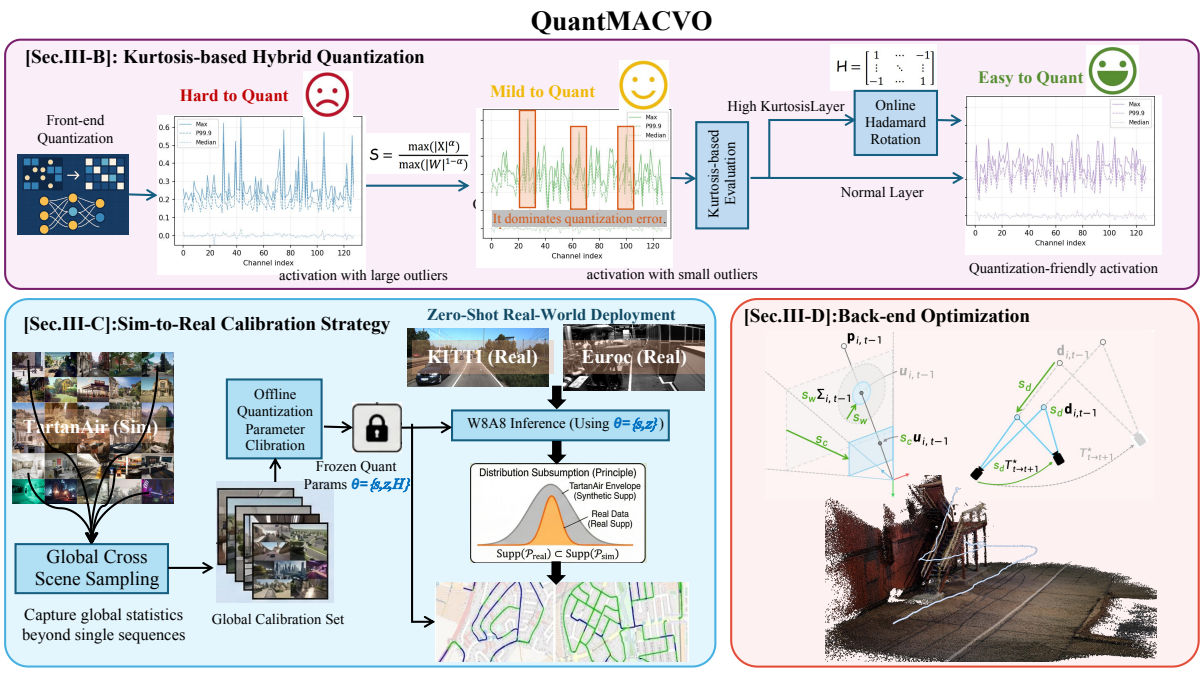

Figure 4.2: Overview of the QuantMAC-VO framework. (a) Kurtosis-based Hybrid Quantization: Layers with high kurtosis are identified via online Hadamard Rotation to suppress activation outliers. **(b) Sim-to-Real Calibration Strategy:** We construct a global calibration set from synthetic data. We derive frozen parameters (θ) that envelop real-world statistics, enabling zero-shot deployment on KITTI and EuRoC. **(c) Back-end Optimization:** The robustly quantized features are used for precise trajectory estimation with our quantized back-end design.

4.3 Method

4.3.1 Preliminaries

MAC-VO [70] is a stereo VO system that combines a learning-based *front-end* for feature matching with an optimization-based *back-end*. The front-end produces geometry cues to lift image measurements into 3D through disparity (or depth), and temporal correspondences through optical flow. It also predicts the corresponding uncertainty for the feature matching in the keypoint selection and back-end optimization.

Frontend (Disparity & Flow). The front-end network $\mathcal{F}$ takes a rectified stereo pair $(I_{L,t}, I_{R,t})$ and a temporal left-image pair $(I_{L,t-1}, I_{L,t})$ as input. It outputs a disparity map $\mathbf{s}_t$ for stereo triangulation and an optical flow field $\mathbf{f}_{t-1\to t}$ for temporal association, together with pixel-wise uncertainty estimates:

$$\begin{aligned} (\mathbf{s}_t,\, \sigma_{s,t}^2) &= \mathcal{F}(I_{L,t}, I_{R,t}), \\ (\mathbf{f}_{t-1\to t},\, \mathbf{\Sigma}_{\mathbf{f},t}) &= \mathcal{F}(I_{L,t-1}, I_{L,t}), \qquad \mathbf{\Sigma}_{\mathbf{f},t} \triangleq \begin{bmatrix} \sigma_{u,t}^2 & 0 \\ 0 & \sigma_{v,t}^2 \end{bmatrix}. \end{aligned} \tag{4.1}$$

For a selected set of keypoints $\mathcal{K}$ in the left image at time $t-1$, the disparity $\mathbf{s}_{t-1}$ provides their 3D coordinates $\mathbf{p}_{k,t-1} \in \mathbb{R}^3$ via stereo triangulation. Let $\mathbf{u}_{k,t} \in \mathbb{R}^2$ denote the 2D image

coordinates of keypoint k at time t; the flow prediction induces the correspondence $\mathbf{u}_{k,t} = \mathbf{u}_{k,t-1} + \mathbf{f}_{k,t-1\to t}$.

Backend (Pose Optimization). Given the 3D–2D correspondences $\{(\mathbf{p}_{k,t-1}, \mathbf{u}_{k,t})\}_{k\in\mathcal{K}}$, the back-end estimates the relative pose $\mathbf{T}_{t,t-1} \in SE(3)$ from frame $t-1$ to t by minimizing a Mahalanobis-weighted reprojection error:

$$\mathbf{T}^{\star}_{t,t-1} = \underset{\mathbf{T}\in SE(3)}{\arg\min} \sum_{k\in\mathcal{K}} \left\| \pi\big(\mathbf{T}\,\mathbf{p}_{k,t-1}, K\big) - \mathbf{u}_{k,t} \right\|^2_{\mathbf{\Sigma}_{\mathbf{f},k,t}}, \tag{4.2}$$

where $\pi(\,\cdot\,, K)$ is the pinhole projection with intrinsics K, and $\mathbf{\Sigma}_{\mathbf{f},k,t}$ is the flow covariance predicted by $\mathcal{F}$ at keypoint k.

Post-Training Quantization. Quantization [73, 74] converts model weights and activations from floating-point (FP) representations into low-bit integer formats, thereby reducing both computational cost and memory footprint. In uniform affine quantization, FP values are mapped to a fixed integer range using three key parameters: the scale factor s, zero-point z, and bit-width b. Given an FP input $\mathbf{x}$ (weights or activations), the quantization and de-quantization process can be described by Eq. 4.3, where $\lfloor\cdot\rceil$ denotes the round-to-nearest operation (RTN), and clamp$(\cdot)$ restricts the result to the valid integer:

$$\mathbf{x}_{\text{int}} = \text{clamp}\left(\left\lfloor \frac{\mathbf{x}}{s} \right\rceil + z;\, 0,\, 2^b - 1\right), \widehat{\mathbf{x}} = s\,(\mathbf{x}_{\text{int}} - z)\,. \tag{4.3}$$

where $\mathbf{x}_{\text{int}}$ represents the quantized integer value, $\hat{x}$ is the de-quantized FP value with an error that is introduced during the quantization process. Based on Eq. 4.3, the FP range of the quantization grid is defined as $[q_{\min}, q_{\max}] = [-sz,\, s(2^b - 1 - z)]$. Input values $\mathbf{x}$ falling outside this range are clipped to the nearest boundary, resulting in clipping error. Increasing the scale factor s can reduce clipping by expanding the representable range, but at the cost of increased rounding error, which is bounded within $\left[-\frac{1}{2}s,\, \frac{1}{2}s\right]$. To balance these two sources of error, modern quantization frameworks commonly adopt MSE-based calibration [89], which searches for the optimal $(q_{\min}, q_{\max})$ that minimizes the Mean Squared Error (MSE) between the original tensor $\mathbf{x}$ and its de-quantized approximation $\widehat{\mathbf{x}}$.

Rotation and Hadamard Matrices. Recent studies [78, 90] have demonstrated that applying orthogonal rotations to the feature space can effectively eliminate activation outliers by redistributing feature magnitudes, yielding a more quantization-friendly distribution. In this chapter, we employ rotation matrices, defined as real orthogonal matrices $\mathbf{R}$ satisfying $\mathbf{R}\mathbf{R}^\top = \mathbf{I}$. To ensure computational efficiency during quantization, we specifically utilize the Walsh-Hadamard matrix $\mathbf{H}_d$ as the rotation operator. For a dimension $d = 2^k$, $\mathbf{H}_d$ is constructed recursively using the Kronecker product $\otimes$:

$$\mathbf{H}_{2^k} = \mathbf{H}_2 \otimes \mathbf{H}_{2^{k-1}}, \quad \text{with} \quad \mathbf{H}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}. \tag{4.4}$$

This recursive structure enables fast matrix-vector multiplication in $\mathcal{O}(d \log_2 d)$ complexity. To further enhance the distribution flatness, we adopt the randomized Hadamard rotation $\tilde{\mathbf{H}} = \mathbf{H}_d\,\text{diag}(\mathbf{s})$, where $\mathbf{s} \in \{-1, +1\}^d$ represents a random sign vector. It is straightforward to verify that $\tilde{\mathbf{H}}$ maintains orthogonality.

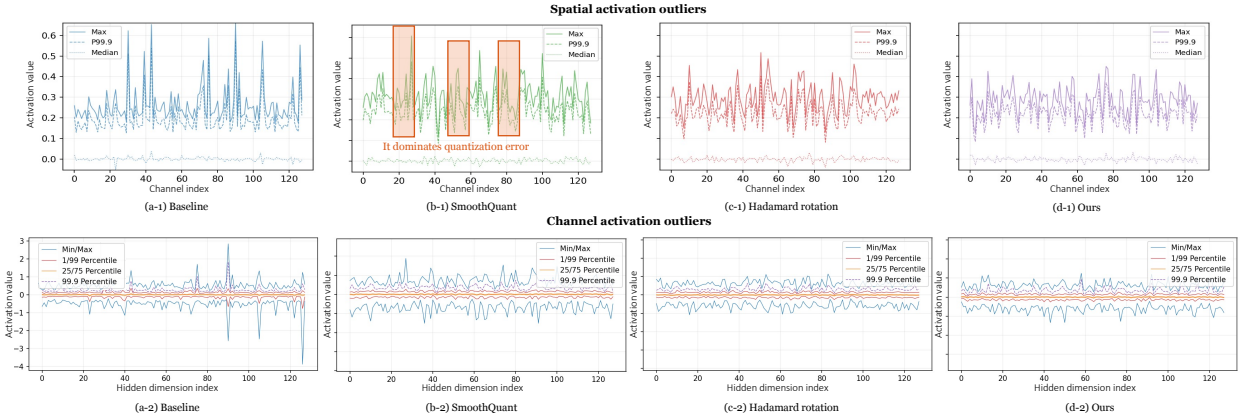

Figure 4.3: Channel and spatial outlier visualization for the conv3 activations. SmoothQuant [1] reduces channel-wise outliers (b-2) but leaves spatially localized outliers (three hotspots in b-1). In contrast, applying a Hadamard rotation to the input activations disperses outliers across positions and channels (c-1, c-2). Our method selectively applies Hadamard rotation to outlier-dominated layers, effectively mitigating both channel- and spatial-outliers (d-1, d-2).

4.3.2 Kurtosis-based Hybrid Quantization

As shown in Figure 4.3, we employ the Hadamard matrix to address both channel outliers and spatial outliers in the activations. While effective, applying Hadamard matrices introduces additional computational overhead during quantization and inference. To balance quantization quality and inference latency, we propose a hybrid quantization strategy. Specifically, Hadamard rotation is applied only to a subset of critical layers where outliers dominate the quantization error, whereas the remaining layers adopt lightweight, overhead-free channel scaling [1].

To determine the importance of different layers, we adopt kurtosis as a selection criterion. Kurtosis serves as a proxy metric for identifying layers that are sensitive to outliers. Intuitively, high-kurtosis activations are dominated by a small number of extreme values, which distort the quantization grid and amplify rounding error for the remaining activations. Compared to prior methods that rely on expensive layer-wise sensitivity profiling [91], kurtosis-based analysis avoids redundant per-layer quantization and repeated forward passes. In Eq. 4.5, we compute the mean μ_ℓ and variance σ_ℓ^2 over the input activations of each layer ℓ across the analysis data, and define the layer-wise kurtosis K_ℓ as the standardized fourth moment from the features of each layer X_ℓ:

$$K_\ell = \frac{\mathbb{E}[(X_\ell - \mu_\ell)^4]}{\sigma_\ell^4}. \tag{4.5}$$

Our kurtosis-based hybrid quantization proceeds in three steps. (1) Kurtosis analysis: We run the full-precision model on an analysis set, compute kurtosis from the input activation distribution of each layer l, and normalize it to obtain a layer score K_l in Eq. 4.6. (2) Critical layers decision: As shown in Figure 4.4, we choose the optimal point using the Kneedle criterion [92]. The point that maximizes the difference between normalized utility and normalized cost. We apply Hadamard rotation only up to the knee point, avoiding fixed top-k choices or heuristic thresholds. (3) PTQ calibration: After fixing the layer-wise quantization strategy, we perform our sim-to-real quantization calibration to update quantization parameters.

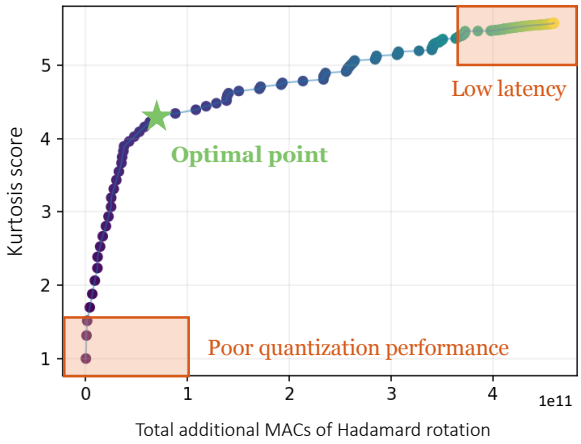

Figure 4.4: The x-axis shows cumulative additional Multiply–Accumulate operations (MACs) of Hadamard rotation and the y-axis shows the cumulation of K_{norm} up to the i-th layer.

$$\mathrm{K}_{\mathrm{norm}}(\ell) = \begin{cases} \dfrac{K_\ell - K_{\min}}{\Delta K} & \text{if } \Delta K > 0, \\ 0 & \text{otherwise.} \end{cases} \tag{4.6}$$

4.3.3 Sim-to-Real Quantization Calibration

Standard Post-Training Quantization (PTQ) algorithms [75, 89, 93] typically rely on a small set of calibration data to determine the quantization parameters (scale factor s and zero-points z). The prevailing wisdom suggests that this calibration set should be sampled directly from the target domain to minimize the distribution mismatch. However, in robotic applications, this requirement is often impractical. Environments are inherently dynamic and unstructured, making it infeasible to re-calibrate the model for every specific scene transition. Consequently, calibrating on a limited real-world sequence often leads to *overfitting*, where narrow clipping ranges fail to accommodate unseen dynamic motions or lighting extremes, causing significant accuracy degradation or even system collapse during deployment. To address this, we propose a Sim-to-Real Quantization Calibration that enables zero-shot generalization to real-world environments without requiring re-calibration on target-domain data.

Hypothesis of Distribution Subsumption. Our approach is grounded in the insight that the activation statistics of robotic perception are governed by the complexity of environmental dynamics. We hypothesize that large-scale synthetic datasets, such as TartanAir [79], exhibit a "superset" of features compared to structured real-world datasets like KITTI [66] or EuRoC [65]. Specifically, synthetic environments are characterized by *high-entropy distributions* containing extreme optical flow magnitudes, aggressive camera rotations, and challenging texture artifacts. In contrast, real-world benchmarks often follow more predictable motion models and constrained lighting conditions.

Therefore, we posit that the activation distribution of the target real domain $\mathcal{P}_{real}$ is statistically subsumed by the synthetic source domain $\mathcal{P}_{sim}$: $\mathrm{Supp}(\mathcal{P}_{real}) \subset \mathrm{Supp}(\mathcal{P}_{sim})$, where $\mathrm{Supp}(\cdot)$ denotes the support of the distribution. By calibrating on $\mathcal{P}_{sim}$, we derive a conservative **quantization envelope** that naturally encompasses the dynamic range of $\mathcal{P}_{real}$ as a subset.

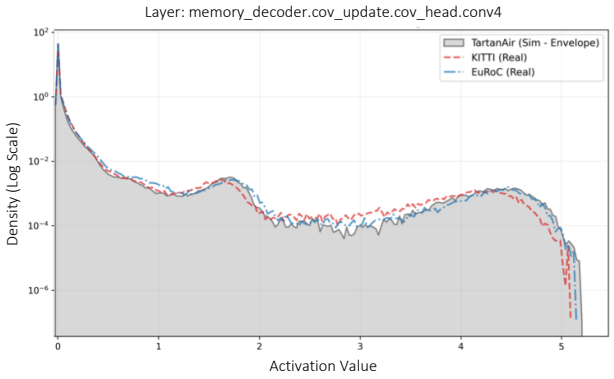

Figure 4.5: Validation of Sim-to-Real Distribution Subsumption. We visualize the activation density (log scale) of the middle layer in the uncertainty head. The synthetic TartanAir dataset (grey filled area) exhibits a high-entropy distribution with heavy tails, effectively forming a quantization envelope that subsumes the narrower distributions of real-world datasets (KITTI and EuRoC).

This hypothesis is empirically validated in Figure 4.5. The activations of the real-world datasets (KITTI and EuRoC) appear as narrow modes concentrated within a limited dynamic range. In contrast, the TartanAir distribution (grey envelope) is significantly broader, covering extreme activation values that represent challenging geometric features or lighting conditions.

Global Cross-Scene Sampling. To effectively approximate the global statistics of $\mathcal{P}_{sim}$, we reject the standard practice of sampling consecutive frames from a single scene, which introduces severe temporal bias. Instead, we implement a Global Cross-Scene Sampling strategy. We construct a global calibration set $\mathcal{D}_{calib}$ by uniformly sampling frames across diverse scenes [79]. This strategy ensures that the calibration set captures the maximum variance of activation outliers.

Formally, adhering to the MSE-based calibration framework defined in Sec. 4.3.1, we determine the optimal quantization range bounds $[q_{\min}, q_{\max}]$ (and subsequently s and z) by minimizing the reconstruction error over the global synthetic set $\mathcal{D}_{calib}$:

$$(q^*_{\min}, q^*_{\max}) = \underset{q_{\min}, q_{\max}}{\operatorname{argmin}} \, \mathbb{E}_{\mathbf{x} \in \mathcal{D}_{calib}} \left[\|\mathbf{x} - \widehat{\mathbf{x}}(q_{\min}, q_{\max})\|_2^2 \right], \tag{4.7}$$

where $\widehat{\mathbf{x}}$ is the de-quantized approximation derived via Eq. 4.3.

Zero-Shot Generalization. This calibration method decouples the quantization process from the deployment environment. Calibrating on KITTI would result in a narrow clipping range (overfitting to the mode), causing severe saturation when unseen outliers occur. By calibrating on the TartanAir "superset", our method derives a conservative range that naturally accommodates real-world statistics. Once calibrated offline using our global synthetic set, the quantized model can be deployed "zero-shot" to various real-world scenes without any fine-tuning or re-calibration. As empirically validated in Sec. 4.4, this method not only matches the accuracy of domain-specific calibration but often outperforms it by preventing the clipping of rare but critical geometric features.

4.3.4 Backend Quantization

The backend pose estimation in Eq. 4.2 is solved with a Gauss-Newton second-order optimizer. While the solver itself must remain numerically stable, the dominant implementation bottleneck

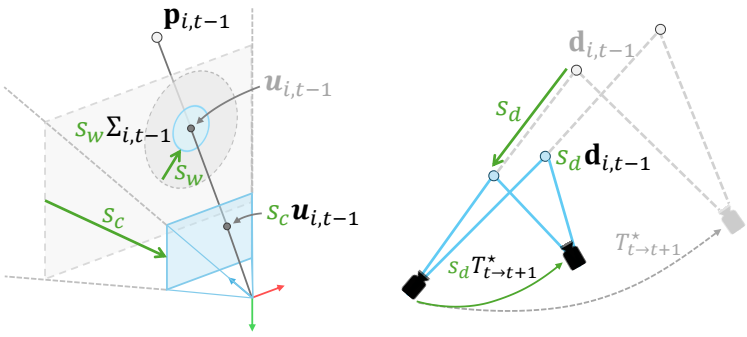

Figure 4.6: Backend scaling factors for FP16 optimization. To prevent overflow/underflow when forming the normal equations, we rescale the backend using three factors: (1) image-plane scaling s_c applied to pixel coordinates; (2) weight scaling s_w applied to the inverse covariance $\mathbf{W} = \mathbf{\Sigma}^{-1}$; and (3) 3D scaling s_d applied to the depths and motion.

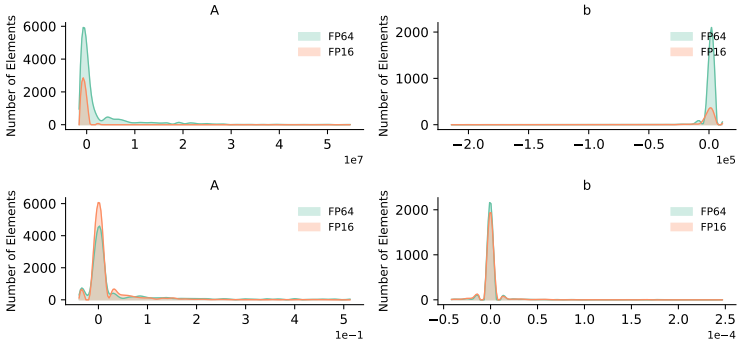

Figure 4.7: Element-wise histograms of the Gauss-Newton normal equation $(\mathbf{A}, \mathbf{b})$. Top: naive FP16 suffers from dynamic-range mismatch, leading to overflow and NaN values. **Bottom:** our range-compression scaling brings $(\mathbf{A}, \mathbf{b})$ into a stable FP16 regime, closely matching the FP64 distribution.

is the floating-point format used for building the normal equations. Compared with FP16, BF16 offers a much wider exponent range but a coarser mantissa; its relative precision is on the order of 8×10^{-3}, which would bound the best-achievable translation refinement to the millimeter-centimeter scale. In contrast, FP16 provides a finer relative precision (on the order of 10^{-3}), which is required by our back-end accuracy target. Therefore, we choose FP16 for the back-end arithmetic where possible.

However, FP16 has a limited representable dynamic range (maximum finite value on the order of 6.5×10^4). In Gauss-Newton, the normal equations are formed as

$$\mathbf{A}\,\Delta = \mathbf{b}, \quad \mathbf{A} = \sum_k \mathbf{J}_k^\top \mathbf{W}_k \mathbf{J}_k, \quad \mathbf{b} = -\sum_k \mathbf{J}_k^\top \mathbf{W}_k \mathbf{r}_k, \tag{4.8}$$

where $\mathbf{J}_k$ and $\mathbf{r}_k$ are the Jacobian and residual of the reprojection error, and $\mathbf{W}_k$ is the inverse covariance from the front-end uncertainty. Even when residuals are well-behaved, the terms in $\mathbf{A}$ can become large due to squaring and accumulation, which can overflow to ∞/NaN in FP16. Hence, the key challenge is to compress the numeric range of intermediate results so that building $\mathbf{A}$ and $\mathbf{b}$ remains within the FP16 range.

Scale-invariant normal equations. Our design relies on the fact that linear systems are

Algorithm 1 FP16 Gauss-Newton optimization with scaling

Input: Observations $\{\mathbf{u}_{k,0}, d_{k,0}, \mathbf{u}^L_{k,1}, \mathbf{u}^R_{k,1}, \mathbf{\Sigma}_k\}_{k=1}^N$,
camera intrinsics K, baseline b, initial pose $\mathbf{T}$
camera scale s_c, 3D scale s_d, and weight scale s_w
Output: Optimized relative pose $\mathbf{T}^\star$

$\mathbf{W}_k \leftarrow s_w \mathbf{\Sigma}_k^{-1}$
$\text{trans}(\mathbf{T}) \leftarrow s_d \cdot \text{trans}(\mathbf{T})$
for $i = 1$ to N_{iter} **do**
  $s_c\mathbf{r}_k, s_c\mathbf{J}_k \leftarrow \mathcal{E}(\mathbf{T}; s_c\mathbf{u}, s_d d, K, s_d b)$
  $s_c\tilde{\mathbf{r}}_k, s_c\tilde{\mathbf{J}}_k \leftarrow \text{Huber}(s_c\mathbf{r}_k, s_c\mathbf{J}_k; s_c\delta)$
  $s_w s_c^2 \mathbf{A} \leftarrow \sum_k s_c\tilde{\mathbf{J}}_k^\top s_w \mathbf{W}_k s_c \tilde{\mathbf{J}}_k$
  $s_w s_c^2 \mathbf{b} \leftarrow -\sum_k s_c\tilde{\mathbf{J}}_k^\top s_w \mathbf{W}_k s_c \tilde{\mathbf{r}}_k$
  $\Delta = \text{CholeskySolve}(s_w s_c^2 \mathbf{A}, s_w s_c^2 \mathbf{b})$ ▷ FP32 Precision
  $\mathbf{T} \leftarrow \text{Exp}(\Delta)\mathbf{T}$
end for
$\text{trans}(\mathbf{T}) \leftarrow 1/s_d \cdot \text{trans}(\mathbf{T})$
return $\mathbf{T}^\star \leftarrow \mathbf{T}$

invariant to a non-zero scaling: $\forall s \neq 0$, $\mathbf{A}\Delta = \mathbf{b}$ and $(s\mathbf{A})\Delta = (s\mathbf{b})$ share the same solution. We exploit this invariance by applying three scale factors, as shown in Figure 4.6, when constructing the normal equations. We provide the high-level pseudo-code of our backend in Algorithm 1.

Weight scaling (w-scale). We scale the weight matrix by a uniform positive scalar s_w: $\mathbf{W}_k \leftarrow s_w \mathbf{W}_k$. This uniformly scales both $\mathbf{A}$ and $\mathbf{b}$ by s_w, leaving Δ unchanged, while shrinking the magnitude of the accumulated normal equations.

Image-plane scaling (c-scale). We scale all image-plane quantities by s_c, including pixel observations $\mathbf{u}$, intrinsics K, and the robustification threshold (Huber δ). Geometrically, this is equivalent to moving the image plane closer to the optical center, which shrinks the pixel coordinates and stabilizes the Jacobians. Under this scaling, both residuals and Jacobians scale as $\mathbf{r}_k \leftarrow s_j \mathbf{r}_k$ and $\mathbf{J}_k \leftarrow s_j \mathbf{J}_k$, which implies $\mathbf{A} \leftarrow s_j^2 \mathbf{A}$ and $\mathbf{b} \leftarrow s_j^2 \mathbf{b}$, again preserving the Gauss-Newton step.

3D scaling (d-scale). Finally, we scale the entire 3D geometry by a factor s_d to avoid overflow in depth-dependent terms such as $(1/z)^2$. Concretely, we scale depth and baseline by s_d and scale the translation component of the $SE(3)$ estimate by s_d before optimization. Unlike w- and j-scales, this operation changes the physical units of translation; therefore, after the FP16 optimization converges, we explicitly restore the scale of translation by dividing s_d.

Scale factor selection. In our implementation, the three scale factors are computed on-the-fly from simple statistics of the current frame pair. For the image-plane scaling, we set

$$s_c = \frac{1}{\max\left(\mathbf{u}^L\right)}, \tag{4.9}$$

so that the largest observed pixel coordinate is mapped to $\mathcal{O}(1)$. For the 3D scaling, we use depth statistics to simultaneously compress large depths and avoid overflow in $(1/z)^2$:

$$s_d = \max\left(\frac{1}{\max(d)}, \frac{0.01}{\min(d)}\right). \tag{4.10}$$

The first term maps the maximum depth to roughly one, while the second enforces $s_d \min(d) \geq 0.01$ to bound $(1/z)^2$ in FP16. Finally, after constructing per-observation weights $\mathbf{W}_k = \mathbf{\Sigma}_k^{-1}$,

Table 4.1: Performance comparison on the TartanAir v2 Hard Dataset. Average is calculated over all trajectories of TartanAir v2 Hard.

Trajectory	W/A Bits	H00		H01		H02		H03		H04		H05		H06		Avg.	
		t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
Baseline	FP32/FP32	0.040	0.048	0.020	0.101	0.016	0.078	0.181	0.424	0.005	0.071	0.071	0.359	0.218	0.791	0.079	0.267
RTN*	INT8/INT8	1.149	3.839	0.562	3.529	0.318	2.225	0.506	4.578	0.311	2.436	0.338	4.717	0.504	4.185	0.527	3.644
SmoothQuant [1]	INT8/INT8	1.102	3.440	0.371	2.806	0.243	1.882	0.382	3.382	0.202	1.680	0.292	3.175	0.337	3.232	0.419	2.800
Quarot [78]	INT8/INT8	0.181	0.193	0.025	0.141	0.020	0.092	0.019	0.116	0.006	0.0880	0.024	0.386	0.204	0.825	0.068	0.263
Ours	INT8/INT8	0.173	0.163	0.025	0.151	0.018	0.085	0.039	0.169	0.005	0.071	0.023	0.382	0.224	0.819	0.072	0.263

* Naive round to nearest.

we normalize them by the maximum weight magnitude and the number of points N to keep the accumulated normal equations well-scaled:

$$s_w = \frac{1}{\max_k \|\mathbf{W}_k\| \, N}, \quad \mathbf{W}_k \leftarrow s_w \mathbf{W}_k. \tag{4.11}$$

In practice, we build $\mathbf{A}$ and $\mathbf{b}$ in FP16 under the above scaling, but solve the 6×6 system in FP32 via Cholesky on a symmetrized $\frac{1}{2}(\mathbf{A} + \mathbf{A}^\top)$ to preserve solver robustness.

4.4 Experiments

4.4.1 Implementation Details

Datasets & Baseline. We follow MAC-VO [70] and evaluate the proposed model and baseline methods on a variety of public datasets, including synthetic dataset TartanAir v2 [79], real-world data from EuRoC [65], and KITTI [66]. These datasets cover a diverse range of hardware configurations, motion patterns, and environments.

Evaluation Metrics. Since the baseline [70] does not contain loop closure or global bundle adjustment, our evaluation follows the baseline and focuses on relative error. We use relative translation error (t_{rel}, m/frame) and rotation error (r_{rel}, °/frame) as:

$$\begin{aligned} t_{\text{rel}} &= \frac{1}{N} \sum_{t=1}^{N} \left\| \mathbf{p}_{t+1} - \mathbf{p}_t - R_t \hat{R}_t^\top (\hat{\mathbf{p}}_{t+1} - \hat{\mathbf{p}}_t) \right\|_2, \\ r_{\text{rel}} &= \frac{180}{\pi} \frac{1}{N} \sum_{t=1}^{N} \left\| \log \left(\hat{R}_{t,t+1}^\top R_{t,t+1} \right) \right\|_2, \end{aligned} \tag{4.12}$$

where $\mathbf{p}_t$ and R_t are ground truth position and rotation, $\hat{\mathbf{p}}_t$ and $\hat{R}_t$ are the estimated position and rotation. $R_{t,t+1} = R_t^\top R_{t+1}$ is the rotation from frame t to frame $t+1$.

4.4.2 Quantization Accuracy Comparison

TartanAir v2 Dataset. TartanAir v2 [79] presents significant challenges for visual odometry due to its rigorous motion and extreme illumination changes. We evaluate our quantization method on the challenging TartanAir v2 Hard dataset, as shown in Table 4.1. Standard methods

Table 4.2: Performance comparison of different methods on the EuRoC Dataset. Average is calculated over all trajectories of EuRoC.

Trajectory	W/A Bits	MH01		MH03		MH05		V102		V201		V203		Avg.	
		t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
Baseline	FP32/FP32	.0014	.0153	.0023	.0187	.0024	.0181	.0036	.0598	.0014	.0339	.0056	.1350	.0028	.0468
RTN*	INT8/INT8	.0083	.1083	.0170	.1924	.0204	.1924	.0131	.2240	.0333	.8706	.1384	3.854	.0384	.9070
SmoothQuant [1]	INT8/INT8	.0064	.0937	.0107	.1161	.0123	.1037	.0104	.1962	.0067	.1499	.0358	1.033	.0137	.4226
Quarot [78]	INT8/INT8	.0015	.0195	.0026	.0251	.0031	.0248	.0046	.0848	.0022	.0460	.0078	.1646	.0036	.0608
Ours	INT8/INT8	.0013	.0167	.0024	.0213	.0026	.0207	.0036	.0572	.0016	.0368	.0059	.1412	.0029	.0490

* Naive round to nearest.

Table 4.3: Performance comparison of different methods on the KITTI Dataset. Average is calculated over all trajectories of KITTI Odometry.

Trajectory	W/A Bits	00		02		04		06		08		10		Avg.	
		t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
Baseline	FP32/FP32	.0176	.0510	.0196	.0400	.0146	.0243	.0127	.0279	.0223	.0338	.0115	.0391	.0164	.0360
RTN*	INT8/INT8	.2664	.3718	.4393	.5070	.9022	.2346	.6867	.1980	.2618	.3205	.2654	.3432	.4703	.3292
SmoothQuant [1]	INT8/INT8	.1747	.2455	.1885	.1966	.8191	.1460	.4060	.2314	.1883	.2139	.2973	.2461	.3456	.2132
Quarot [78]	INT8/INT8	.0176	.0523	.0196	.0410	.0140	.0262	.0122	.0288	.0224	.0352	.0119	.0404	.0163	.0373
Ours	INT8/INT8	.0174	.0518	.0195	.0405	.0139	.0254	.0121	.0297	.0223	.0345	.0117	.0396	.0161	.0369

* Naive round to nearest.

suffer severe degradation under these extreme conditions; naive RTN and SmoothQuant [1] yield high translation and rotation errors, respectively, failing to handle the dynamic activation range. In contrast, our method achieves state-of-the-art performance, outperforming SmoothQuant and matching the advanced rotation-based method Quarot [78]. Crucially, our W8A8 model achieves near-lossless accuracy compared to the FP32 baseline, confirming that our approach preserves geometric fidelity for efficient inference.

EuRoC Dataset. To validate the real-world generalization capability of our method, we evaluate on the EuRoC MAV dataset [65], which features aggressive flight dynamics. As shown in Table 4.2, standard post-training quantization methods (naive RTN) struggle to maintain trajectory precision; and SmoothQuant [1] yields large translation and rotation errors, representing a significant degradation compared to the FP baseline. In contrast, our method achieves an average t_{rel} of 0.003, significantly outperforming SmoothQuant and even surpassing the rotation-based Quarot [78] (0.004). Notably, our W8A8 model delivers near-lossless accuracy virtually identical to the FP32 baseline, empirically confirming that our Sim-to-Real calibration strategy robustly covers real-world statistics for zero-shot deployment.

KITTI Dataset. To further validate our robustness and consistency in outdoor settings, we evaluate our method on the KITTI Odometry dataset [66], a standard benchmark for outdoor autonomous driving characterized by large-scale urban scenes and high-speed motion. As presented in Table 4.3, naive RTN and SmoothQuant [1] exhibit significant performance drops in this complex outdoor environment, failing to maintain reliable localization. In contrast, our method demonstrates exceptional robustness, consistently achieving state-of-the-art accuracy across all trajectories. Most notably, our W8A8 model delivers near-lossless performance virtually indistinguishable from the FP32 baseline, proving that our Sim-to-Real calibration strategy effectively captures the activation statistics required for zero-shot deployment in autonomous driving appli-

Table 4.4: **Scaling Laws for Sim2Real Quantization Calibration.** We evaluate the quantization performance on relative translation error (t_{rel}, m/frame) and rotation error (r_{rel}, °/frame) across synthetic and real-world datasets as we increase the size and diversity of the synthetic calibration set ($\mathcal{D}_{calib}$). The calibration frames are uniformly sampled from increasing numbers of TartanAir sequences.

Calibration Source	Calib. Size (# Frames)	W/A Bits	TartanAir (Sim)		EuRoC (Real)		KITTI (Real)		Avg.	
			r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}
Baseline		FP32 / FP32	0.079	0.267	.0028	.0468	.0164	.0360	.0351	.1246
Target Domain Calib.*	256	INT8 / INT8	.0743	.2680	.0028	.0469	.0163	.0361	.0290	.1250
Ours (Sim-Calib)	16	INT8 / INT8	.0855	.3326	.0029	.0488	.0161	.0370	.0338	.1494
	32	INT8 / INT8	.0811	.2938	.0029	.0487	.0162	.0368	.0359	.1352
	64	INT8 / INT8	.0755	.2838	.0029	.0481	.0162	.0364	.0356	.1315
	128	INT8 / INT8	.0750	.2813	.0028	.0478	.0161	.0365	.0294	.1309
	256	INT8 / INT8	.0748	.2780	.0028	.0475	.0161	.0363	.0292	.1292
	512	INT8 / INT8	.0748	.2778	.0028	.0476	.0162	.0362	.0291	.1293

* Standard PTQ calibrated on the test domain itself.

cations.

4.4.3 Scaling Laws for Sim-to-Real Calibration

Table 4.5: We progressively validate the effectiveness of each proposed component, including Hybrid Quantization (Hybrid Quant), Sim-to-Real Calibration (Sim2Real Calib), and Backend Quantization (Backend Quant). The results demonstrate that each module contributes significantly to recovering the performance of the quantized model.

Setting	W/A Bits	TartanAir (Sim)		EuRoC (Real)		KITTI (Real)		Avg.	
		r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}
FP Model	FP32 / FP32	.0790	0.2670	.0028	.0468	.0164	.0360	.0351	.1246
RTN	INT8 / INT8	.5270	3.644	.0384	.9070	.4703	.3292	.4639	1.733
+ Hybrid Quant	INT8 / INT8	.0770	.2840	.0035	.0550	.0182	.0373	.0352	.1338
+ Sim2Real Calib	INT8 / INT8	.0720	.2630	.0029	.0490	.0161	.0369	.0325	.1240
+ Backend Quant	INT8 / INT8	.0748	.2780	.0028	.0475	.0161	.0363	.0335	.1288

To validate our hypothesis that a sufficiently diverse synthetic calibration set can encompass real-world statistics, we investigate the **scaling laws** of our calibration method. We vary the size of the global calibration set $\mathcal{D}_{calib}$ by uniformly sampling frames from an increasing number of synthetic TartanAir environments (ranging from 16 to 512 frames). We then evaluate the quantized model's zero-shot performance across three distinct domains: the unseen TartanAir test split, KITTI (outdoor driving), and EuRoC (indoor MAV). The quantitative results are summarized in Table 4.4.

Sim-to-Real Correlation. We observe a strong correlation between the diversity of synthetic calibration data and cross-domain generalization. As the calibration size increases from 16 to 256 frames, the relative translation error (t_{rel}) decreases monotonically. Crucially, the performance gains on the synthetic validation set (TartanAir) translate directly to the real-world benchmarks. This empirically confirms our *Distribution Subsumption* hypothesis: by capturing the long-tail distribution of the synthetic "superset", we progressively construct a robust quantization envelope that inherently covers the subsets of real-world statistics.

Saturation and Efficiency. The performance improvement follows a logarithmic trend, showing rapid gains initially and saturating around 256 frames. Extending the calibration size

to 512 frames yields negligible marginal gains, indicating that the quantization parameters have converged to an optimal "envelope" that accommodates potential outliers.

Table 4.6: End-to-end latency comparison on NVIDIA Jetson Thor across input image resolutions under W8A8, reported as throughput (fps). SmoothQuant [1] is theoretically equal latency with RTN.

Resolution	Baseline	RTN/SmoothQuant [1]	Quarot [78]	Ours
640×640	1.16	3.90 (↑3.36×)	1.27 (↑1.09×)	**3.35** (↑2.89×)
480×640	1.67	5.08 (↑3.04×)	1.79 (↑1.07×)	**4.33** (↑2.59×)
320×320	4.78	9.37 (↑1.96×)	5.40 (↑1.13×)	**8.76** (↑1.83×)
224×224	7.67	11.72 (↑1.53×)	8.48 (↑1.11×)	**10.91** (↑1.42×)

4.4.4 Latency Evaluation

We evaluate runtime efficiency by deploying our model on NVIDIA Jetson Thor and measuring end-to-end latency, comparing against the FP32 MAC-VO baseline and quantized baselines including RTN, SmoothQuant [1], and QuaRot [78]. We report throughput in fps (frames per second). All methods are evaluated under W8A8. Each quantized model is exported to ONNX [94] and compiled into a TensorRT [95] engine for deployment. The FP32 baseline is measured using the original MAC-VO [70] implementation. We fix the network depth of the front-end to 12 and evaluate latency on different image resolutions from 224×224 to 640×640.

As shown in Table 4.6, applying Hadamard rotation to all quantized layers (QuaRot [78]) provides only a marginal speedup (avg. 1.1× FPS). This indicates that the rotation computations incur noticeable latency overhead. In contrast, our method achieves up to 2.9× speedup over the entire baseline, despite the added rotation operations. RTN and SmoothQuant [1] introduce no additional inference-time operators. However, they show higher FPS across all resolutions but suffer from accuracy degradation as discussed in Sec. 4.4.2.

4.4.5 Ablation Study

To verify the effect of each component in our framework, we conduct a comprehensive ablation study as shown in Table 4.5. Directly applying standard Round-to-Nearest (RTN) quantization leads to catastrophic degradation, increasing the average relative translation error, rendering the system unusable. After introducing our Hybrid Quant strategy, it significantly mitigates quantization noise, recovering the majority of the accuracy. Furthermore, by incorporating our Sim-to-Real Calibration method with global synthetic sampling, the model achieves better generalization on real-world datasets. Finally, with the inclusion of our backend quantization, the fully integrated system achieves performance comparable to the FP32 baseline, ensuring robust deployment across diverse environments.

4.5 Sim-to-Real Calibration Performance Curve

To visually analyze the impact of calibration data diversity, we plot quantization error as a function of the number of synthetic calibration frames in Figure 4.8. The curves reveal a *scaling law* for Sim-to-Real quantization: both translation error (blue) and rotation error (red)

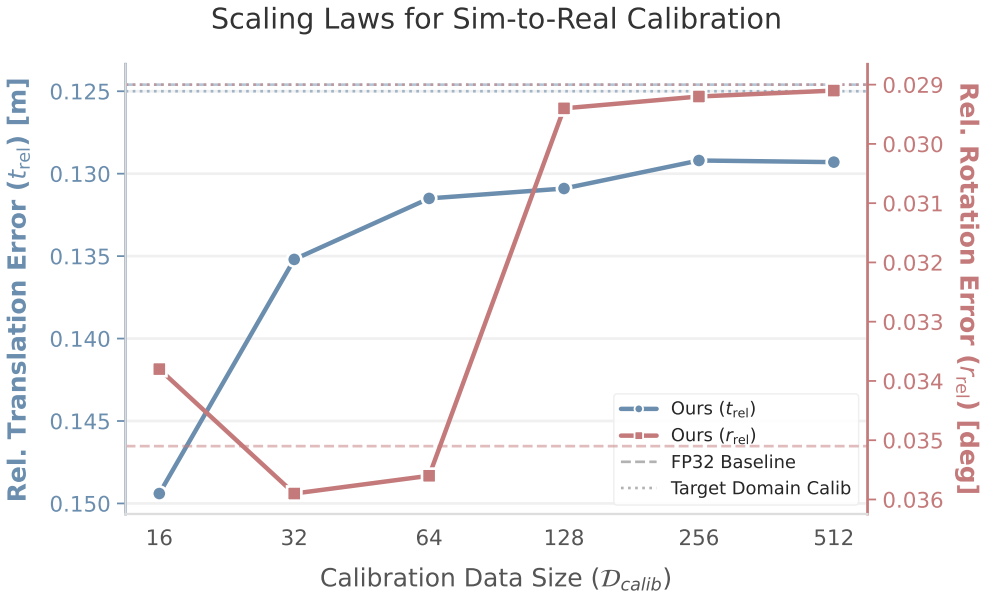

Figure 4.8: Scaling Laws for Sim-to-Real Calibration. We visualize the quantization performance on real-world datasets as a function of the synthetic calibration data size ($\mathcal{D}_{calib}$). The left and right Y-axes represent relative translation error (t_{rel}) and rotation error (r_{rel}), respectively. Note that the Y-axes are inverted, so that *higher* positions indicate better performance. The curves demonstrate a monotonic performance improvement that saturates around 256 frames.

decrease monotonically as the calibration set size grows from 16 to 256 frames. The error curves begin to flatten around 128 frames and essentially saturate by 256 frames. At this point, the performance of our method becomes nearly indistinguishable from that of the target-domain calibration baseline. This result suggests that roughly 256 carefully diversified synthetic frames are sufficient to construct a quantization envelope that adequately captures the statistics of established real-world datasets such as KITTI and EuRoC.

4.6 Conclusion and Discussion

This chapter presents QuantMAC-VO, a post-training quantization framework tailored for learning-based visual odometry with metrics-aware covariance. By leveraging a kurtosis-guided hybrid quantization scheme and a Sim-to-Real calibration strategy, the method effectively mitigates the impact of non-stationary distributions and outlier activations inherent in VO tasks. We empirically demonstrated a scaling law where synthetic calibration diversity directly translates to real-world robustness, enabling a 1.42× to 2.89× speedup on the NVIDIA Jetson Thor platform with comparable accuracy.

A primary limitation is that the backend optimization still relies on floating-point arithmetic (albeit with explicit scaling) to preserve numerical stability, preventing a fully integer-only pipeline. Future work will focus on exploring fully quantized graph optimization to eliminate remaining floating-point overhead, and investigating lower-bit quantization (e.g., W4A4) to further reduce memory footprint for deployment on ultra-low-power microcontrollers.

Together with MAC-VO (Chapter 3), this chapter completes Part I of the thesis: metrics-

aware visual uncertainty can be learned, integrated into a full VO pipeline, and efficiently deployed on edge hardware while preserving both pose accuracy and uncertainty quality.

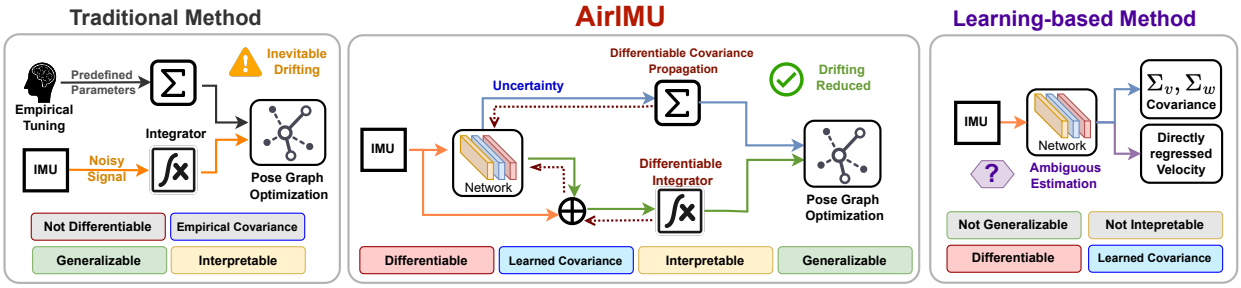

Figure 5.1: Left: Traditional methods with empirically set uncertainty parameters often fail to capture noise inherent in the sensor accurately. **Middle:** AirIMU corrects the IMU noise and predicts the corresponding uncertainty of the integration. Notably, its covariance propagation is differentiable, promoting efficient training. **Right:** Learning-based methods predict velocity and the corresponding covariance but lack interpretability and generalizability across different data domains.

Chapter 5

AirIMU: Learning Uncertainty Propagation for Inertial Odometry

5.1 Introduction

Inertial Measurement Unit (IMU), capable of sensing high-frequency linear acceleration and angular velocity, is an essential component in almost all kinds of robots [96–98]. As a result, inertial odometry (IO), which aims to estimate the relative movements of robots using IMU measurements, has broad applications in autonomous driving [98], navigation [99], augmented reality [100], and human motion tracking [101]. Due to the IMUs' relative independence from the external environment, IO is insensitive to environmental disturbances such as illumination changes and visual obstructions [96]. Therefore, IO has great potential in robust state estimation under complex operation conditions, especially compared to exteroceptive sensors like cameras and LiDAR [6]. Nevertheless, the performance of existing IO algorithms, either the model-based [102] or the data-driven methods [103], is still unsatisfactory for real-world applications [96].

Kinematic model-based IO methods have excellent generalizability but often suffer from complex and multifaceted noise [104]. The IMU sensor noises can be classified into two categories: the deterministic errors and the non-deterministic errors [105]. The deterministic errors are the

systematic biases in the measurements that can be consistently predicted or modeled, often arising from factors such as installation imperfections, and calibration errors [106]. Conversely, the non-deterministic errors, which are also known as stochastic errors, are related to factors like mechanical shocks, vibration influences, thermal effects, mechanical defects, and calculation ambiguities [107–109]. Many methods employ linearized models to handle the deterministic errors by accounting for additive sensor bias, scale factors, and axis misalignment [110–112] and handle non-deterministic errors using zero-mean Gaussian function [6, 105]. However, these simplified linear models struggle to fully account for non-deterministic uncertainties from the natural environments, highlighting a critical area for improvement. Data-driven IO, which use neural networks to predict the motion and implicitly fit the IMU noise, often face challenges with interpretability and generalizability. To be more specific, these methods often overfit in specific motion patterns such as pedestrian [3] or vehicle [113], and fail to generalize to scenarios outside of the training distribution. For example, the model trained on hand-held platforms running on flat ground fails to generalize to robots climbing stairs or drones [114]. Moreover, replacing the motion models with the neural networks undermines the model's interpretability.

Recent advances in hybrid methods show the potential to merge the benefits of both the model-based method and the data-driven method. For example, some researchers fuse the data-driven motion model with the IMU kinematic motion model using EKF [115, 116] or batched optimization [117]. These solutions improve the robustness of the state estimation but still struggle with generalizability. On the other hand, efforts have been made in training neural networks to model the error in gyroscope [113], accelerometer[118], IMU bias evolution [114]. Despite these advancements, none of these algorithms explicitly model the uncertainty of the IMU measurements in the IMU pre-integration. When deploying the data-driven model with visual-inertial odometry [114, 118], these algorithms still assume the IMU covariance model as a manually calibrated stochastic variable. These gaps hinder the deployment of hybrid methods for real-world robots.

To close these gaps, this chapter presents a hybrid method AirIMU as illustrated in Figure 5.3. It leverages the data-driven method to model the non-deterministic noise and the kinematic motion model to ensure generalizability in novel environments. To empower the AirIMU with the ability to learn the uncertainty and noise model, we build a differentiable IMU integrator and a differentiable covariance propagator on manifolds. Unlike the previous data-driven covariance prediction models [115, 116] that only predict the covariance of the output translation within a fixed window, we predict the accumulated covariance for a long-time pre-integration. Moreover, we jointly train the uncertainty module with the noise correction module, which can capture better IMU features and benefit both tasks.

In the experiments, we demonstrate the adaptability to a full spectrum of IMUs encompassing various grades and modalities, as illustrated in Figure 5.2. To validate the effectiveness of our learned covariance in sensor fusion, we design a pose graph optimization (PGO) system that fuses the AirIMU outputs with the GPS signals. To the best of our knowledge, we are the first to train a deep neural network that models IMU sensor uncertainty through differentiable covariance propagation. The main contributions of this chapter are:

- This chapter proposes AirIMU, a hybrid method that leverages the data-driven method to model the non-deterministic error, and the kinematic motion model to ensure generalizability in novel environments.
- We propose to jointly train the IMU noise and uncertainty through the differentiable in-

tegrator and covariance propagator. This training strategy improves 17.8% of the IMU preintegration results in Subt-MRS benchmark [96].

- Our method demonstrates adaptability across multiple grades of IMUs, from low-cost automotive-grade IMUs to high-end navigation-grade IMUs. We further evaluate AirIMU on the large-scale visual terrain navigation dataset ALTO [2] cruising over 262 km. Additionally, we illustrate the effectiveness of the learned uncertainty in a GPS-IMU PGO experiment, achieving a 31.6% improvement in optimization.
- We develop a tool package for the differentiable IMU integrator and covariance propagator that supports batched operations and the cumulative product, which speeds up the training process. This package is grounded on *PyPose* [63] library and is open-source at: https://pypose.org/tutorials/imu

5.2 Related Works

5.2.1 Traditional Inertial Odometry

Traditional model-based IO algorithms utilize the IMU kinematic motion model [102] to estimate the relative motion and are often integrated with other auxiliary sensors like GPS, Camera [6], and LiDAR [119] with filter-based method or batched optimization. To deploy IO on mobile robots, especially those equipped with low-cost IMUs, modeling IMU deterministic errors is indispensable [104], which generally include the additive sensor bias, scale factors, and axis misalignment [120].

To address the non-deterministic error, factors like nonlinearity, g-sensitivity [112], size effect [121], vibration [108], thermal effect [122], and Coriolis force [123] are rigorously addressed. These methods, however, rely on expensive motion simulators such as shakers and rate tables [124], which is not available in practice. To calibrate the low-cost IMUs, the IO algorithms utilize the IMU measurements under static conditions [104, 110], or auxiliary sensors like the camera [105, 125] and LiDAR [126] for calibration. Kalibr [105] is one of the most widely used calibration algorithms, calibrating the IMU intrinsic and extrinsic parameters between the camera and the IMU. However, all these calibration methods focus on modeling the linear behavior of the IMU sensor under static conditions or controlled environments, which often struggle to fully capture the noise inherent in the natural environment and sensor defects. This reveals a significant gap in the current IMU calibration and the practices of IO.

5.2.2 Learning Inertial Odometry

A growing trend in leveraging a data-driven model for IO has been witnessed, such as IMU odometry network [103, 127], EKF-fused IMU odometry [115, 116, 128] and learning-based IMU calibration [129]. In RIDI [127], a supervised Support Vector Machine (SVM) classifier is introduced to determine IMU placement modes using accelerometer and gyroscope measurements. Mode-specific Support Vector Regressions (SVRs) are then employed to estimate pedestrian velocities based on motion types, enabling corrective adjustments to sensor readings. IONet [103] introduced deep learning to regress the translation within a fixed temporal window. Building on this concept, RoNIN [3] improves the accuracy of the data-driven IO with multiple neural

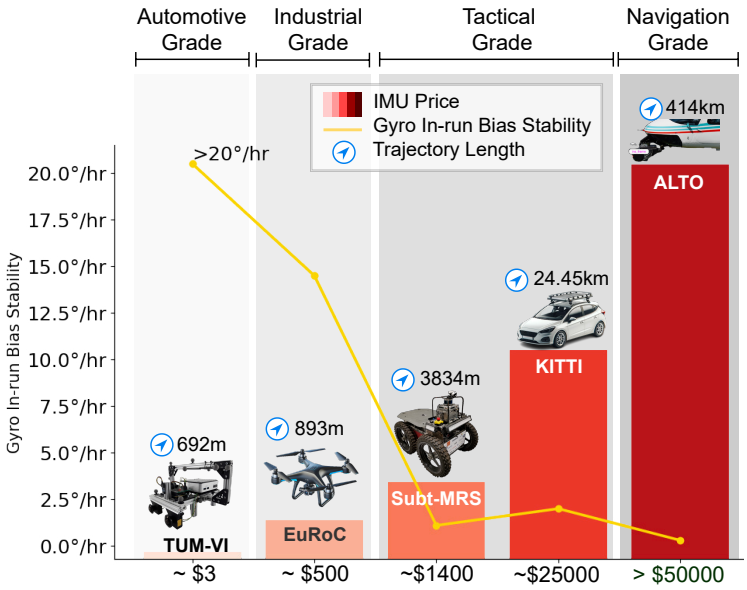

Figure 5.2: We evaluate our algorithm on IMUs with different prices and drift rates. Sensor quality is evaluated based on the gyroscope's in-run bias stability metric. Our evaluation spans the full spectrum of IMUs, including automotive-grade, industrial-grade, tactical-grade, and navigation-grade IMUs.

network architectures, including Temporal Convolutional Networks (TCN) and Long Short-Term Memory (LSTM). RIO [128] explores rotation-equivalence as a self-supervisory mechanism for training IO models. These studies utilize data-driven motion estimation models to replace the IMU integration process, thereby reducing the drift associated with IMU double integration. Inspired by these IMU odometry network studies, TLIO [115] integrates the data-driven motion estimation model with the IMU integrator through a stochastic-cloning Extended Kalman Filter (EKF). This integrated approach optimizes position, orientation, and sensor biases, solely with IMU data. IDOL [116] further extends this framework with a data-driven orientation estimator, generating more accurate results with better consistency. Although these methods are promising in pedestrian datasets like RoNIN [3], the motion estimation heavily relies on prior knowledge about human movements, such as the consistent walking pattern of pedestrians and constant velocity assumption, which fail to generalize to other platforms like drones and vehicles. Moreover, these studies often assume the IMU measurement is transformed to the world coordinates or aligning the y-axis with gravity, which is impractical in real scenarios.

Inspired by the advancements of the data-driven methods in modeling unknown parameters, prior researchers employ neural networks to implicitly calibrate the IMUs [118, 129, 130]. Brossard et al. [129] train a Convolutional Neural Network (CNN) to denoise the low-frequency error in gyroscopes. Zhang et al. [118] employed a Long Short-Term Memory (LSTM) network to preprocess the raw IMU, and subsequently channeled the preprocessed IMU measurements into a filter-based visual-inertial odometry system. Deep IMU Bias [114], utilizes neural networks to model the sensor bias, and incorporates the learned bias as an additional constraint in the bundle adjustment. These studies showcase the potential of data-driven methods in modeling IMU errors, especially when the sensor's physical conditions are difficult to capture. However, none of the referenced algorithms have explicitly tackled the challenge of modeling the uncertainty for IMU integration. When deploying these data-driven models, the predefined covariance parameters can no longer fit the learned IMU sensor model, undermining the IO's accuracy and robustness.

5.2.3 IMU Uncertainty Model

The uncertainty model in IO, which quantifies the reliability of inertial measurements, is the key to optimizing both filter performance and the batched optimization results. Traditional IO follows the kinematics model formulated in the IMU integration to propagate the covariance matrix from the predefined IMU uncertainty parameters [102]. A commonly used solution to calibrate the uncertainty parameters is the Allen deviation [131], which quantifies the sensor bias and bias instability by analyzing the variance of the sensor errors in a static position. However, these parameters often require empirical tuning and manual re-calibration during testing due to unknown environmental disturbances and mechanical imperfections.

Recent researches in data-driven methods demonstrate the potential to model the uncertainty model with neural network [59]. The AI-IMU [113] proposes a dead-reckoning algorithm for vehicles, which assumes that the lateral and vertical velocities are approximately null. To model the confidence degree of these null assumptions, the authors train a CNN network from IMU to predict two constant uncertainty parameters corresponding to lateral and vertical velocity. DualProNet [132] is another attempt to use a deep neural network to predict the uncertainty parameters in IMUs. This work, however, requires high-frequency ground truth covariance labels for all IMU measurements, which are generally non-available during practice. In TLIO [115], the authors jointly train a covariance model of the predicted velocity with a fixed temporal window, which is later fused in an EKF.

In response to these challenges, our method leverages the differentiable covariance propagation to learn the high data-rate uncertainty. This approach alleviates the need for high-frequency ground truth labels and bridges the gap between uncertainty modeling and practical limitations.

5.3 AirIMU Model

To balance the accuracy and generalizability, the AirIMU model decouples the non-deterministic error model from the precisely defined kinematic motion model, as depicted in Figure 5.3. Specifically, it utilizes a data-driven model to capture the error and uncertainty inherent in the IMU, while employing the kinematic-based method to integrate the state and covariance. By supervising the integrated state and covariance, the AirIMU jointly trains the sensor error and uncertainty models throughout the differentiable integrator. To accelerate the training speed with long sequences, we propose a tree-like structure for cumulative operation and redesign the covariance propagation to facilitate batched operation and parallel computing. The refined state and covariance outputs are subsequently incorporated into the PGO module, where they are fused with auxiliary sensors such as GPS.

Overall, we show the data-driven model in Sec. 5.3.1, the kinematic model in Sec. 5.3.2, loss functions in Sec. 5.3.3. Moreover, we detail our efforts of accelerating the integration and covariance propagation in Sec. 5.3.4. Lastly, we present the PGO module that fuses the AirIMU with auxiliary sensors like GPS in Sec. 5.3.5.

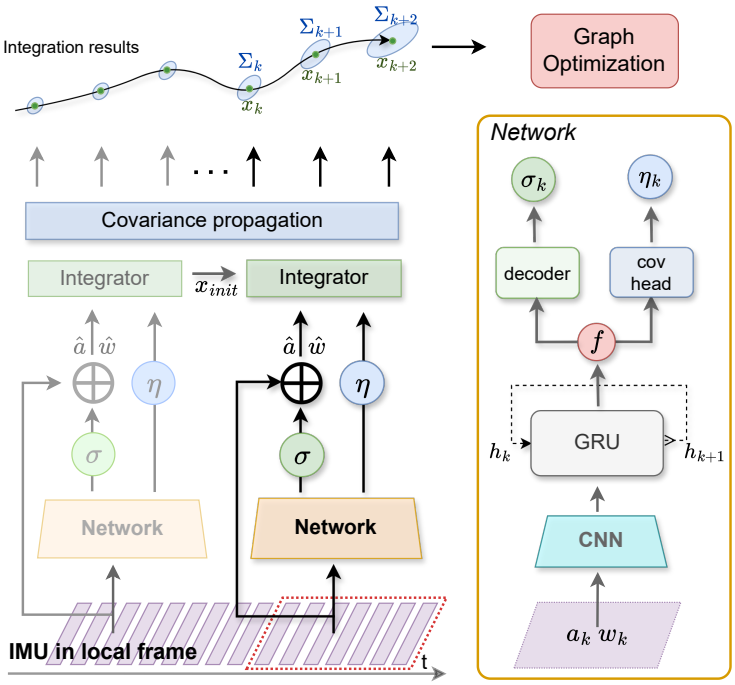

Figure 5.3: Right: We employ a CNN-GRU encoder to capture IMU features (f) from raw IMU accelerometer (a_k) and gyroscope (w_k) measurements. Subsequently, these features are decoded to obtain IMU corrections (σ_{acc}, σ_{gyro}). **Left:** We add the raw measurements with learned corrections, generating corrected IMU measurements ($\hat{a}_k$ and $\hat{w}_k$). The learned uncertainties (η) are propagated to estimate the corresponding covariance matrix (Σ).

5.3.1 Data-driven Module

As shown in the right part of Figure 5.3, the AirIMU model utilizes an encoder-decoder network structure to process raw IMU data, which includes three-axis acceleration a and three-axis angular velocity w, both measured in the body frame B. The network outputs corrections for acceleration and angular velocity, denoted as ${}^{B}\sigma^{acc}$ and ${}^{B}\sigma^{gyro}$, respectively. Additionally, it calculates the corresponding uncertainty measures for the gyroscope η^{gyro} and accelerometer η^{acc}.

To leverage and understand the hidden relationship between the IMU noise and uncertainty, we train a shared encoder network to capture the common representations of the raw IMU measurements for both tasks. In the encoder model, we first utilize a 1D CNN network to learn the lower-level features like the pattern of the accelerometer and gyroscope. To capture the temporal dependency from the previous IMU data, we combine the CNN network with gated recurrent unit (GRU) layers to generate the IMU feature.

With the shared features, we build a correction decoder to estimate the error ${}^{B}\sigma^{gyro}, {}^{B}\sigma^{acc}$ in the following:

$$\left[{}^{B}\sigma^{gyro}, {}^{B}\sigma^{acc}\right] = f_{\pi}\left({}^{B}w, {}^{B}a\right), \tag{5.1}$$

where π represents the learnable parameters of the decoder. For the covariance, we train another decoder with a similar structure to predict the uncertainty of each corresponding frame.

This uncertainty is predicted by a covariance model denoted as:

$$[\eta^{gyro}, \eta^{acc}] = \Sigma_\theta \left({}^B w, {}^B a \right), \tag{5.2}$$

where θ represents the learnable parameters of the covariance decoder model.

Overall, the corrected acceleration and angular velocity ${}^B\tilde{a}$ and ${}^B\tilde{w}$, are formulated as the summation of the original input ${}^B a.{}^B w$, the network corrections ${}^B\sigma_{acc}, {}^B\sigma_{gyro}$, and noise of the measurements:

$${}^B\tilde{a} = {}^B a + {}^B\sigma_{acc} + \eta_k^{acc}, \;\; {}^B\tilde{w} = {}^B w + {}^B\sigma_{gyro} + \eta_k^{gyro}. \tag{5.3}$$

Combined with these outputs with the learned uncertainty $\eta_k^{gyro}, \eta_k^{acc}$, we utilize the IMU kinematic model to estimate the states ${}^W x_j$, and more importantly, to propagate the corresponding covariance $\Sigma_{i,j}$.

5.3.2 Differentiable Integration and Covariance Module

Labeling the ground truth of the high-frequency data requires expensive motion simulators, which is infeasible due to the cost. Therefore, AirIMU builds a differentiable integrator and covariance propagator to supervise the data-driven module throughout the integration and covariance propagation.

The IMU pre-integration follows Newton's kinematic laws, which are well-defined in [102]. The integrated state x_k includes orientation R, velocity v, and position p within the reference frame $\{W\}$ in the k-th frame. The integration from i-th frame to j-th frame is formulated as:

$$\begin{aligned}
\Delta R_{ij} &= \int_{t\in[t_i,t_j]} {}^B\tilde{w}_k \; dt \\
\Delta v_{ij} &= \int_{t\in[t_i,t_j]} R_k {}^B\tilde{a}_k \; dt \\
\Delta p_{ij} &= \int\int_{t\in[t_i,t_j]} R_k {}^B\tilde{a}_k \; dt^2 \;,
\end{aligned} \tag{5.4}$$

where the ΔR_{ij}, Δv_{ij} and Δp_{ij} denotes the increments of rotation, velocity and position from frame i to frame j respectively. Given these increments, the j-th state of the robot ${}^W x_j$ in the world frame $\{W\}$ can be predicted through:

$$\begin{aligned}
{}^W R_j &= \Delta R_{ij} \otimes {}^W R_i \\
{}^W v_j &= {}^W R_i * \Delta v_{ij} + {}^W v_i \\
{}^W p_j &= {}^W R_i * \Delta p_{ij} + {}^W p_i + {}^W v_i \Delta t_{ij},
\end{aligned} \tag{5.5}$$

where the ${}^W R_i$, ${}^W v_i$, and ${}^W p_i$ are the state of i-th frame.

The propagation of the IMU covariance also follows the IMUs' kinematic model. Following the conventions established by Forster et al. [102], the covariance of the predicted state is characterized as: $[\delta\phi_k^T, \delta v_k^T, \delta p_k^T]^T \sim \mathbb{N}(0_{9\times1}, \Sigma_k)$, where the $\delta\phi_k$, δv_k and δp_k represents the zero-mean Gaussian noise for rotation, velocity and position. We apply a similar covariance propagation strategy as [102] to model the covariance matrix $\Sigma_{i,i}$ of the IMU integration from frame i to frame i+1 through iterative propagation. Starting from the initial covariance matrix

$\Sigma_{i,i} = 0_{9\times9}$, we predict the covariance matrix as:

$$\Sigma_{i,i+1} = A\Sigma_{i,i}A^T + B_g\text{diag}(\eta_i^{gyro})B_g^T + B_a\text{diag}(\eta_i^{acc})B_a^T, \tag{5.6}$$

where state transition matrix A is defined as:

$$A = \begin{bmatrix} \Delta R_{kk+1}^T & 0_{3\times3} & 0_{3\times3} \\ -\Delta R_{ik}({}^B a_k^\wedge)\Delta t & I_{3\times3} & 0_{3\times3} \\ -1/2\Delta R_{ik}({}^B a_k^\wedge)\Delta t^2 & I_{3\times3}\Delta t & I_{3\times3} \end{bmatrix}. \tag{5.7}$$

The B matrix addresses the frame-by-frame uncertainty of the acceleration and the angular velocity.

$$B_g = \begin{bmatrix} J_r^k \Delta t \\ 0_{3\times3} \\ 0_{3\times3} \end{bmatrix}, \quad B_a = \begin{bmatrix} 0_{3\times3} \\ \Delta R_{ik}\Delta t \\ 1/2\Delta R_{ik}\Delta t^2 \end{bmatrix}. \tag{5.8}$$

The covariance matrix is 9×9 matrix, and we denote it as:

$$\Sigma_{i,j+1} = \begin{bmatrix} \Sigma_{i,j+1}^r & 0_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & \Sigma_{i,j+1}^v & 0_{3\times3} \\ 0_{3\times3} & 0_{3\times3} & \Sigma_{i,j+1}^p \end{bmatrix}, \tag{5.9}$$

where $\Sigma_{i,j+1}^r, \Sigma_{i,j+1}^v \text{and} \Sigma_{i,j+1}^p$ are 3×3 matrix representing the covariance of rotation, velocity, and position respectively.

5.3.3 Loss Function and Training Strategy

To supervise the learnable model in AirIMU, we design the state loss (5.10) for noise correction and covariance loss (5.11) for uncertainty estimation. Given the differentiable integrator defined in (5.4) and (5.5), we train the denoising module $f_\pi(w, a)$ by supervising the state loss with the position, orientation, and translation in the world frame $\{W\}$.

$$\begin{aligned} L_r &= \|\log({}^W R_i \otimes {}^W \hat{R}_i)\|_h, \\ L_v &= \|{}^W v_i - {}^W \hat{v}_i\|_h, \\ L_p &= \|{}^W p_i - {}^W p_i\|_h. \end{aligned} \tag{5.10}$$

The $\|\cdot\|_h$ denotes the Huber function, chosen for its robustness in penalizing the large error terms, particularly in lengthy integration sequences. We transform the orientation error to log space in the loss function (5.10) to supervise the rotation in the manifold, which reduces the single value during training.

To supervise the covariance, we use a similar loss function derived in Russell et al. [59]. For

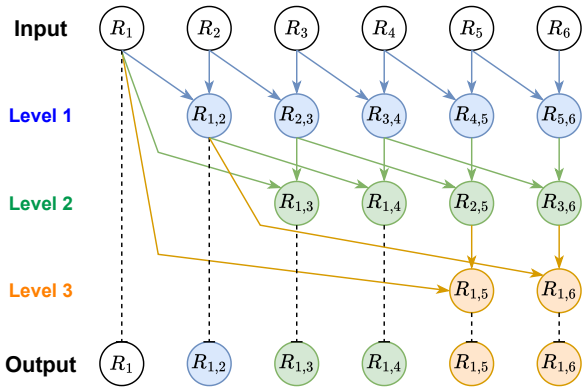

Figure 5.4: This graph presents the parallel scan we use to accelerate the cumulative product in our IMU integration. Given a sequence of gyroscope input from R_1 to R_n, the algorithm iterates through $\log(n)$ layers. In each layer i, it updates the last $N - 2^i$ elements by multiplying the first 2^i elements. In this algorithm, each layer is a parallel batch that supports parallel computation.

the covariance of rotation $\Sigma^r_{i,j}$, velocity $\Sigma^v_{i,j}$ and position $\Sigma^p_{i,j}$, we design the covariance losses as:

$$
\begin{aligned}
L_r^{cov} &= \frac{1}{2}\left(\left\|\log({}^W R_j \otimes {}^W \hat{R}_j)\right\|^2_{\Sigma^r_{i,j}} + \ln(\det \Sigma^r_{i,j})\right), \\
L_v^{cov} &= \frac{1}{2}\left(\left\|{}^W v_j - {}^W \hat{v}_j\right\|^2_{\Sigma^v_{i,j}} + \ln(\det \Sigma^v_{i,j})\right), \\
L_p^{cov} &= \frac{1}{2}\left(\left\|{}^W p_j - {}^W \hat{p}_j\right\|^2_{\Sigma^p_{i,j}} + \ln(\det \Sigma^p_{i,j})\right),
\end{aligned}
\tag{5.11}
$$

where y is the ground-truth label and $f(w, a)$ output the integrated state. The $\Sigma_\theta(x)$ denotes the covariance model that predicts the covariance matrix given the learned uncertainty η. To simplify and stabilize the training process, our training focuses on the diagonal components of the covariance matrix. The final loss function can be denoted as:

$$
L = L_r + L_v + L_p + \varepsilon(L_r^{cov} + L_v^{cov} + L_p^{cov}), \tag{5.12}
$$

where we set the weight of the covariance loss as $\varepsilon = 1 \times 10^{-3}$. We chose this parameter to mitigate the impact of relatively small covariance values, which otherwise lead to large covariance losses.

In contrast to previous methods that utilized data augmentation techniques such as random bias and rotation equivalence, we avoid applying data augmentation to sensor model training. Our approach preserves the original noise in the raw data, which is critical for the AirIMU model to learn the noise pattern in the real sensor. For most datasets, we set the length of the training segment as 1000 frames (5 seconds). This duration allows us to capture both the drift and the long-term features essential for IMU integration.

For the optimizer, we employ the Adam with a learning rate of 1×10^{-3} with a weight decay 1×10^{-4}. The weighting of each loss term and learning rate may be adjusted to accommodate the specific characteristics of different types of IMU, particularly with different bias instability of gyroscopes and accelerometers.

5.3.4 Implementation & Acceleration

In our training process, we extend the supervision period to enhance the effectiveness of the training, which is particularly crucial with high-end IMUs that only exhibit drift after long periods of integration. While extending the training over long segments can be time-consuming, our AirIMU model addresses this challenge by fully leveraging parallel computing capabilities powered by *PyTorch*. In this section, we introduce the algorithm we designed to accelerate the cumulative product and cumulative summation to facilitate parallel computation during training. Additionally, we reformulate the iterative covariance propagation to enable batched operation for long segments. These efforts not only accelerate the training process but also optimize the efficiency of real-time applications.

Specifically, the IMU preintegration and covariance propagation defined in (5.4) and (5.6) relies heavily on the cumulative product of $SO3$ on manifolds and cumulative summation of the acceleration. To accelerate the integration, we design a parallel scan algorithms as shown in Figure 5.4, where each node represents an element in $SO3$. In this implementation, the integration with a length of N can be separated into $\log_2(N)$ iterations. During each iteration i, it updates each node with the preceding node positioned 2^i place before it, which can be parallel computed. This approach achieves $O(\log N)$ time complexity while maintaining the $O(N)$ memory usage.

Different from the segment tree strategy proposed by Brossard et al. [129], which predicts only the final state $R_{1,6}$ of the gyroscope integration, our approach is designed to calculate every intermediate result throughout the integration within the same level of time. The intermediate results significantly accelerates the calculation of the matrix of A, and thereby accelerates the covariance propagation. Moreover, we not only use this approach in gyroscope integration, but also for accelerometer integration and the covariance propagation. Overall, we integrate this algorithm into two operators: the `cumsum` for the cumulative summation and the `cumprod` for the cumulative product of the $SO3$ on manifolds.

The conventional solution outlined in (5.6) iteratively calculates the covariance of each state. This iterative approach, while methodically sound, requires performing covariance propagation thousands of times, even for supervising brief segments lasting a few seconds. Such an iterative method is inefficient for training, leading to significant computational burdens. To address this inefficiency and fully utilize the parallel computing capabilities, we reformulate the covariance propagation for $\Sigma_{i,j}$ to two steps.

First of all, we calculate the matrix list $C^A_{i,j}$ with $\prod_{k=j}^{i} A_k$ through the `cumprod` of the matrix A_k, and the matrix list $C^B_{i,j}$ consisting the initial covariance $\Sigma_{i,i}$ and the matrix B.

$$\begin{aligned} C^A_{i,j} =&[\prod_{k=j}^{i} A_k, \prod_{k=j}^{i+1} A_k, ..., A_j, I_{9\times 9}], \\ C^B_{i,j} =&[\Sigma_{i,i}, B_i, ..., B_{j-1}, B_j]. \end{aligned} \tag{5.13}$$

Secondly, the covariance $\Sigma_{i,j+1}$ can be calculated by the product over $C^A_{i,j}$ and $C^B_{i,j}$:

$$\Sigma_{i,j+1} = C^A_{i,j}\text{diag}(C^B_{i,j}){C^A_{i,j}}^T. \tag{5.14}$$

With this re-implementation, we are now able to supervise the IMU integration over extended

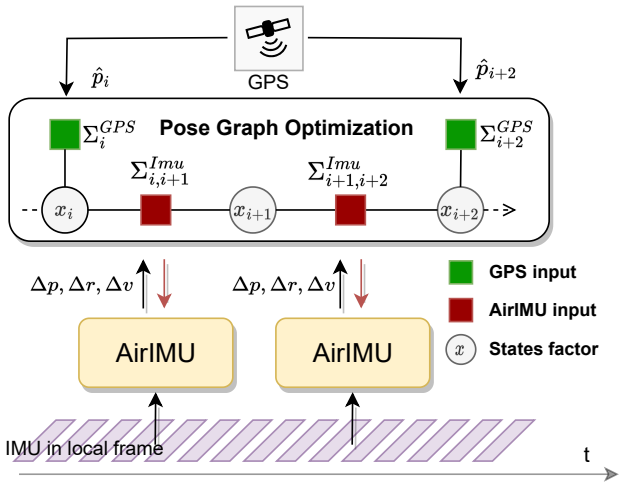

Figure 5.5: Pose graph optimization of a GPS signal with IMU pre-integration. The state of the graph is defined as x_i constrained by two different observations: IMU (Δp, Δv and Δr) and GPS ($\hat{p}_i$). We back-propagate the gradient through the IMU pre-integration and covariance propagation to learn the integration and noise model.

sequences. For instance, in the helicopter navigation scenario described in Sec. 5.4.6, our model is trained with unusually long segments that exceed one minute in duration. This extended supervision period captures the drift inherent in the navigation-grade IMU after long-term integration, which is crucial for achieving reliable navigation in aerial environments.

5.3.5 IMU-GPS PGO System

To recover the inevitable drift from inertial odometry, auxiliary sensors like GPS are employed to fuse with the IMU measurements in the PGO. When fusing the IMU measurements with other sensor, the covariance of the IMU integration $\Sigma_\theta(x)$, plays a critical role in the fusion. In this section, we describe the construction of a PGO that incorporates both the AirIMU model and GPS. We demonstrate how leveraging the learned covariance from AirIMU enhances the optimization process. This integrated system not only compensates for the drift inherent in IMU pre-integration but also effectively utilizes the complementary sensors, resulting in more reliable and accurate navigation results.

In Figure 5.5, we show the system pipeline and the factor graph incorporating GPS signals. The objective of the PGO is to optimize the state of the robot x, which is modeled as the factor in our PGO. From the AirIMU, we take the pre-integrated increments (Δp, Δv, and Δr) as the observation. These observations are then propagated alongside the initial state $x_i(R_i, v_i, p_i)$ to calculate the state $x_j(R_j, v_j, p_j)$, forming the basis to compute the IMU residual:

$$e_{ij}^{IMU}(\Delta v_{ij}, \Delta p_{ij}, \Delta r_{ij}) = \begin{bmatrix} \Delta R_{ij}\, {}^W R_i ({}^W R_j)^{-1} \\ {}^W v_i + {}^W R_i * \Delta v_{ij} - {}^W v_j \\ {}^W p_i + {}^W R_i * \Delta p_{ij} - {}^W p_j \end{bmatrix}. \tag{5.15}$$

To calculate the corresponding covariance matrix Σ_{ij}^{IMU} of the IMU integration. We leverage the predicted uncertainty from the network $[\eta_k^{acc}, \eta_k^{gyro}] = \Sigma_\theta(w, a)$ and employ it in (5.6) to

Table 5.1: Dataset summary showing the comprehensive evaluation across different IMU grades and platforms

Datasets	**Duration**	**IMU**	**Modality**
EuRoC [97]	22m29s	ADIS16448	Drone
TUM-VI [133]	13m31s	BMI160	Handheld
SubtMRS [96]	2h52m	Epson M-G365	Ground robot
KITTI [98]	43m44s	OXTS RT 3000	Vehicle
ALTO [119]	2h12m	NG LCI-1	Helicopter

propagate the covariance matrix Σ_{ij}^{IMU}. Since the initial state x_i is updated after each optimization, we also update the corresponding covariance $\Sigma_{i,i}$ of the i^{th} state during the optimization. This ensures the covariance of the state is bounded, giving the other sensors' measurements and advancing the reliability.

We define the GPS signal as $\hat{p}_i \in R^3$ to constrain the state of position ${}^W p_i$, where the residual of GPS measurements are:

$$e_i^{GPS}({}^W\hat{p}_i) = {}^W\hat{p}_i - {}^W p_i \tag{5.16}$$

where the measurement of the GPS is ${}^{\hat{W}} p_i$ with a corresponding covariance matrix defined as a diagonal matrix Σ_i^{GPS}. The optimization with both the $e_i^{GPS}(\hat{p}_i)$ and $e_{ij}^{IMU}(\Delta v_{ij}, \Delta p_{ij}, \Delta r_{ij})$ can be formulated as:

$$X^* = \arg\min_X \left\{ \sum_{k \in G} \|e_k^{GPS}\|^2_{\Sigma_k^{GPS}} + \sum_{l \in L} \|e_l^{IMU}\|^2_{\Sigma_l^{IMU}} \right\} \tag{5.17}$$

where G and L is the set of GPS and IMU observations and the X is the factor. We can solve this optimization by Levenberg-Marquardt algorithms.

5.4 Experiments

As shown in Figure 5.2, we comprehensively evaluate our method across a full spectrum of IMUs, from low-cost automotive-grade to high-end navigation-grade IMUs, each featured on different platforms and modalities. The price and gyro in-run bias stability are plotted, revealing a clear correlation between cost and performance stability. At the high end of the spectrum, we have the `NG LITEF LCI-1` in the ALTO dataset [2] with a cost exceeding $50,000, showcasing exceptional bias stability in helicopters. The tactical-grade IMUs, like the `OXTS RT3003` in the KITTI dataset [98] and the `Epson M-G365` in the Subt-MRS [96], are priced at approximately $25,000 and $1,400, deployed on UGVs and vehicles respectively. On the low-cost end, we feature the industrial-grade IMU `BMI 160` on drones at around $500 and the automotive-grade IMU `ADIS 16448` at $3 used in handheld platforms, demonstrating the feasibility of using low-cost IMUs for certain applications. These datasets evaluate the adaptability of our model not only across multiple grades of IMUs but also across various platforms and modalities.

In our experiments, we demonstrate the dual purposes of the AirIMU: state estimation and

uncertainty estimation.

State Estimation: We demonstrate the AirIMU model's superior performance in IMU integration, outperforming both learning-based IMU de-noising algorithms and traditional IMU calibration algorithms. Notably, the AirIMU model achieves a tenfold increase in accuracy compared to learning-based inertial odometry methods in short-term integration. We also demonstrate the potential of the AirIMU model in GPS-denied helicopter navigation using high-end IMUs. This assessment spans a cruising distance over 262 km, providing a realistic and challenging environment for evaluating our method.

Learned Uncertainty: We assess our model with a particular emphasis on the potency of the learned uncertainty. An ablation study focused on a GPS-PGO experiment reveals an average improvement of 31.6% in utilizing the learned uncertainty. Furthermore, we conduct an ablation study to highlight the benefits of jointly training the uncertainty module with the IMU noise correction. This dual training approach significantly improves the network's ability to extract features from IMUs, resulting in a performance enhancement of 17.8% compared to models solely trained on noise correction.

5.4.1 Evaluation Metrics

We define the following metrics to evaluate the performance of our model over assigned time intervals:

Rotational Root Mean Squared Error (R-RMSE, rad),

$$\text{R-RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|\log\left(\hat{R}_{i,i+\Delta t}^{T} R_{i,i+\Delta t}\right)\right\|_2^2}, \tag{5.18}$$

calculates the RMSE of the rotations over the assigned time intervals on the manifold space.

Positional Root Mean Squared Error (P-RMSE, m)

$$\text{P-RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|p_{i+\Delta t} - p_i - R_i\hat{R}_i^{T}\left(\hat{p}_{i+\Delta t} - \hat{p}_i\right)\right\|_2^2}, \tag{5.19}$$

calculates the RMSE of the relative displacements over the assigned time intervals.

Absolute Translation Error (ATE, m)

$$\text{ATE} = \frac{1}{N}\sum_{i=1}^{N}\|p_i - \hat{p}_i\|_2, \tag{5.20}$$

calculates the mean of absolute distance between the estimated positions and the ground truth over all time points.

To establish a baseline for comparison, we implement the IMU pre-integration using PyPose [63] based on the work of Forster et al. [102], which we denote as **Baseline**. This serves as a reference for evaluating the performance of IMU integration with raw data. In our experiments, each dataset contains IMUs with diverse mechanical conditions and modalities. Since each sensor

Table 5.2: The ROE (Unit: °) and RPE (Unit: meter) of IMU Pre-integration over 1 second (200 frames) on the TUMVI dataset.

Seq.	Baseline		Brossard et al.[129]		Kalibr [105] + Baseline		AirIMU	
	ROE	RPE	ROE	RPE	ROE	RPE	ROE	RPE
Room 2	2.3161	0.7652	0.7075	-	0.7006	0.0785	**0.6765**	**0.0770**
Room 4	2.8239	0.7558	0.4460	-	0.4397	0.0571	**0.3930**	**0.0540**
Room 6	2.3407	0.8521	0.4029	-	0.3923	0.4096	**0.3743**	**0.4093**
Avg.	2.4936	0.7910	0.5188	-	0.5109	0.1817	**0.4813**	**0.1801**

features a distinct noise model, training a universal model that can generalize across all IMUs and platforms without compromising accuracy is infeasible. Therefore, we train distinct sensor models tailored to different IMUs.

5.4.2 IMU Pre-integration Accuracy

In this section, we evaluate the accuracy of the AirIMU model in IMU pre-integration. We compare against two existing learning-based IMU denoising methods: Zhang et al. [118] and Brossard et al. [129] on the EuRoC dataset [97]. Additionally, we showcase how our method enhances the integration precision of the traditional calibration algorithm Kalibr [105] when tested on the TUM-VI dataset [133].

TUM-VI Dataset. As described in Sec. 5.2.1, traditional methods typically employ a linear model to calibrate IMU intrinsic, such as bias. In the TUM-VI dataset [133], IMU was calibrated using Kalibr [105] as a reference. We leverage the calibrated data to examine whether our AirIMU model can further improve traditional IMU calibration methods.

To benchmark the TUM-VI dataset, we select `room 1`, `room 3`, and `room 5` for training, while using the remaining sequences for evaluation. To ensure fairness, we train Brossard et al.'s model with the same training set and show the results on the same evaluation set. We evaluate the ROE and RPE over 1-second intervals to assess gyroscope and accelerometer integration.

In the experiment, Kalibr effectively corrects sensor offsets using linear models. Grounded on these results, AirIMU improves upon this by modeling the non-deterministic error in a data-driven manner, demonstrating a further improvement compared to the Kalibr. As shown in Table 5.2, AirIMU exhibits robustness and superiority in improving IMU integration accuracy. For the orientation, the average ROE of AirIMU is 0.4813°, which is superior to 2.4936° from the raw-data integration, 0.5188° from Brossard et al. and 0.5109° from Kalibr. For relative translation, AirIMU achieves an average RPE of 0.1801m, better than the 0.7910m from raw-data integration and the 0.1817m from Kalibr.

EuRoC Dataset. In this section, we evaluate the pre-integration performance of AirIMU with existing learning-based IMU de-noising methods [118, 129]. These studies were assessed using different visual-inertial odometry and evaluation benchmarks. Specifically, Zhang et al. passed the preprocessed IMU measurements to their own visual-inertial algorithm for evaluation. On the other hand, Brossard et al. passed their corrections to the Open-VINS framework [134] for visual-inertial fusion. As a result, it is infeasible to directly compare absolute positional error, which makes it difficult to assess the effectiveness of the IMU denoising module.

Considering these differences, in our evaluation, we compare each method separately using the benchmarks proposed in their papers, respectively. Zhang et al. reported the results of P-RMSE and R-RMSE over ten frames of IMU measurements. Thus, we evaluate 10-frame IMU pre-integration results in Table 5.3. For Brossard et al., we show both the R-RMSE and ROE in Table 5.4. To ensure fairness, we split the training and testing set in the same way as Zhang et al. and Brossard et al.

Table 5.3: IMU integration result of 10 frames on EuRoC Dataset with P-RMSE (Unit: cm) and R-RMSE (Unit: °)

Seq.	Zhang et al. [118]		Baseline		AirIMU	
	R-RMSE	P-RMSE	R-RMSE	P-RMSE	R-RMSE	P-RMSE
MH02	0.143	0.280	0.230	0.046	**0.020**	**0.040**
MH04	0.212	0.580	0.229	0.326	**0.025**	**0.325**
V103	0.384	0.530	0.231	0.048	**0.036**	**0.042**
V202	0.390	0.830	0.240	0.028	**0.037**	**0.027**
V101	0.928	0.300	0.228	0.030	**0.021**	**0.020**
Avg.	0.411	0.504	0.232	0.096	**0.028**	**0.091**

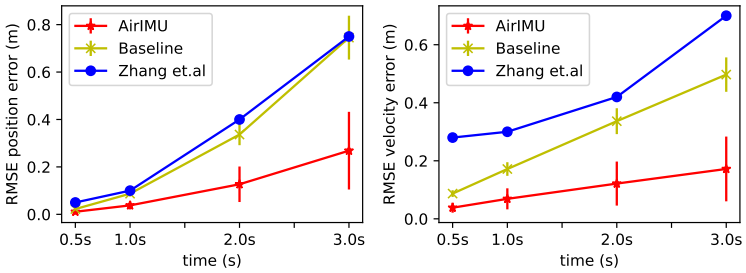

Figure 5.6: We present the RMSE of both translation and velocity over intervals of 0.5s, 1s, 2s, and 3s. The results illustrate the accumulation of errors throughout the integration, where AirIMU remains accurate after integration.

Notably, our method achieves a ten-fold improvement in accuracy compared to the reported P-RMSE and R-RMSE for short-term integration. Moreover, Zhang et al. provided P-RMSE values over specific time intervals: 0.5, 1, 2, and 3 seconds. Their reported values were approximately 0.05, 0.10, 0.40, and 0.75 meters, respectively. In contrast, using the same intervals, our method achieved P-RMSE values of 0.011, 0.038, 0.127, and 0.27 meters. This translates to an average improvement of three times in terms of positional accuracy. Additionally, we present the velocity RMSE results in Figure 5.6.

Since Brossard et al. reported their findings using ROE while Zhang et al. reported R-RMSE, we show both the R-RMSE and ROE in our evaluation for consistency with prior experiments. Results over the one-second intervals (200 frames) are shown in Table 5.4.

Table 5.4: Gyroscope integration on EuRoC Dataset, we show the R-RMSE and the ROE (Unit: °).

Seq.	Baseline		Brossard et al. [129]		AirIMU	
	RMSE	ROE	RMSE	ROE	RMSE	ROE
MH02	4.5800	4.5799	0.1255	0.0871	**0.0978**	**0.0789**
MH04	4.5406	4.5391	0.3556	0.1067	**0.0836**	**0.0708**
V103	4.4909	4.4870	0.2181	0.1935	**0.2107**	**0.1884**
V202	4.7000	4.6924	0.2595	0.2389	**0.2366**	**0.2157**
V101	4.5275	4.5252	**0.1346**	**0.1173**	0.1413	0.1241
Avg.	4.5678	4.5647	0.2189	0.1487	**0.1305**	**0.1127**

In Table 5.4, AirIMU shows superior accuracy and reliability on IMU gyroscope integration. The Baseline's average R-RMSE and ROE are 4.5678° and 4.5647°, respectively. These errors are primarily due to a significant gyroscope offset in the EuRoC dataset, which causes notable drift. In contrast, AirIMU demonstrates significantly better performance with R-RMSE and ROE values of 0.1305° and 0.1127°, respectively, representing a ten-fold improvement over the Baseline. Moreover, when compared to Brossard et al., AirIMU consistently achieves better results across most sequences, with improvements of 40.3% in R-RMSE and 24.2% in ROE on average.

5.4.3 Learning-based Inertial Odometry Comparison

In Sec. 5.2.2, we identified two main approaches to data-driven IO. The first utilizes end-to-end trained networks to estimate the velocity. Within this category, we examine the estimation results derived from the ResNet, which is prevalently employed in methods such as RoNIN [3]. The second approach involves EKF-fused inertial odometry like TLIO [115]. When integrating the neural network's predictions with IMU signals, the EKF in TLIO adeptly addresses system biases, enhancing accuracy.

In the pedestrian dataset, the subject is highly correlated with the motion patterns. As the walking pace typically shows periodic acceleration cycles and the velocity of the pedestrian is constant in most scenarios, models like RoNIN and TLIO can explicitly estimate the subject's velocity by motion patterns. However, this assumption does not hold in autonomous driving datasets like KITTI and UAV navigation datasets like EuRoC, since both vehicles and drones exhibit different velocities and multiple moving patterns. Therefore, we compare our model against existing learning inertial odometry algorithms, including RoNIN and TLIO, on the EuRoC and

Table 5.5: The P-RMSE of IMU integration over 1 second (200 frames) on EuRoC dataset (Unit: meter).

Seq.	Baseline	RoNIN [3]	TLIO[115]	AirIMU
MH02	0.1899	0.5186	0.3030	**0.0234**
MH04	0.1904	1.0377	0.8230	**0.0415**
V103	0.2116	0.7387	0.6530	**0.0583**
V202	0.1520	0.7497	0.5090	**0.0851**
V101	0.1903	0.4964	0.3430	**0.0363**
Avg.	0.1868	0.7082	0.5262	**0.0489**

KITTI datasets.

EuRoC Dataset. We assessed the P-RMSE over an interval of 200 frames (equivalent to 1s), comparing these values to the short-term integration outcomes from AirIMU and Baseline in Table 5.5. Compared to the output from the directly regressed neural network, the pre-integration results from AirIMU demonstrate a substantially greater accuracy — by a factor of 17. This distinction is especially noticeable when we consider the drone data. Unlike the pedestrian dataset that possesses clear motion patterns or modalities, the dynamics of the drone remain unpredictable, making it challenging for the network to accurately predict the velocity.

KITTI Dataset. To demonstrate the generalizability across different collection dates, we employ our algorithms on the KITTI dataset[98], renowned for its state estimation benchmarks with vision, IMU, and LiDAR. We train the AirIMU on the KITTI Odometry datasets, specifically collected on `2011_09_30` and `2011_10_03`. To evaluate the data from different collection dates, we conduct the quantitative analysis and qualitative comparisons on the KITTI raw datasets collected from `2011_09_26` and `2011_09_28`. This setup ensures the AirIMU's robustness across temporal variations.

To adjust the low IMU frequency characteristic of the KITTI dataset, the GRU layer in the AirIMU model is removed. Furthermore, we set the window size for pre-integration to 5 seconds during the training, introducing supervision on long-term integration. Notably, the trajectories 0016 (from `2011_09_28`) and 0057 (from `2011_09_26`) are removed from the test dataset due to the misaligned ground truth poses verified by manual inspection.

The ROE and RPE for Baseline, RoNIN, and AirIMU on the test sequences are shown in Table 5.6. Quantitative results show that the AirIMU outperforms the Baseline method on most

Table 5.6: The ROE (Unit: °) and RPE (Unit: meter) of IMU integration over 1 second (10 frames) on the test dataset. RoNIN refers specifically to the ResNet-50 model[3].

Seq.	Baseline		RoNIN		AirIMU	
	ROE	RPE	ROE	RPE	ROE	RPE
0014	0.4137	0.1785	-	12.2424	**0.3927**	**0.1699**
0018	0.2451	**0.0715**	-	3.8301	**0.2388**	0.0857
0022	0.5319	0.2893	-	4.9544	**0.4912**	**0.2841**
0029	0.3435	0.393	-	6.6598	**0.3245**	**0.3562**
0035	0.3682	0.0969	-	2.0726	**0.3563**	**0.0956**
0039	**0.5273**	0.1703	-	4.4768	0.5514	**0.1613**
0106	0.2052	0.1568	-	8.8109	**0.2026**	**0.1512**
Avg.	0.3764	0.1938	-	4.8909	**0.3653**	**0.1863**

trajectories and demonstrates a more than 20 times lower RPE than the RoNIN. Remarkably, the RoNIN model shows an exceptionally high error compared to the Baseline method. This is because the vehicle's speed changes rapidly, which makes it difficult for the network to predict the velocity.

The trajectory 0022 from `2011_09_26` estimated by Baseline, RoNIN, and AirIMU, is plotted in Figure 5.7 along with the velocity error over the entire trajectory for qualitative analysis. The trajectory estimated by AirIMU significantly reduces the velocity error compared to the Baseline method. On the other hand, the Baseline tends to quickly accumulate errors in both velocity and orientation when the vehicle is turning, while AirIMU alleviates this problem. We also highlight the deficiency of learning-based IO in the purple trajectory. Due to the lack of prior knowledge of initial velocity, the RoNIN model shows multiple unexpected sharp turns on straight-motion segments, while AirIMU maintains its accuracy.

Combining the data-driven module and kinematic model, AirIMU outperforms the learning IO over a long sequence in KITTI. In the demonstrated trajectory, the ATE estimated by AirIMU (30.40 m) is only half of that estimated by the Baseline method (60.14 m), and the RoNIN model (48.19 m).

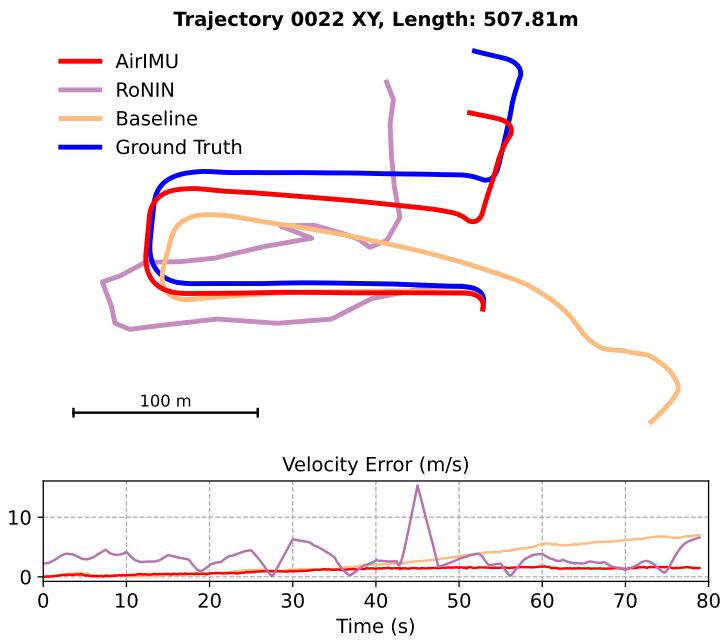

Figure 5.7: Trajectory 0022 in KITTI-Raw dataset estimated by RoNIN (purple), Baseline (orange) and AirIMU (red). The graph below shows the accumulated velocity error (Unit: m/s) through the entire trajectory. AirIMU significantly alleviates the velocity error accumulation on the y-axis compared to the Baseline. As a result, AirIMU maintains its accuracy after integration, while the Baseline method exhibits noticeable drift during the third turn.

5.4.4 IMU-GPS PGO

The uncertainty model of the IMU pre-integration is pivotal when fusing the IMU measurements with other sensors, such as the GPS. As shown in Table 5.3, our AirIMU model can achieve high accuracy in the short-time pre-integration. The integrated trajectory, however, will drift in long-term integration due to the unavoidable noise inherent in the sensor. Therefore, when fusing the IMU pre-integration with low-frequency auxiliary measurements, the propagated covariance model determines how well the optimization can trust the IMU measurements. We conduct an ablation study with different IMU sensor models and covariance setups to assess the effectiveness of our learned uncertainty proposed in the AirIMU model. In the ablation study, we utilize the GPS-PGO system we formulated in Figure 5.5, where GPS was considered as an unary constraint to the factor.

A comprehensive evaluation of the IMU covariance requires controlling the noise generated from the fused sensors. To this end, we generate simulated GPS signals with Gaussian noise of $\sigma_{GPS} = 0.1$ m at a frequency of 1 Hz in Table 5.7. This setup aims to replicate a typical RTK-GPS signal and allows us to evaluate the baseline performance of the AirIMU covariance under standard operating scenarios. Furthermore, to explore the model's efficacy in less ideal conditions, we also simulate GPS signals at a lower frequency of 0.1 Hz in Table 5.8. This setup is designed to test the effectiveness of our learned uncertainty model when faced with sporadic or diminished GPS data inputs.

The PGO experiment uses the following settings:

Table 5.7: Ablation study of the GPS PGO with 1 Hz on the EuRoC dataset with different covariance settings.

Seq.	MH02	MH04	V103	V202	V101
Raw IMU w/ VinsMono Cov	0.1493 ± 0.0044	0.1491 ± 0.0050	0.1549 ± 0.0058	0.1542 ± 0.0051	0.1478 ± 0.0041
AirIMU w/ VinsMono Cov	0.1484 ± 0.0044	0.1486 ± 0.0049	0.1547 ± 0.0057	0.1540 ± 0.0049	0.1469 ± 0.0039
Raw IMU w/ learned Cov	0.1184 ± 0.0041	0.1327 ± 0.0048	0.1310 ± 0.0051	0.1569 ± 0.0078	0.1265 ± 0.0055
AirIMU w/ learned Cov	**0.1026 ± 0.0035**	**0.1024 ± 0.0039**	**0.1039 ± 0.0028**	**0.1060 ± 0.0046**	**0.1019 ± 0.0041**

Results show the mean and the standard deviation with ATE in meter m.

Table 5.8: Ablation study of the GPS PGO with 0.1 Hz on the EuRoC dataset with different covariance settings.

Seq.	MH02	MH04	V103	V202	V101
Raw IMU w/ VinsMono Cov	2.1454 ± 0.0150	3.6112 ± 0.0255	1.4474 ± 0.0309	1.6840 ± 0.0165	1.4740 ± 0.0178
AirIMU w/ VinsMono Cov	1.9235 ± 0.0181	0.2955 ± 0.0270	0.3085 ± 0.0289	**0.8681 ± 0.0214**	0.3455 ± 0.0161
Raw IMU w/ learned Cov	2.5088 ± 0.0368	4.3450 ± 0.0280	1.4248 ± 0.0227	2.1213 ± 0.0184	1.1941 ± 0.0141
AirIMU w/ learned Cov	**0.2414 ± 0.0240**	**0.2754 ± 0.0274**	**0.2905 ± 0.0216**	0.8927 ± 0.0189	**0.2948 ± 0.0144**

Results show the mean and the standard deviation with ATE in meter m.

(a) Raw IMU with covariance from VinsMono's setting [6].
(b) AirIMU with the covariance from the VinsMono's setting.
(c) Raw IMU with proposed learning covariance model.
(d) AirIMU with proposed learning covariance model. The proposed covariance model of **(5.4.4)** is propagated through the Equation. 5.6 with the learned uncertainty model in $\Sigma_\theta(w, a)$. The covariance matrix of **(5.4.4)** is calculated by the VinsMono's EuRoC configuration, where the noise standard deviation of accelerometer $\eta^{\text{acc}} = 0.08$ and the noise standard deviation of gyroscope $\eta^{\text{gyro}} = 0.004$. These parameters are later applied in the equation 5.6 to calculate the covariance matrix of the IMU pre-integration Σ^{IMU}. The covariance matrix of the GPS signals is defined to be the inverse of a diagonal matrix:

$$\Sigma_k^{GPS} = \text{diag}([1/\sigma_{GPS}^2, 1/\sigma_{GPS}^2, 1/\sigma_{GPS}^2])$$

To ensure a fair comparison, we repeated the experiment 10 times with randomly generated GPS signals. The average ATE and standard deviation are summarized in Table 5.8 and Table 5.7. Compared to **(5.4.4) AirIMU with VinsMono** setups, **(5.4.4) AirIMU model with its learned covariance** significantly enhances the ATE of the optimization results with 21.6%. Compared to **(5.4.4) raw IMU with VinsMono's covariance**, **(5.4.4) AirIMU with the learned covariance** reduces the ATE from 0.1505 to 0.1033, improving 31.6%.

In Table 5.8, we explore the impact of unstable GPS update conditions, specifically 1 GPS reading every 10 seconds. Due to this low GPS frequency (0.1 Hz), IMU must be integrated over long intervals, leading to an accumulation of integration errors. This presents significant challenges for sensor fusion, affecting overall system accuracy. **(5.4.4) AirIMU with learned covariance** reduces the error from 2.0724 to 0.3990, improving 80.7% in ATE.

In Figure 5.8, we plot the optimized trajectory of `MH04` and `MH02`, where the drone flies through the machine hall and returns to its original position. The yellow trajectory **(5.4.4)** and

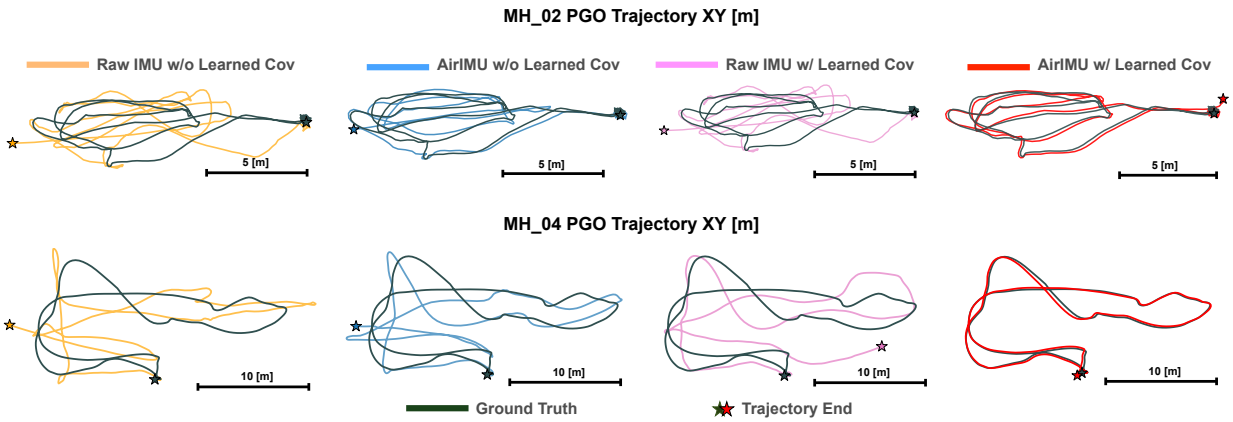

Figure 5.8: Qualitative analysis of IMU GPS PGO results. In this experiment, the GPS frequency is set to 0.1 Hz. Despite using identical IMU measurements, AirIMU with a learned covariance yields optimal fusion results.

the blue trajectory (5.4.4), which employ referenced covariance parameters, have a large drift in the experiment. In these two trajectories, the orientation of the trajectory endpoint is the opposite compared to the ground truth. In contrast, the pink trajectory (5.4.4), which employed learned covariance with raw IMU data, shows a marked improvement in navigational accuracy and a smaller return drift. Overall, our model (5.4.4) with the learned covariance illustrates the best optimization performance, maintaining high accuracy and successfully returning to the starting position. These results demonstrate the decisive role of our learned covariance in the PGO.

5.4.5 Ablation Study on Multi-robot Underground Dataset

In this section, we demonstrate the synergistic effect achieved through the joint training of the covariance model and the correction model. We evaluate our model on the SubT-MRS dataset[96], a multi-robot, multi-modal, and multi-degraded challenging dataset designed to push SLAM towards all-weather environments.

Our experiments selected the multi-agent UGV datasets from the LiDAR-inertial track, collected in the DARPA Subterranean (SubT) Challenge's Final Event and Urban Circuit Event. These agents' platform is equipped with Epson M-G365 IMU. In the Final Challenge, three agents (UGV1, UGV2, UGV3) explored three uncharted underground environments: tunnel, urban, and cave spaces. In the Urban Challenge, two agents (UGV1 and UGV2) explored an unfinished nuclear power plant. We divided the data from different agents into sequences based on the operational scenarios, such as corridors, warehouses, and long-time stops. The duration of each sequence ranges approximately from 4 to 10 minutes.

To quantify the effectiveness of the covariance-aware module in the IMU noise corrections, we conduct experiments with several different setups: **(i) Baseline:** This setup serves as our control, integrating the raw IMU data. It helps to benchmark the other setups.
(ii) Average Bias: We correct the IMU data by calibrating the biases in the accelerometer and gyroscope from the training dataset. This strategy is widely adopted in traditional IMU calibration algorithms, setting a foundation for comparing with learning-based methods.
(iii) AirIMU w/o Cov: In this configuration, the AirIMU network is solely trained to correct

Table 5.9: The RPE (Unit: meter) of IMU Pre-integration over 5 seconds (1000 frames) on SubT-MRS.

Datasets	Robots	Sequences	Baseline	Average Bias	AirIMU (w/o Cov)	AirIMU
Final Challenge	UGV1	warehouse 1	2.7934	**1.8828**	2.2540	2.2306
	UGV2	tunnel room	1.0965	1.0578	**0.7447**	0.7568
		tunnel stop 4	1.0169	1.1519	**1.0280**	1.0709
		tunnel corridor 3	1.4248	1.2095	**0.8479**	0.8745
	UGV3	cave corridor 3	3.5067	1.8609	1.8290	**1.6627**
		tunnel corridor 5	2.7762	1.3399	1.4243	**1.2495**
Urban Challenge	UGV1	factory 2	4.1769	2.9747	2.8173	**2.6525**
	UGV2	factory 4	2.8622	2.3964	2.2895	**1.3393**
		factory 6	1.6454	2.0282	1.5293	**0.9805**
		factory 9	2.1193	2.8264	1.7837	**0.7534**
	Average		2.3418	1.8729	1.6517	**1.3571**

IMU noise without supervising the covariance explicitly. This setup tests the network's basic capabilities in noise correction.
(iv) AirIMU with Cov: This advanced setup explicitly supervises the uncertainty in the IMU integration. It demonstrates the synergistic effect of jointly training the covariance module with the IMU noise correction.

During the training stage, we randomly segment the sequences and group them with the same robots in the Urban Challenges and the Final Challenges into training sets and testing sets. Since each robot in this dataset had a different calibration process before the challenge, we cannot fit a general IMU model for every sequence. Therefore, we train different models for different robots in each challenge. Unlike the previous experiment, we don't prioritize the rotation error because the gyroscope of the Epson IMU has relatively good in-bias stability. So, we pick an equal weight ratio of $1:1:1$ for position, velocity, and rotation losses.

In Table 5.9 and Table 5.10, jointly training AirIMU with covariance propagation **(5.4.5) AirIMU W/ Cov** demonstrates the best result among the experiment groups. This improvement may be attributed to the fact that supervising the covariance alongside the correction module improves the network's ability in feature extraction, and thereby benefits each other. Compared to **(5.4.5) Baseline**, our proposed **(5.4.5) AirIMU w/ Cov** has an average improvement of 72.6% in ROE and 42.1% in RPE. It outperforms the **(5.4.5) Average Bias** for 1.69% in ROE and 27.5% in RPE. Remarkably, compared to **(5.4.5) AirIMU W/O Cov**, it improves 12.6% in the ROE and 18.2% in the RPE on average.

In Figure 5.9, we qualitatively analyze the sequence `factory 9` and show the RPE and ROE across the entire trajectory. In this sequence, the vehicle starts in the factory and ends in the

Table 5.10: The ROE (Unit: °) of IMU Pre-integration over 5 seconds (1000 frames) on SubT-MRS.

Datasets	Robots	Sequences	Baseline	Average Bias	AirIMU (w/o Cov)	AirIMU
Final Challenge	UGV1	warehouse 1	2.5365	0.3604	0.4908	**0.3558**
	UGV2	tunnel room	0.3887	0.1875	0.1796	**0.1731**
		tunnel stop 4	0.3681	0.0853	0.0877	**0.0692**
		tunnel corridor 3	0.4028	0.2125	0.1982	**0.1907**
	UGV3	cave corridor 3	2.1469	**0.3104**	0.4129	0.3330
		tunnel corridor 5	2.1634	0.3597	0.4806	**0.3588**
Urban Challenge	UGV1	factory 2	0.6389	**0.3302**	0.3425	0.4613
	UGV2	factory 4	0.9198	0.7144	0.7188	**0.5285**
		factory 6	0.4585	**0.1648**	0.1735	0.2157
		factory 9	0.4618	0.1969	0.2018	**0.1865**
	Average		1.0485	0.2922	0.3286	**0.2873**

storage room for 3 minutes, where it remains stationary, waiting for further instruction. During the stationary period, the IMU shows a different bias pattern compared with the other sequences that are in motion. In the long-time stop period, the **(5.4.5) Average Bias** degrades the performance compared to the **(5.4.5) Baseline** with the raw-IMU integration. This is because the sensors' bias in the training set doesn't have the same distribution as the testing sequences. During the calibration stage, the bias model memorizes the average offset of the training set. But when the robot stops, the IMU has a different bias distribution. While our proposed AirIMU model significantly reduces the positional error. This experiment demonstrates the stability and accuracy of AirIMU, especially when the testing sequences have different distributions from the training sets. Moreover, the ablation study shows the synergistic effect of covariance supervision and noise correction in AirIMU.

5.4.6 Large-scale Helicopter IMU Pre-integration

Many existing learning-based IOs are evaluated indoors or on small-scale datasets with limited duration and speeds. To bridge this gap, we employed Visual Terrain Relative Navigation dataset ALTO[2] from a Bell 206L (Long-Ranger) helicopter to evaluate the performance of the AirIMU model with high-end navigation grade IMU. As shown in Figure 5.10, this dataset encompasses flights over 414 km, featuring average speeds over 150 $\mathrm{km\,h^{-1}}$ and cruising heights between 200 and 500 meters. This presents a robust testing ground for the AirIMU model's capabilities.

During the model training, we select Flight B as the training set and Flight A as the testing set, where each flight in this dataset has a duration exceeding 40 minutes. Given the long

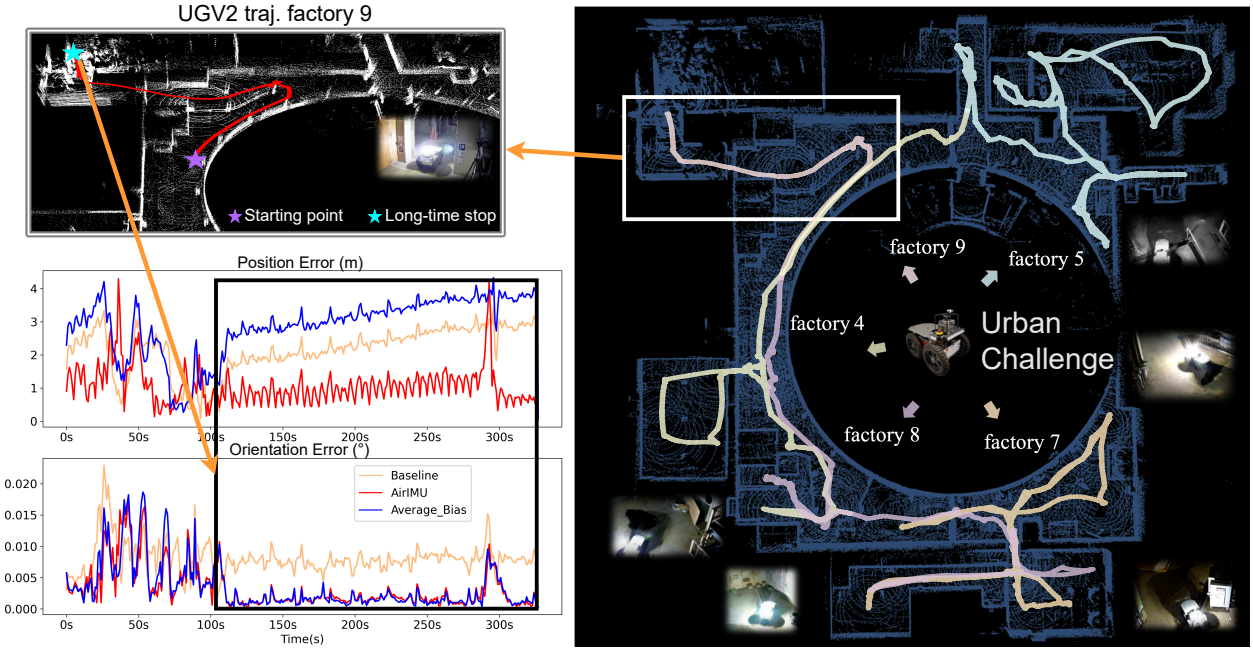

Figure 5.9: We select the sequence `factory 9` from UGV2 in the DARPA SubT Urban Circuit experiments. This sequence captures the robot navigating over an obstacle and remains stationary for a period. During the stationary period (highlighted with white box), (**5.4.5**) **Average Bias** degrades the integration performance compared to raw-data integration. This is because the bias patterns differ when the robot is stationary. **AirIMU** successfully captures the biases while the IMU is stationary.

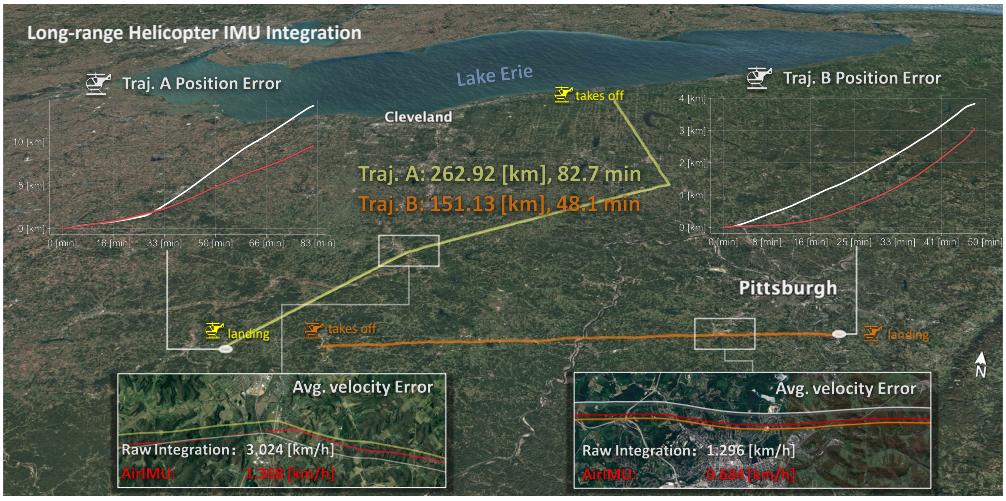

Figure 5.10: Trajectories from the ALTO Dataset [2]: raw-IMU is represented by a white line and AirIMU by a red line. In these trajectories, AirIMU clearly reduces position errors. We also compare the average velocity error, where AirIMU significantly reduces the error for 42.8% in Traj. A and the 50% in Traj. B.

cruising time, we seek to expedite training and enhance robustness by removing the GRU layer from our feature encoder. It's worth noting that only minimal drift is observed, owing to the high precision of the navigation-grade IMU sensor (`NG LCI-1`). We utilize the AirIMU to integrate the entire trajectory, assessing integration quality via speed error. Landing drift is also measured by comparing the integrated trajectory's endpoint with the actual landing spot. As shown in Figure 5.10, compared to the Baseline method, the AirIMU model demonstrates remarkable

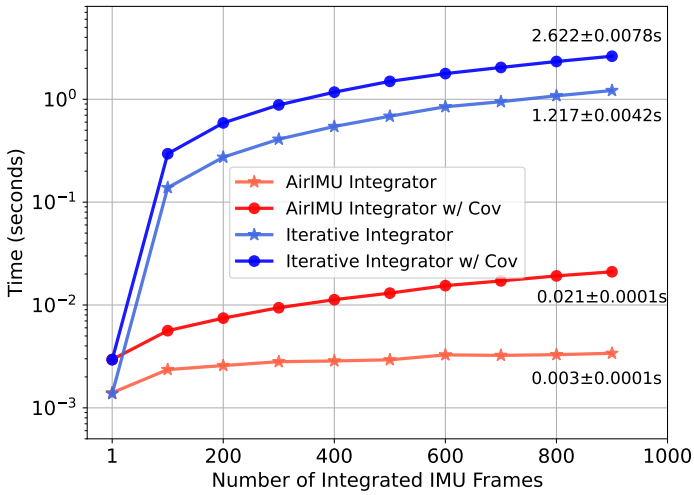

Figure 5.11: Time analysis of the AirIMU integrator with the integrator in an iterative manner. We integrate IMU input with different lengths (from 1 frame to 1000 frames) and display the time consumption in the log scale. The AirIMU integrator demonstrates a hundredfold improvement in efficiency.

enhancements. For Flight A, there's a speed error reduction of 42.8% (from 3.02 $\mathrm{km\,h^{-1}}$ to 1.37 $\mathrm{km\,h^{-1}}$). Flight B experiences a decrease of 50.0% (from 1.29 $\mathrm{km\,h^{-1}}$ to 0.68 $\mathrm{km\,h^{-1}}$), proving the AirIMU model's efficiency in estimating helicopter velocities. Regarding the landing drift, the AirIMU model significantly curtails discrepancies. For Flight A, the drift reduces from 3968 m to just 1064 m. Flight B sees a reduction from 1228 m to 479 m. This experiment showcases the adaptability of the AirIMU in large-scale inertial navigation tasks.

5.4.7 Time Analysis

An efficient differentiable integrator with covariance propagation is critical to training and deploying AirIMU on large-scale datasets. In this section, we evaluate the time consumption of the integrator we proposed in Sec. 5.3.4. We set up the experiments into four groups: **(a)** AirIMU integrator with batched covariance integrator
(b) AirIMU integrator without covariance integrator
(c) Iterative IMU integrator with covariance propagation
(d) Iterative IMU integrator without covariance propagation

For each group, we calculate the time consumption of the integration with different lengths of IMU inputs spanning from 1 to 1000 frames. We repeat the experiment 200 times and calculate the mean and standard deviation of time consumption.

As shown in Figure 5.11, the average time for the iterative integrator to integrate 1000 frames is 1.217s, and the average time for the AirIMU integrator is 0.021s, which is 60 times more efficient. Moreover, we compare the time consumption for the integrator with covariance propagation, where iterative covariance propagation takes 2.622s, and AirIMU with covariance propagation only takes 0.021s. Overall, the AirIMU integrator demonstrates a hundredfold improvement in time consumption, which improves the efficiency of training and deploying AirIMU for inertial navigation.

5.5 Limitation and Discussion

In this chapter, we propose a hybrid method that can adapt to a full spectrum of IMUs. However, we've observed that the AirIMU model trained on a single IMU does not readily generalize to a different IMU with different specifications and operating frequencies. For instance, a model trained for the tactical-grade IMU `Epson G365`, used in the SubT-MRS dataset, does not apply to the automotive-grade IMU `ADIS16448` in the TUMVI dataset due to their different physical design. Our current model is specialized, focusing on a single IMU's unique noise profile and uncertainty characteristics rather than attempting to create a one-size-fits-all model. If we need to deploy the AirIMU model to a new type of IMU, fine-tuning with new IMU data is necessary.

In the future, we may explore the potential for generalizing through different sensors within the same models or types of IMUs. For example, whether the sensor model trained on one `Epson G365` can be adapted to other `Epson G365` IMUs, potentially a more versatile application to scale up in manufacturing the same types of IMU.

5.6 Conclusion

We propose AirIMU, a hybrid IO system with a differentiable integrator and covariance propagator. To demonstrate the efficacy of the learned covariance model, we conduct an ablation study on a GPS PGO experiment, revealing that the learned covariance decreased the loss of PGO by 31%. We evaluate our model in all spectrums of IMUs' grades, spanning the entry-level IMUs (Automotive Grade) to the high-end IMUs (Navigation Grade). Our experiments demonstrate that the AirIMU model achieves outstanding performance across the board. The evaluation also includes IMUs operating in a wide range of environments and featuring multiple agent modalities. We find that AirIMU not only excels at regular indoor environments but also performs well with a ground vehicle dataset over 2 km and a large-scale helicopter dataset over 414 km. Additionally, we open-source the differentiable IMU integrator and the covariance propagation, which is implemented in the *PyPose* framework. Given these advancements, this chapter provides a strong foundation for the inertial uncertainty branch of our thesis, establishing the differentiable covariance propagation paradigm that will be extended to visual-inertial fusion in subsequent chapters.

Table 6.1: The ATE (Unit: m) and RTE (Unit: m) on the Blackbird dataset. **Seen** are sequences where the training and testing datasets are derived from the same trajectory. **Unseen** are those where the testing dataset includes sequences never used in training, demonstrating the model's generalizability.

	Seq.	Baseline‡		AirIMU‡		RoNIN†		TLIO†		IMO†*		AirIO Net		AirIO EKF	
		ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE
Seen	Clover	15.769	2.027	6.163	**0.294**	3.074	1.367	1.464	0.797	0.381	0.681	0.434	0.368	**0.367**	0.391
	Egg	66.17	7.677	13.293	3.801	2.449	2.552	2.227	2.398	1.153	0.828	0.713	0.391	**0.408**	**0.344**
	halfMoon	19.165	3.386	4.174	0.746	1.263	0.864	0.956	0.475	0.761	**0.24**	0.491	0.256	**0.457**	0.253
	Star	20.49	4.556	4.347	1.933	3.15	3.266	0.68	0.784	2.13	3.066	**0.442**	0.401	0.477	**0.341**
	Winter	15.781	2.395	7.445	0.743	1.031	1.089	0.616	0.755	**0.219**	0.206	0.348	**0.147**	0.307	0.164
	Avg.	27.475	4.008	7.084	1.503	2.193	1.828	1.189	1.042	0.929	1.004	0.486	0.312	**0.403**	**0.299**
Unseen	Amper-sand	41.138	5.026	24.738	4.379	5.467	5.321	4.665	4.078	17.664	10.039	**2.303**	**1.145**	2.334	1.154
	Sid	13.108	2.055	16.832	2.131	18.581	10.665	7.323	8.427	9.967	6.992	**0.717**	0.527	0.874	**0.49**
	Oval	15.365	2.4	8.312	1.553	20.399	12.12	1.276	1.045	5.67	2.863	0.976	0.583	**0.828**	**0.54**
	Sphinx	3.429	2.713	2.828	2.024	11.537	7.762	2.005	2.408	5.629	5.395	**1.145**	**1.008**	1.178	1.033
	BentDice	28.307	2.342	54.261	4.39	11.453	6.792	2.119	1.747	6.143	4.519	1.360	1.000	**1.331**	**0.955**
	Avg.	20.269	2.907	21.403	2.896	13.487	8.532	3.478	3.541	9.015	5.962	**1.300**	0.853	1.309	**0.834**

* Leverage thrust information as the input data for training and inference.

† Leverage ground truth orientation to transform the input data. ‡ Dead-reckoning IMU pre-integration.

Chapter 6

AirIO: Learning Inertial Odometry with Enhanced IMU Feature Observability

6.1 Introduction

Inertial Measurement Units (IMUs) are inexpensive and ubiquitous sensors that provide linear accelerations and angular velocities. Due to the size, weight, power, and cost (SWAP-C) constraints, inertial odometry (IO) based only on light and low-cost sensors is ideal for Unmanned Aerial Vehicle (UAV) applications ranging from 3D mapping [24], exploration [135] to physical interaction [136]. Compared to exteroceptive sensors like vision and LiDAR, IMUs are unaffected by visual degradation factors such as motion blur [137] or dynamic object interruption [38], which are common in agile UAV flights [138]. Despite these advantages, current IO solutions struggle to accurately adapt to the complex motion models of UAVs due to the IMU inherent noise and bias, leading to large drift over time, particularly during agile maneuvers.

Most recent advances in learning-based IO have focused on pedestrian and legged robots [3, 114, 115], utilizing motion priors like stride length [127] and repetitive gait patterns [139, 140] beyond IMU pre-integration [102]. In contrast, multirotor UAV motion involves rigorous maneuvers affecting orientation and thrust, resulting in highly dynamic, nonlinear behavior without clear priors. Small attitude changes can cause significant variations in velocity and position, complicating the IO process and making pedestrian-focused methods less effective [138]. As a result, deploying learning-based IO on UAVs often requires additional sensors, such as tachometers [141] or control inputs like thrust commands [138]. This raises the main questions addressed in this chapter:

Why is learning IO not directly applicable to UAVs? How can learning IO be effectively deployed for UAVs?

In this chapter, we investigate the effective representation of learning-based IO in multirotor UAV motion and propose a solution that relies solely on IMU data. Through rigorous feature analysis and examination of UAV dynamics, we identify that the commonly used global-frame representation is less effective for dynamic, agile maneuvers due to the less observable attitude information compared to IMU data in the body-frame representation. To address this, we introduce AirIO, a learning-based IO method that explicitly encodes attitude information and predicts velocity using body-frame representation. We further integrate an EKF that fuses IMU pre-

integration with the learning-based model for odometry estimation. We show that our approach outperforms state-of-the-art algorithms [138] in various scenarios and environments, even without additional sensors or control information.

The main contributions of this chapter are:

1. We conduct the analysis of the extracted features from different representations using t-Distributed Stochastic Neighbor Embedding (t-SNE) and principal component analysis (PCA). The results show that body-frame representation is more expressive and observable. Simply changing the input representation can significantly improve IO accuracy by an average of 66.7%.
2. Building on this, we develop AirIO, a learning-based IO that encodes the attitude information and fuses it with the body-frame IMU data for velocity prediction. This explicit encoding further improves accuracy by 23.8%. Additionally, we integrate an uncertainty-aware IMU preintegration model and a learned motion network into EKF for odometry estimation.
3. We validate our approach on extensive experiments. It outperforms existing IO algorithms without the need for additional sensors or control information. It also demonstrates generalizability to the unseen datasets.

6.2 Related Work

6.2.1 Model-based Inertial Odometry

Traditional model-based inertial odometry (IO) methods rely on kinematic models [102] to estimate relative motion and are often integrated with exteroceptive sensors such as cameras [6] and LiDAR [24]. While these methods achieve high accuracy, their dependence on external sensors makes them vulnerable to disturbances caused by agile motion or dynamic environments. To address the SWAP challenge of lightweight UAV navigation, Ref [142] introduced a method that estimates tilt and velocity by fusing tachometer and IMU data. Although effective, this approach still relies on auxiliary sensors, limiting its applicability in environments where such sensors are unavailable or unreliable.

6.2.2 Learning Inertial Odometry

Recent advances in deep learning have sparked research in data-driven methods for learning velocity and displacement from inertial data [3, 103, 127, 143]. Approaches such as IONet [103], A2DIO [144], and NILoc [143] formulate inertial navigation as a sequential task by predicting relative position displacements using IMU data. Subsequent work has incorporated these learned displacements into EKF [115, 116, 145] or batched optimization frameworks [117]. To ensure consistency in IMU measurements, many of these methods transform the input IMU data into global coordinates using ground-truth orientation. For pedestrian navigation, RoNIN [3] proposed a Heading-Agnostic Coordinate Frame (HACF), aligning the accelerometer's gravity vector with the z-axis and constraining rotation to the horizontal plane. This framework has been widely adopted for learning-based IO [115, 116] due to its effectiveness in simplifying pedestrian motion.

Building on HACF, RIO [128] introduced rotation-equivalence data augmentation to self-supervise pose estimation after orientation alignment. Since pedestrians and wheeled vehicles predominantly move along the horizontal plane, HACF effectively removes yaw dependency from the representation, thereby reducing learning complexity and improving generalizability. Despite the success in pedestrian navigation and wheeled robots [3, 146], these representations are less effective for multirotor UAV motion, which requires more sophisticated dynamic modeling. In this chapter, we show that the commonly used global coordinate representations hinder neural networks from effectively capturing the dynamic complexities of UAVs.

6.2.3 Inertial Odometry for Multirotor UAVs

Existing learning-based IO methods have primarily focused on pedestrian datasets, limiting their generalizability to platforms with more demanding motion dynamics, such as UAVs. Approaches like AI-driven pre-processing and down-sampling of high-speed inertial data have been proposed for better state estimation [147]. Due to its simplicity, most of the existing methods continue adopting the global coordinate frame, such as [138, 141, 148]. DIDO [141] estimates the thrust of UAVs using tachometer data and addresses unmodeled forces by incorporating a neural network. Building on this, the IMO [138] incorporates thrust information and IMU data within an EKF framework, which implicitly captures the drone dynamics. However, IMO relies on additional control signals to capture the drone's dynamics, as shown in Figure 6.7, which may suffer from overfitting on training datasets. DIVE [148], a more recent work for UAV IO, encodes orientation alongside gravity-removed global-frame acceleration. While this representation improves the robustness of learning-based IO, our findings in Sec. 6.5 indicate that it remains suboptimal for capturing UAV dynamics comprehensively.

6.3 Methodology

The goal of learning IO is to estimate relative position transform (velocity) from the IMU data in the local body IMU frame. Most existing methods employ transforming the IMU frame to global frame or removing gravity from the raw IMU data[3, 128, 148]. Our study reveals that these widely used representations, such as global coordinate frames, HACF, and gravity-removed frames, limit the effectiveness of IMU feature extraction. Thus, this section proposes a different feature representation and the corresponding state estimation design. It consists of three parts: feature representation analysis, attitude-encoded network design, and a tightly coupled EKF system.

6.3.1 IMU Coordinate Frame

Canonical Body Coordinate Frame: The IMU accelerometer measures the net force per unit mass acting on the sensor. Due to the measurement mechanism, it measures not only the object actual accelerations due to motion ${}^{\mathcal{B}}\dot{\mathbf{v}} = \frac{{}^{\mathcal{B}}\mathbf{F}_{net_i}}{m}$ in the sensor's local coordinate frame (usually aligned with object body frame), but also the constant acceleration due to Earth's gravity

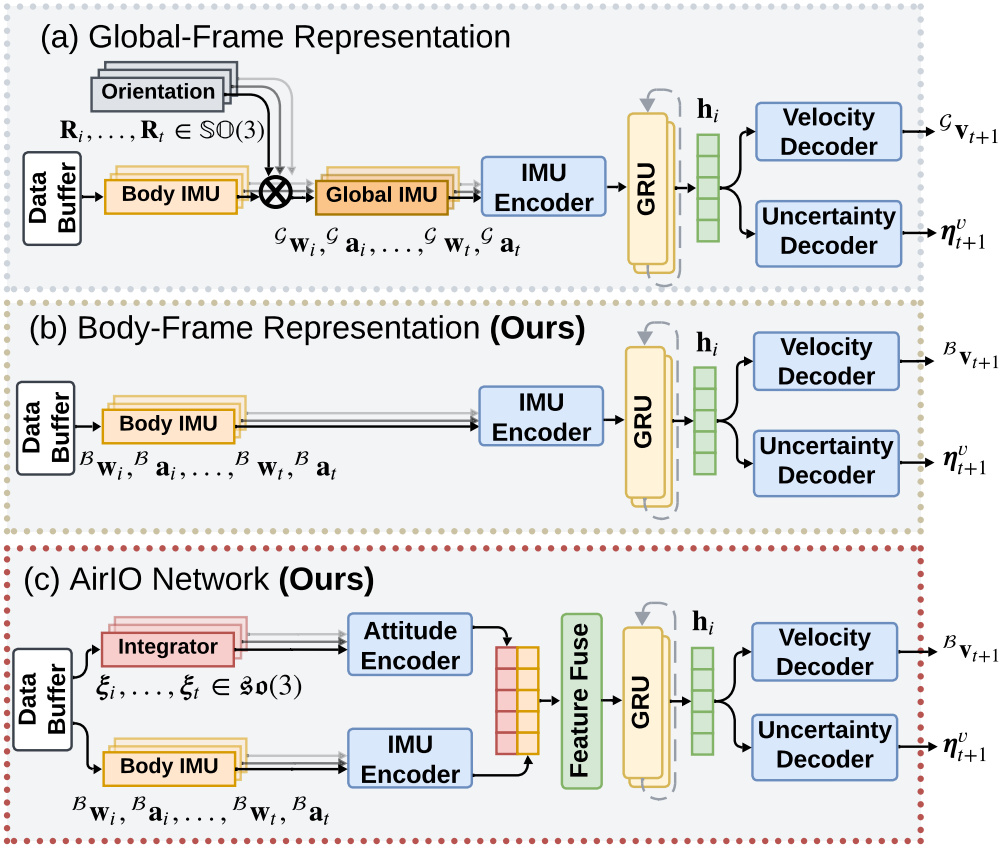

Figure 6.1: (**a**) Existing learning IO usually transforms the IMU input to the global coordinate using ground-truth orientation. (**b**) We found that preserving the IMU data in the body coordinate, along with gravitational information, can significantly improve accuracy. (**c**) Our AirIO model leverages the body-frame IMU and explicitly encodes the UAV's orientation to estimate the body-frame velocity.

${}^{\mathcal{G}}\mathbf{g}$ directed toward the Earth's center. Thus, the accelerometer measurement at timestamp i as:

$$ {}^{\mathcal{B}}\mathbf{a}_i = \frac{{}^{\mathcal{B}}\mathbf{F}_{net_i}}{m} + \mathbf{R}_i^{\top}{}^{\mathcal{G}}\mathbf{g}, \tag{6.1} $$

where m is the object mass, and $\mathbf{R}_i \in \mathbb{SO}(3)$ is the rotation for vector from body frame $\mathcal{B}$ to the global frame $\mathcal{G}$. ${}^{\mathcal{F}}(\cdot)$ denotes quantity represented in frame $\mathcal{F}$.

Global Coordinate Frame: Many existing methods transform the IMU data from body frame $\mathcal{B}$ to HACF or global frame $\mathcal{G}$ by applying an estimated transform rotation $\hat{\mathbf{R}}_i$, e.g.

$$ {}^{\mathcal{G}}\mathbf{a}_i = \hat{\mathbf{R}}_i \frac{{}^{\mathcal{B}}\mathbf{F}_{net_i}}{m} + \hat{\mathbf{R}}_i \mathbf{R}_i^{\top}{}^{\mathcal{G}}\mathbf{g} \tag{6.2} $$

The rotation $\hat{\mathbf{R}}_i$ can often be obtained by simple IMU preintegration or the estimation results from an EKF, which is often accurate enough, such that $\hat{\mathbf{R}}_i \mathbf{R}_i^{\top} \approx \mathbf{I}$.

Notice that both (6.1) and (6.2) represents the same physical process. The key difference lies in how the attitude information is coupled with different components: in (6.1), it is coupled with the gravitational force, whereas in (6.2), it is coupled with the motion force. In (6.1), since the gravity acceleration ${}^{\mathcal{G}}\mathbf{g}$ is a constant, the motion force and attitude form a linear combination.

In contrast, in (6.2), the attitude couples with motion force in a nonlinear manner, with gravity acting as an additional constant that does not contribute extra useful information. Thus, the motion force and attitude are intuitively easier to estimate in (6.1), while (6.2) requires a more complex network structure for estimation.

6.3.2 Representation Analysis

We perform feature analysis to verify our intuition. Specifically, we extract the latent features $\mathbf{h}_i$ from the IMU feature encoder for the input under different representations in Figure 6.1. We conduct multiple feature analyses to assess their effectiveness and representativeness.

Principal Component Analysis PCA is a statistical method that reduces the dimensionality of a data set by replacing correlated variables with a smaller set of uncorrelated principal components. To analyze the expressivity of the input representation, we performed PCA on the feature latent space for different input coordinates. We concatenate the latent features from all samples to form a feature matrix and perform PCA on the latent feature matrix to analyze its underlying structure by computing the singular value of the normalized feature matrix in a numerically stable way. Each singular value represents the corresponding magnitude scaling of one principal feature component. We then quantify the cumulative explained variance (or the energy coverage) for the top k principal components in PCA. Specifically, we evaluate on two datasets: the widely used UAV dataset Blackbird[149] collected for agile autonomous flight, which covers large UAV maneuvers, and our custom simulation dataset Pegasus, which was collected in the NVIDIA IsaacSim simulator with Pegasus autopilot [150], featuring diverse maneuvers in an ideal environment without external disturbance, allowing us to work with cleaner data.

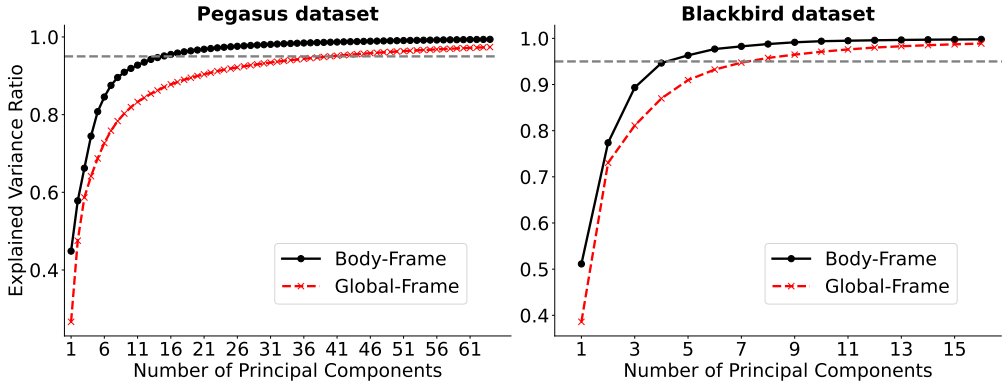

Figure 6.2: The cumulative explained variance of the latent features for the Pegasus dataset (**Left**) and Blackbird dataset (**Right**). In both datasets, the latent features derived from body frames (black line) require fewer principal components to achieve comparable total energy.

Figure 6.2 shows the cumulative energy coverage of the latent feature derived from two different IMU input representations: global and body frames. The red and black curves represent the energy coverage of the top k principal components. Specifically, for the Blackbird dataset, it shows that the top $k = 5$ principal components in body-frame representation already cover 95% of the total energy. In contrast, global-frame representation requires the top $k = 8$ principal components to achieve the same level of energy. Similarly, in the Pegasus datasets, the body-frame representation requires top $k = 15$ principal components and the global frame require top $k = 40$ principal components to cover 95% of the total energy. It clearly shows that models trained on the

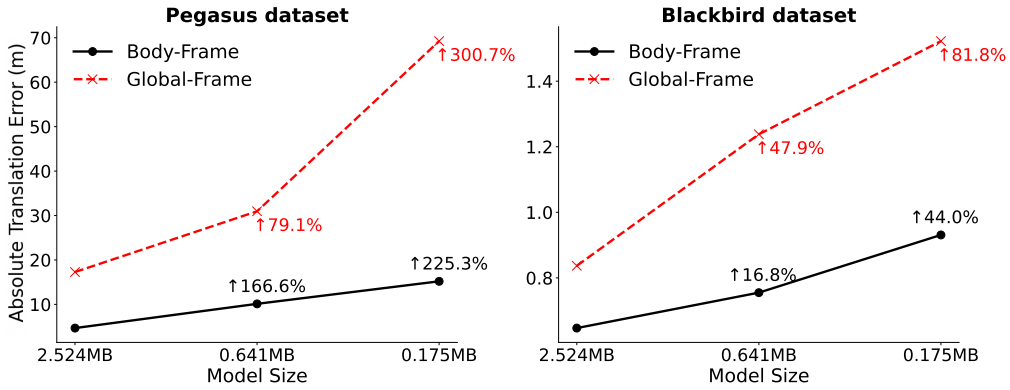

Figure 6.3: The ATE for body-frame and global-frame models at different model sizes on Pegasus dataset (**Left**) and Blackbird dataset (**Right**). The text denotes the relative percentage increase in ATE.

body representation input can achieve comparable performance with fewer features, highlighting the efficiency and expressivity of this representation in capturing the essential features. It also implies that models trained under body frame have better compressibility and can be designed more lightweight. Figure 6.3 shows the comparison of the absolute translation error (ATE) for the models at different scales under both body and global frames. As the model size decreases, the prediction performance for model under body-frame representation degrades more smoothly and consistently yields lower errors. See Sec. **??** for more studies on the model compressibility.

t-SNE [151] We also perform t-SNE on Pegasus dataset, a dimensionality reduction approach that projects high-dimension data into a low-dimensional map to analyze the feature representation qualitatively. As shown in Figure 6.4, based on the network prediction (here is the velocity), we color code each data point by the velocity magnitude from low to high. In the global frame, the points exhibit a highly entangled distribution and different velocity levels overlap, indicating poor separability and less representative encoding. In contrast, the feature distribution of the body coordinate frame is well-separated, where different velocity levels form more coherent clusters. This suggests that the body frame facilitates a more discriminative and effective representation of latent features.

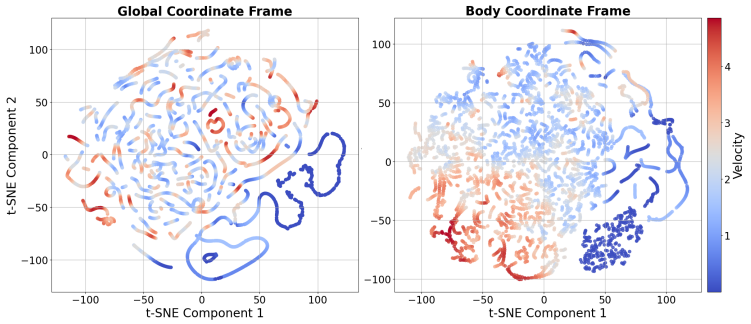

Figure 6.4: t-SNE analysis on Pegasus dataset with each point colored based on the velocity magnitude from low to high, represented by blue to red.

In summary, the analysis from Sec. 6.3.1 and Sec. 6.3.2 indicates that i). Good prior information will help for the estimation since it couples with the motion force in a linear or nonlinear manner, as shown in (6.1)(6.2). ii). The nonlinear coupling between the motion force and attitude

in the global-frame representation complicates the estimation problem. Body-frame representation proves to be more expressive and discriminative, leading to a light network for prediction. Notice that while the analysis is indeed applicable to general robots via standard IMU equations, UAVs present a compelling test case due to their agile motion. Unlike many mobile or legged robots with mainly yaw-dominant movement, UAVs often exhibit significant roll and pitch maneuvers, resulting in more pronounced impact of frame representation difference.

6.3.3 AirIO Motion Network & Training

Based on the previous analysis, we introduce a separate encoding of the drone's attitude $\boldsymbol{\xi} \in \mathfrak{so}(3)$ to the motion network beyond the IMU encoder with body-frame representation, shown in Figure 6.1(c). After fusing this information, the network predicts the body-frame velocity ${}^{\mathcal{B}}\hat{\mathbf{v}}$ and the corresponding uncertainty $\hat{\boldsymbol{\eta}}^v$. The motion network can be written as: $({}^{\mathcal{B}}\hat{\mathbf{v}}, \hat{\boldsymbol{\eta}}^v) = f_{\boldsymbol{\theta}}({}^{\mathcal{B}}\mathbf{w}, {}^{\mathcal{B}}\mathbf{a}, \boldsymbol{\xi})$.

Inspired by [61], we map the drone's orientation to the $\mathfrak{so}(3)$ Lie algebra space for compact and continuous representation of 3D rotations and smoother network gradient to mitigate numerical instability. We then leverage a CNN encoder to encode $\mathfrak{so}(3)$.

After concatenating the features from the attitude encoder and the IMU encoder, we employ bi-directional GRU layers to extract the latent feature $\mathbf{h}_i$ and model the temporal dependencies in the drone dynamics. We leverage this feature to estimate the body-frame velocity and the corresponding uncertainty by two MLP decoders.

To jointly supervise the velocity and the corresponding uncertainty ${}^{\mathcal{B}}\hat{\mathbf{v}}_i$, we designed a combined loss function:

$$\mathcal{L} = \mathcal{L}_{\text{Huber}} + \lambda \mathcal{L}_C, \tag{6.3}$$

where the $\lambda = 1e^{-4}$. The $\mathcal{L}_{\text{Huber}}$ is the Huber loss between the predicted body-frame velocity ${}^{\mathcal{B}}\mathbf{v}_i$ and the body-frame ground-truth velocity ${}^{\mathcal{B}}\hat{\mathbf{v}}_i$, defined as:

$$\mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2}({}^{\mathcal{B}}\mathbf{v}_i - {}^{\mathcal{B}}\hat{\mathbf{v}}_i)^2 & \text{if } |{}^{\mathcal{B}}\mathbf{v}_i - {}^{\mathcal{B}}\hat{\mathbf{v}}_i| < \delta \\ \delta \cdot \left(|{}^{\mathcal{B}}\mathbf{v}_i - {}^{\mathcal{B}}\hat{\mathbf{v}}_i| - \frac{1}{2}\delta\right) & \text{otherwise} \end{cases}, \tag{6.4}$$

where we set the $\delta = 0.005$.

To supervise the uncertainty of the velocity, we employ an uncertainty loss inspired by [115], assuming that the estimated velocity uncertainty follows a Gaussian distribution:

$$\mathcal{L}_C = ({}^{\mathcal{B}}\mathbf{v}_i - {}^{\mathcal{B}}\hat{\mathbf{v}}_i)\hat{\boldsymbol{\Sigma}}_i^{v-1}({}^{\mathcal{B}}\mathbf{v}_i - {}^{\mathcal{B}}\hat{\mathbf{v}}_i)^T + \ln(\det \hat{\boldsymbol{\Sigma}}_i^v), \tag{6.5}$$

where $\hat{\boldsymbol{\Sigma}}_i^v$ is 3×3 covariance matrix for the i^{th} data. Similar to the setting in [115], we simplified the covariance as $\hat{\boldsymbol{\Sigma}}_i^v = \text{diag}(\hat{\boldsymbol{\eta}}_i^{v2})$, where $\hat{\boldsymbol{\eta}}_i^v$ is learned uncertainty from network.

For orientation $\boldsymbol{\xi}$, we use the ground truth during training and replace it with EKF-estimated values during testing.

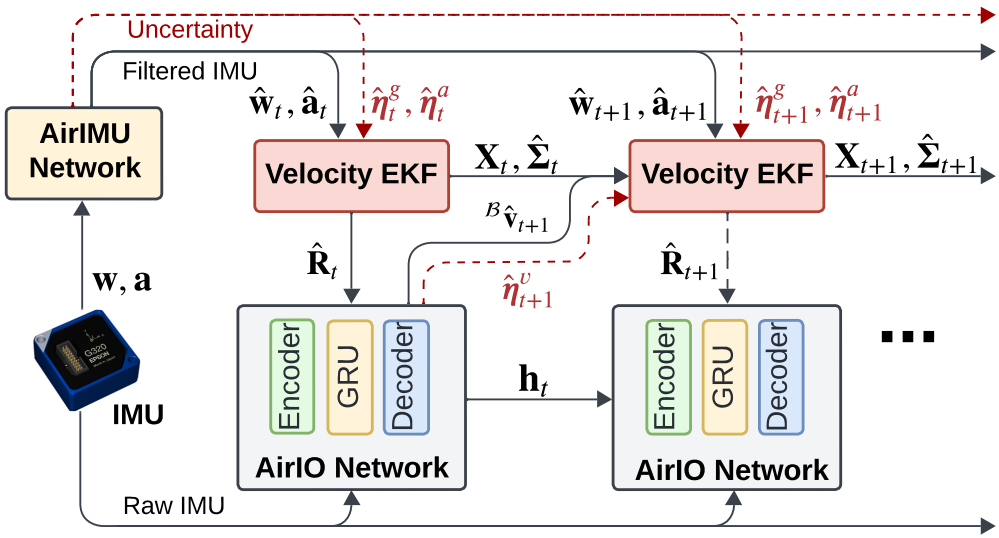

Figure 6.5: AirIO system pipeline. A tightly coupled EKF that fuses the AirIO motion network with a learning-based IMU preintegration network AirIMU. Instead of using constant uncertainty parameters, we employ learned uncertainty $\hat{\boldsymbol{\eta}}^v$ from the AirIO network and the learned gyroscope uncertainty $\hat{\boldsymbol{\eta}}^g$ and accelerometer $\hat{\boldsymbol{\eta}}^a$ from AirIMU in our EKF system.

6.3.4 Extended Kalman Filter

Different from the existing IO methods [115, 138] which employ a motion network to capture relative translation and maintain historic states to optimize relative pose for state pairs, AirIO network models the body-frame velocity. As a result, we only need to constrain the velocity of the current frame, which significantly reduces the number of state parameters and alleviates the need for frame culling. This setup isolates the current state of the drone from the previous state, which simplifies the EKF structure. The full state of the filter is $\mathbf{X} = (\mathbf{R}_i, {}^{\mathcal{G}}\mathbf{v}_i, {}^{\mathcal{G}}\mathbf{p}_i, \mathbf{b}_{a_i}, \mathbf{b}_{g_i})$, where $\mathbf{b}_{a_i}$ and $\mathbf{b}_{g_i}$ are the bias of the accelerometer and the gyroscope respectively. Following the common setup of the error-based filtering method, we define the error-state of the current filter as $\delta\mathbf{X} = (\delta\boldsymbol{\xi}_i, \delta{}^{\mathcal{G}}\mathbf{v}_i, \delta{}^{\mathcal{G}}\mathbf{p}_i, \delta\mathbf{b}_{a_i}, \delta\mathbf{b}_{g_i})$. For the rotation error, we define $\delta\boldsymbol{\xi}_i = \log_{SO3}(\mathbf{R}_i\hat{\mathbf{R}}_i^{-1}) \in \mathfrak{so}(3)$, where $\log_{SO3}(\cdot)$ denotes logarithm map operator.

AirIMU feature correction and uncertainty estimation To filter the noise and, more importantly, quantify the uncertainty of the IMU pre-integration, we leverage AirIMU [61] to pre-process the raw IMU data. This model estimates the corrections of the gyroscope and the accelerometer, and also the uncertainty of the IMU sensor:

$$(\hat{\boldsymbol{\sigma}}_{g_i}, \hat{\boldsymbol{\sigma}}_{a_i}) = g_{\boldsymbol{\theta}}(\mathbf{w}_i, \mathbf{a}_i), \quad (\hat{\boldsymbol{\eta}}_{g_i}, \hat{\boldsymbol{\eta}}_{a_i}) = \Sigma_{\boldsymbol{\theta}}(\mathbf{w}_i, \mathbf{a}_i), \tag{6.6}$$

where the θ is the parameter of the neural network. The AirIMU network is trained by a differentiable IMU integrator and covariance propagator. We later filtered the IMU measurement by the predicted correction $\hat{\mathbf{a}} = \mathbf{a} + \hat{\boldsymbol{\sigma}}_{a_i}, \hat{\mathbf{w}} = \mathbf{w} + \hat{\boldsymbol{\sigma}}_{g_i}$. In the EKF system, the learned uncertainty $(\hat{\boldsymbol{\eta}}_{g_i}, \hat{\boldsymbol{\eta}}_{a_i})$ is leveraged in the sensor covariance modeling.

Filter Propagation The kinematic motion model is:

$$\begin{aligned}
\mathbf{R}_{i+1} &= \mathbf{R}_i \cdot \mathrm{Exp}(\hat{\mathbf{w}}_i - \mathbf{b}_{g_i})\Delta t, \\
\mathbf{v}_{i+1} &= \mathbf{v}_i + \mathbf{R}_i(\hat{\mathbf{a}}_i - \mathbf{b}_{a_i})\Delta t, \\
\mathbf{p}_{i+1} &= \mathbf{p}_i + \mathbf{v}_i\Delta t + \frac{1}{2}\Delta t^2\mathbf{R}_i(\hat{\mathbf{a}}_i - \mathbf{b}_{a_i}), \\
\mathbf{b}_{a_{i+1}} &= \mathbf{b}_{a_i}, \mathbf{b}_{g_{i+1}} = \mathbf{b}_{g_i},
\end{aligned} \tag{6.7}$$

Here, the $\mathrm{Exp}(\cdot)$ denotes mapping from the log space to exponential space. The linearized propagation model is:

$$\delta\mathbf{X}_{i+1} = \mathbf{A}_i \cdot \delta\mathbf{X}_i + \mathbf{B}_i \cdot \mathbf{n}_i, \tag{6.8}$$

where $\mathbf{n}_i = [\hat{\boldsymbol{\eta}}_{g_i}, \hat{\boldsymbol{\eta}}_{a_i}, \boldsymbol{\eta}_{b_g}, \boldsymbol{\eta}_{b_a}]^T$. The $\hat{\boldsymbol{\eta}}_{g_i}, \hat{\boldsymbol{\eta}}_{a_i}$ are the learned IMU uncertainty from the AirIMU network [61] with the i-th frame. The $\boldsymbol{\eta}_{b_g}, \boldsymbol{\eta}_{b_a}$ are the random walk uncertainty set as a constant across different frames. The corresponding covariance propagation of the state covariance $\mathbf{P}_{i+1}$ follows:

$$\mathbf{P}_{i+1} = \mathbf{A}_i\mathbf{P}_i\mathbf{A}_i + \mathbf{B}_i\mathbf{W}_i\mathbf{B}_i \tag{6.9}$$

where the $\mathbf{W}_i$ is the covariance matrix of the i-th input, including the learned uncertainty of the IMU $\hat{\boldsymbol{\eta}}_{g_i}, \hat{\boldsymbol{\eta}}_{a_i}$ and sensor bias random walk $\boldsymbol{\eta}_{b_g}, \boldsymbol{\eta}_{b_a}$.

Filter Update The measurement update is as follows:

$$h(\mathbf{X}) = \hat{\mathbf{R}}_i^T \cdot \mathbf{v}_i = \hat{\mathbf{v}}_i + \hat{\boldsymbol{\eta}}_{v_i}. \tag{6.10}$$

where $\boldsymbol{\eta}_{v_i}$ is a random variable that follow the Gaussian distribution $\mathcal{N}(0, \hat{\boldsymbol{\Sigma}}_i^v)$ learned by the network. The Jacobian matrix $\mathbf{H}$ is computed as:

$$\begin{aligned}
\mathbf{H}_{\delta v_i} &= \frac{\partial h(\mathbf{X})}{\partial \delta \mathbf{v}_i} = \hat{\mathbf{R}}_i^T \\
\mathbf{H}_{\delta \xi_i} &= \frac{\partial h(\mathbf{X})}{\partial \delta \boldsymbol{\xi}_i} = \hat{\mathbf{R}}_i^T[\mathbf{v}_i]_\times
\end{aligned} \tag{6.11}$$

Finally, the Kalman Gain and the update can be computed:

$$\begin{aligned}
\mathbf{K} &= \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \hat{\boldsymbol{\Sigma}}_i^v)^{-1} \\
\mathbf{X} &\leftarrow \mathbf{X} \oplus (\mathbf{K}(h(\mathbf{X}) - \hat{\mathbf{v}}_i)) \\
\mathbf{P} &\leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\hat{\boldsymbol{\Sigma}}_i^v\mathbf{K}^T
\end{aligned} \tag{6.12}$$

Operator $\oplus$ denotes the additional operation except for the orientation, where the update performs $\mathbf{R} \leftarrow \mathrm{Exp}(\boldsymbol{\xi}) \cdot \mathbf{R}$.

6.4 Experiments

6.4.1 Experiment Setup

Datasets We evaluate the proposed approach on two quadrotor datasets: the real-world EuRoC dataset [97] and the challenging drone racing dataset Blackbird [149]. EuRoC datasets are a

widely used benchmark for evaluating odometry. Blackbird dataset is an indoor flight dataset, presenting more aggressive maneuvers and high-speed dynamics. For ablation studies, we additionally include our custom simulated dataset collected using the NVIDIA IsaacSim simulator with Pegasus autopilot [150]. The Pegasus dataset also presents diverse maneuvers but in the ideal environment without external disturbance, allowing us to work with cleaner data for ground-truth accuracy.

Baseline Several state-of-the-art methods are selected for comparison. For model-based IO, we include *Baseline* [102], the IMU preintegration approach in raw IMU data, and *AirIMU* [61], a hybrid method that corrects the raw IMU noise with a data-driven method and integrates it with the IMU kinematic function. For learning-based IO, we compare against an end-to-end trained model *RoNIN* [3] and an EKF-fused algorithm *TLIO* [115]. For learning-based IO methods designed for UAVs, we evaluate *IMO* [138], a control signal-embedded IO specifically developed for drone racing, which incorporates additional thrust signals to better capture drone motion.

Evaluation Metrics We use the following metrics for evaluation: 1) **Absolute Translation Error** (ATE, m): the average Root Mean Squared Error (RMSE) between the estimated and ground-truth positions over all time points:

$$\text{ATE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\mathbf{p}_i - \hat{\mathbf{p}}_i\|_2^2}, \tag{6.13}$$

2) **Relative Translation Error** (RTE, m): the average RMSE of the relative displacements over predefined time intervals.

$$\text{RTE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left\| \mathbf{p}_{i+\Delta t} - \mathbf{p}_i - \mathbf{R}_i \hat{\mathbf{R}}_i^T \left(\hat{\mathbf{p}}_{i+\Delta t} - \hat{\mathbf{p}}_i\right) \right\|_2^2}, \tag{6.14}$$

Our evaluation adopts a 5-second interval configuration.

We define the improvement percentage as the relative reduction in error compared to the baseline. Specifically, it is computed as the difference between the error of baseline and the proposed method, divided by the baseline error.

Training Details We use the Adam optimizer with an initial learning rate of 0.001. A learning rate scheduler is implemented following the ReduceLROnPlateau, with a patience of 5 epochs and a decay factor of 0.2. The batch size is 128. Our training and testing window size is 1000 frames (equal to 5 seconds in the EuRoC dataset). In the CNN encoder, we include a dropout layer with $p = 0.5$ to reduce overfitting.

6.4.2 Results

Evaluation on Blackbird Dataset

We employ the same sequences from Blackbird as used in IMO [138] and adopt their same train-test split configuration. We name these sequences as **SEEN** sequences because the training and testing data are from the same sequence. To evaluate generalization, we select five additional trajectories from BlackBird dataset as **UNSEEN** sequences, which are excluded from the training

set.

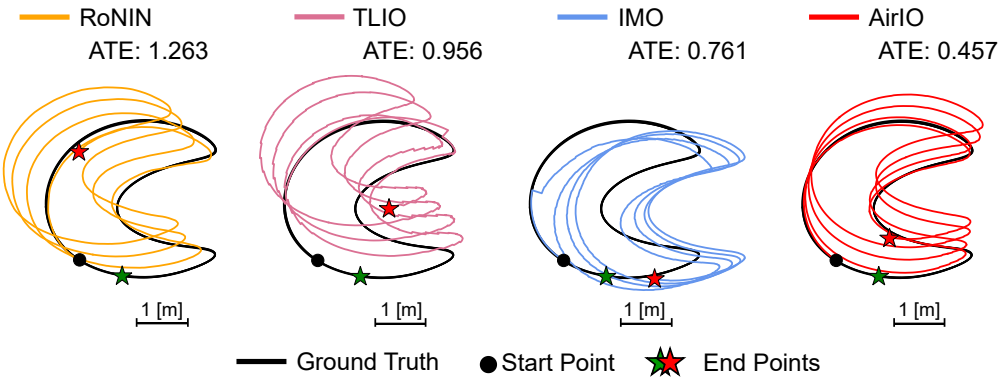

Figure 6.6: Trajectories of **SEEN** sequence `halfMoon` from the Blackbird dataset. AirIO, relying solely on IMU, outperforms IMO by 30% in ATE.

For the seen sequences, as shown in Table 6.1, RoNIN and TLIO fail to capture the aggressive drone maneuvers as they are designed for pedestrian navigation with clear slow motion patterns. AirIO significantly outperforms these baselines, improving by 66.1% in ATE and 71.3% in RTE over TLIO. Furthermore, AirIO exceeds IMO by 56.6% in ATE and 70.2% in RTE, relying solely

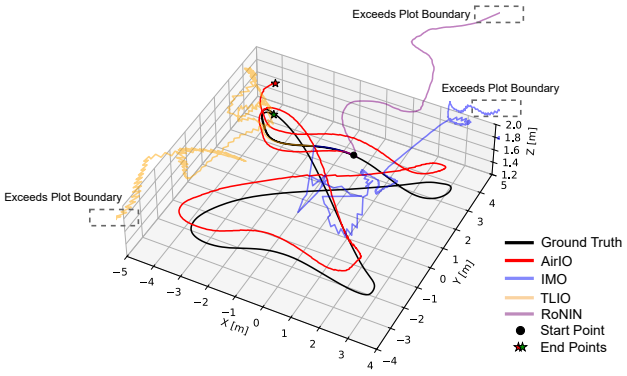

Figure 6.7: Trajectories of **UNSEEN** sequence `sid` from the Blackbird dataset by RoNIN, TLIO, IMO, and AirIO.

on IMU measurement without integrating external thrust information. We also show that our algorithm can outperform the multi-sensor fusion algorithms like the visual-inertial odometry on the drone racing data. For more details, please check Sec. **??**.

In addition, it is worth noting that AirIO demonstrates remarkable generalizability to unseen sequences, as shown in Figure 6.7. While other methods suffer from significant drift, AirIO consistently maintains precise estimations. The baseline methods tend to overfit the training data, performing well within the training distribution but failing to generalize to UNSEEN data. Compared to IMO, AirIO shows a substantial improvement of 85.5% in ATE and 86.0% in RTE.

Evaluation on EuRoC Dataset

We then evaluate the proposed method on EuRoC dataset, as shown in Table 6.2. AirIO outperforms both RoNIN and TLIO by 52.9% and 54.4% in ATE, respectively. Thanks to effective

uncertainty modeling, AirIO leverages uncertainties learned from both the IMU kinematic model and motion network, ensuring optimal data fusion and improved performance. After integrating EKF, AirIO further improves accuracy, showing an average additional 17.4% improvement in ATE.

Table 6.2: The ATE (Unit: m) and RTE (Unit: m) on EuRoC dataset.

Seq.	RoNIN†		TLIO†		AirIO Net		AirIO EKF	
	ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE
MH02	5.902	2.121	7.281	2.451	4.917	**0.936**	**2.478**	0.987
MH04	8.586	4.542	8.626	5.498	2.726	1.093	**2.308**	**1.005**
V103	3.240	1.695	7.863	2.580	3.844	**1.519**	**3.05**	1.552
V202	7.445	2.303	6.260	2.783	4.823	**1.303**	**4.206**	1.313
V101	8.576	1.918	4.814	1.946	**2.917**	1.137	3.844	**1.103**
Avg.	6.75	2.516	6.969	3.052	3.846	1.198	**3.177**	**1.192**

† Leverage ground truth orientation to transform the input data for inference.

Real-Time Deployment and Analysis

We evaluate the real-time performance of our algorithm on both an NVIDIA Jetson Orin AGX and a desktop with a low-end RTX 2060 GPU, using an IMU operating at 200 Hz. On Orin, AirIO achieves an average runtime of 74.23 ms, while on the desktop, it runs in 28.92 ms. These results indicate the feasibility of our algorithm for real drone deployment.

6.5 Ablation Study

We perform an ablation study to investigate the impact of different feature representations. Experiments with the following input representation scenarios are conducted on the real-world dataset EuRoC and the simulated dataset Pegasus.

i. **Body**: The network takes body-frame IMU measurements defined in (6.1) as input.

ii. **Global**: The network takes global-frame IMU measurements as input. The global-frame IMU is transformed from raw IMU data, as defined in (6.2).

iii. **Body + Attitude**: Building upon (i), the drone orientation is encoded as auxiliary inputs to the network.

iv. **Global + Attitude**: Building upon (ii), the drone orientation is encoded as auxiliary inputs to the network.

v. **Body – $\mathbf{R}_i^\top {}^{\mathcal{G}}\mathbf{g}$**: Building upon (i), we consider the motion acceleration by removing the rotation-coupled gravity term $\mathbf{R}_i^\top {}^{\mathcal{G}}\mathbf{g}$ from (6.1).

vi. **Global – ${}^{\mathcal{G}}\mathbf{g}$**: Building upon (ii), we consider the motion acceleration by removing the gravity term ${}^{\mathcal{G}}\mathbf{g}$ from (6.2).

To ensure a fair comparison, scenarios (i), (iii), and (v) leverage ground-truth orientation to transform estimated velocity into global frame.

Table 6.3 shows the results of the ablation study for all variants. More results can be found in Sec. **??**. From these results, we address the following questions:

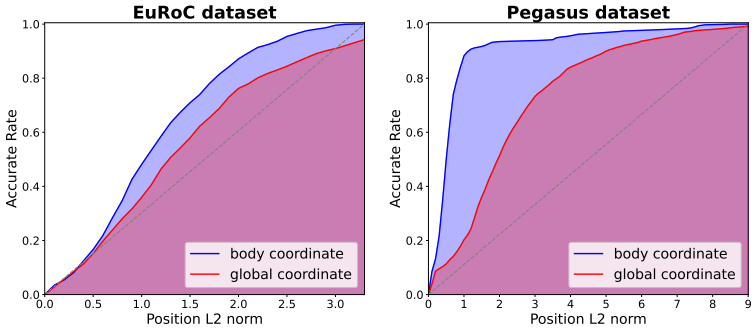

Figure 6.8: Accuracy AUC of EuRoC dataset **(Left)** and Pegasus dataset **(Right)**.

6.5.1 How Effective is the Body-frame Representation?

Compared to (ii) **Global**, (i) **Body** demonstrates significantly higher precision on the Pegasus. This new input representation results in an average ATE improvement of 74.1%. Scenario (i) on EuRoC dataset and Blackbird Unseen sequences exhibit similar trends.

We further evaluate the system performance under two representation using the Accuracy Area Under the Curve (AUC) metric, which quantifies the performance across various error thresholds. Specifically, we calculate the relative position error over a 5-second interval. As shown in Figure 6.8, on both datasets, scenario (i) achieves higher accuracy across all error thresholds and reaches near-perfect accuracy much more quickly. These results highlight the effectiveness of body-frame representation which efficiently captures implicit attitude information of IMU data. Simply encoding the body frame IMU features improves the system performance.

6.5.2 How Effective is the Attitude Encoding?

As illustrated in Figure 6.9, on the EuRoC dataset, (iii) **Body + Attitude** achieves a further 30.3% improvement over the scenario (i), outperforming scenario (ii) by 67.3% in ATE. Additionally, we evaluate attitude encoding on global-frame representation (iv) **Global + Attitude**. This also yields an improvement of approximately 29.6% over scenario (ii) on EuRoC dataset, validating the efficiency of encoding attitude information. Encoding the drone's attitude information enhances the network's ability for state estimation.

6.5.3 Shall We Keep Gravity in the Feature Representation?

Many learning-based IO methods remove gravity from the IMU encoding [148]. However, as discussed in Sec. 6.3.1, gravitational acceleration is a critical component for body-frame representation as it explicitly embeds drone rotation through the term $\mathbf{R}_i^{T\mathcal{G}}\mathbf{g}$, as shown in 6.1. After removing this rotation-coupled gravity term, scenario (i) loses essential attitude information. Table 6.3 demonstrates that (v) **Body** $-\mathbf{R}_i^{T\mathcal{G}}\mathbf{g}$ causes significant performance degradation: the ATE

Table 6.3: Ablation study on the EuRoC and Pegasus datasets comparing different feature representations. Evaluation metric: ATE (Unit: m).

	Seq.	Global (6.2) $-{}^{\mathcal{G}}\mathbf{g}$ Global		Body (6.1) $-\mathbf{R}_i^{T\mathcal{G}}\mathbf{g}$	+Global Body +Attitude		Body +Attitude
EuRoC	MH02	18.048	17.892	16.136	9.522	4.537	**2.531**
	MH04	15.904	6.895	8.977	4.572	3.561	**2.250**
	V103	7.007	7.086	6.586	4.068	6.824	**3.107**
	V202	9.662	3.496	6.161	10.274	**3.248**	4.310
	V101	8.142	15.11	7.082	7.117	5.478	**4.287**
	Avg.	11.753	10.096	8.988	7.110	4.730	**3.297**
Pegasus	TEST_1	22.115	9.689	9.534	6.151	5.225	**3.418**
	TEST_2	15.745	15.185	14.729	5.373	**4.489**	5.719
	TEST_3	19.467	26.960	3.580	14.545	3.734	**1.786**
	Avg.	19.109	17.278	9.281	8.690	4.483	**3.641**
Blackbird	Amper-sand	21.93	17.863	18.639	14.487	**1.977**	2.492
	Sid	11.866	7.915	5.686	2.307	1.108	**0.651**
	Oval	7.212	6.743	1.656	7.354	1.353	**0.913**
	Sphinx	3.327	3.572	2.117	2.784	1.266	**1.163**
	BentDice	15.5	8.876	6.989	9.626	1.915	**1.249**
	Avg.	11.967	8.994	7.018	7.312	1.524	**1.294**

increases by 90.0% on EuRoC, 107.0% on Pegasus dataset, and 360.5% on Blackbird's Unseen sequences.

6.6 Conclusion and Discussion

In this chapter, we investigate an effective representation of learning-based IO for multirotor UAVs and propose a solution that relies solely on IMU data. We identify that the commonly used global-frame representation is less effective for dynamic, agile maneuvers due to the less observable attitude information compared to IMU data represented in body-frame. Based on this, we develop the AirIO system by explicitly encoding attitude information and integrating it with an EKF. We show that our approach outperforms the state of the art [138] in various scenarios and environments without relying on additional sensors or control information. Our method achieves on average 66.7% improvement in accuracy. Furthermore, explicitly encoding attitude information into the motion network results in an additional 23.8% improvement. We validate the real-time capability of AirIO on an NVIDIA Jetson AGX Orin. Future work includes deploying the system on real UAVs for closed-loop evaluation.

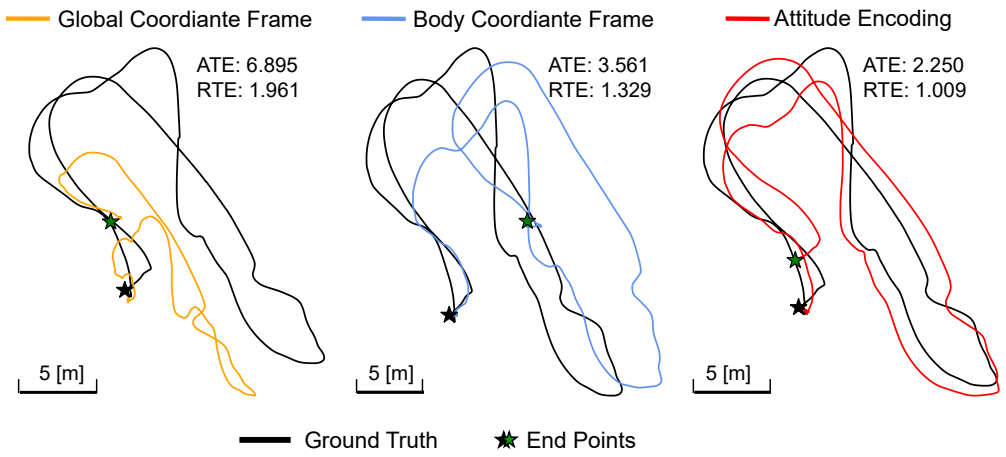

Figure 6.9: Performance of `MH_04_difficult` in EuRoC dataset across different representations. (ii) **Global (Left)** is suboptimal. Using IMU data in (i) **Body (Center)** improves significantly. Encoding UAV orientation (iii) **Body + Attitude (Right)** further improves the performance.

Chapter 7

MAC-I^2: Metrics-Aware Visual-Inertial Initialization and Calibration

7.1 Introduction

Visual-inertial odometry (VIO) is a cornerstone technology for autonomous systems, AR/VR, and robotics, providing robust state estimation by fusing visual and inertial measurements [6]. Accurate initialization is essential for estimating gravity, initial velocity, and IMU bias, which together define a physically consistent state for the motion estimation. Beyond initialization, maintaining reliable performance in real-world deployments further requires calibration to be performed online, as sensor characteristics and environmental conditions can vary over time. However, existing approaches are often fragile in practice, as illumination changes, dynamic objects, motion blur, and occlusions degrade feature tracking and frequently lead to initialization or calibration failure.

Classical pipelines such as VINS-Mono [6] and OKVIS [7] bootstrap the state by estimating camera motion and structure from visual correspondences (e.g., via BA/SfM) and aligning these estimates with IMU integration to correct biases and initialize gravity and velocity. Consequently, initialization success depends on the availability of robust feature tracks across multiple frames, and can fail under illumination changes, dynamic objects, motion blur, and occlusions. Further, real deployments often require online camera-IMU extrinsic refinement [152] because target-based offline calibration is performed in controlled environments with calibration patterns [111], while in-the-wild operation can experience gradual changes in the effective extrinsic. Crucially, both initialization and online calibration remain constrained by the quality of visual correspondences; when visual features are unreliable, accurate state bootstrap and online calibration become difficult.

Additionally, uncertainty modeling remains a fundamental challenge in current visual-IMU initialization and calibration. Due to the complexity and variability of real-world environments, sensor uncertainty is difficult to capture with analytic models. As a result, most existing approaches [6, 134] rely on hand-tuned or fixed visual and inertial covariance models to balance the relative trust between vision and IMU measurements during the optimization. These covariance parameters are typically constant or heuristically defined, and often require careful retuning across datasets, sensor configurations, and operating environments. Such reliance on manual tun-

ing limits generalization, increases engineering overhead, and complicates real-world deployment.

Recent progress in learning-based methods [61, 70] offers a promising alternative. By leveraging learning-based representations, these approaches learn rich visual features as well as uncertainty characteristics from data, capturing complex effects that are difficult to encode manually. In parallel, recent learning-based inertial odometry methods have begun to predict and propagate inertial uncertainty more faithfully (e.g., AirIMU [61]). Together, these advances motivate us to incorporate learned visual features and IMU models, along with their corresponding uncertainty estimates, into a tuning-free framework for visual-inertial initialization and online calibration. By modeling sensor uncertainty in a metrics-aware manner, we enable camera-IMU initialization and calibration that generalizes across environments without manual parameter tuning.

Motivated by these observations, this chapter presents MAC-I^2, a tuning-free VI initialization and online calibration framework that leverages learned visual and IMU covariances for principled sensor fusion. Building on MAC-VO (Chapter 3) and AirIMU (Chapter 5), we derive visual pose covariances from learned feature-matching uncertainties and adopt a learning-based IMU model to predict IMU integration covariances. Both visual and inertial covariances are metrics-aware, enabling principled and tuning-free VI initialization and calibration. Our learned covariances are metrics-aware across testing sequences, enabling effective fusion without manual tuning, and the system runs in real time on Jetson Orin. The main contributions are as follows:

- We propose MAC-I^2, a tuning-free VI initialization and online calibration framework that uses learning-based visual and IMU features.
- We introduce metrics-aware visual and inertial uncertainty models for initialization and calibration, achieving optimal performance without human re-tuning.
- We demonstrate significantly improved robustness and initialization success rates over state-of-the-art methods in challenging scenarios involving illumination changes, dynamic objects, and occlusions.
- Ablation studies and analysis validate the effectiveness of the learned covariance and their metrics-aware properties.

7.2 Related Work

7.2.1 Visual Inertial Initialization

Visual-inertial initialization is one of the most critical components in state estimation for robotics [6, 72], where initialization failures can propagate catastrophically through downstream processing. Early closed-form approaches [153–155] offered computational efficiency but suffered from robustness limitations due to unbiased gyroscope assumptions and failure to account for IMU measurement noise and feature correspondence uncertainties. Modern optimization-based methods address these limitations through MAP estimation [72] that explicitly models IMU uncertainty, probabilistic constraints [156] that handle feature correspondence noise, and decoupled solutions [157] that prevent error coupling between rotation and translation estimation. Robustness has been further enhanced through stereo approaches that provide additional geometric constraints [158, 159], multi-feature integration that reduces reliance on single feature types [160],

architectural innovations that prevent error coupling [161, 162], and foundational systems that integrate these principles [6, 134, 163]. All these methods aim to extract more robust visual features for initialization by diversifying constraints, improving feature quality, and handling uncertainties. Despite these advances, VI initialization remains limited by its reliance on good visual feature matching and static environment assumptions [164], making it vulnerable to dynamic objects, illumination changes, and motion blur.

7.2.2 Visual IMU Extrinsic Calibration

Traditional visual-IMU extrinsic calibration methods rely on offline calibration procedures using calibration targets and known geometric patterns [165]. Classical approaches like Kalibr [111] require structured calibration sequences with checkerboard patterns to establish spatial and temporal relationships between camera and IMU sensors. Online extrinsic calibration methods have emerged to address the limitations of offline approaches, with techniques that integrate calibration into the SLAM framework [166] and enable automatic parameter estimation without calibration targets. Recent advances include combined dynamic and static calibration frameworks [167], multi-sensor calibration systems for visual-IMU-wheel encoders [168], and temporal calibration methods that handle time synchronization between sensors [169]. However, these traditional methods often require structured environments.

7.2.3 Learning-based Algorithms

Recent advances in learning-based approaches have demonstrated significant potential for improving IMU modeling and uncertainty estimation in calibration contexts. Methods like AirIMU [61] leverage neural networks to learn sophisticated IMU noise models and provide more accurate uncertainty estimates compared to traditional analytical models. Learning-based approaches have also been applied to feature extraction and matching [170], with rotation-equivariant features [171] and cycle-correspondence learning [172], improving robustness in challenging calibration scenarios. By incorporating learned priors about sensor behavior and environmental factors, these approaches can better handle the complex, non-linear relationships in sensor calibration, ultimately leading to more robust parameter estimation across diverse operating conditions.

7.3 Notations and Preliminaries

7.3.1 Notations

For VI initialization and calibration, we define the state as:

$$\begin{aligned} \mathcal{X} &= \big[\mathbf{x}_0, \mathbf{x}_1, \dots \mathbf{x}_n, \mathbf{g}^{c_0}, \mathbf{b}^g, \mathbf{b}^a, \mathbf{T}_c^b\big], \\ \mathbf{x}_k &= \big[\mathbf{T}_{b_k}^{c_0}, \mathbf{v}_{b_k}^{b_k}\big], \end{aligned} \tag{7.1}$$

where $\mathbf{T}_{b_k}^{c_0} \in \mathrm{SE}(3)$ denotes the pose of the k-th IMU frame with respect to the first camera frame. $\mathbf{v}_{b_k}^{b_k}$ denotes the velocity of the k-th IMU frame expressed in its local frame. $\mathbf{g}^{c_0}$ denotes the gravity vector expressed in the first camera frame. $\mathbf{b}^g$ and $\mathbf{b}^a$ denote the gyroscope and accelerometer biases, respectively, which are assumed to remain constant during the initialization

and calibration process. $\mathbf{T}_{\mathbf{c}}^{\mathbf{b}}$ denotes the extrinsic transformation from the camera frame to the IMU frame. For initialization with known camera-IMU extrinsic, $\mathbf{T}_{\mathbf{c}}^{\mathbf{b}}$ can be removed from the state variables.

7.3.2 IMU Integration and Learning-based IMU Model

The IMU integration follows the standard approach on SO(3) manifold as proposed in [173]:

$$\begin{aligned}
\mathbf{q}_{b_j}^{c_0} &= \mathbf{q}_{b_i}^{c_0} \boldsymbol{\gamma}_{b_j}^{b_i}, \\
\mathbf{v}_{b_j}^{c_0} &= \mathbf{v}_{b_i}^{c_0} + \mathbf{g}^{c_0} \Delta t_{ij} + \mathbf{R}_{b_i}^{c_0} \boldsymbol{\beta}_{b_j}^{b_i}, \\
\mathbf{p}_{b_j}^{c_0} &= \mathbf{p}_{b_i}^{c_0} + \mathbf{v}_{b_i}^{c_0} \Delta t_{ij} + \frac{1}{2} \mathbf{g}^{c_0} \Delta t_{ij}^2 + \mathbf{R}_{b_i}^{c_0} \boldsymbol{\alpha}_{b_j}^{b_i},
\end{aligned} \tag{7.2}$$

where $\mathbf{q}_{b_i}^{c_0} \in \mathrm{SO}(3)$ denotes the rotation quaternion from the i-th body frame to the initial camera frame c_0, and $\mathbf{R}_{b_i}^{c_0}$ denotes its matrix form. $\boldsymbol{\gamma}_{b_j}^{b_i}$, $\boldsymbol{\beta}_{b_j}^{b_i}$, and $\boldsymbol{\alpha}_{b_j}^{b_i}$ denote IMU preintegration terms for rotation, velocity, and translation, defined as:

$$\begin{aligned}
\boldsymbol{\gamma}_{b_j}^{b_i} &= \prod_{k=i}^{j-1} \mathrm{Exp}\left(\boldsymbol{\omega}_k \Delta t\right), \\
\boldsymbol{\beta}_{b_j}^{b_i} &= \sum_{k=i}^{j-1} \mathbf{R}_{b_k}^{b_i} \mathbf{a}_k \Delta t, \\
\boldsymbol{\alpha}_{b_j}^{b_i} &= \sum_{k=i}^{j-1} \left(\boldsymbol{\beta}_{b_k}^{b_i} \Delta t + \frac{1}{2} \mathbf{R}_{b_k}^{b_i} \mathbf{a}_k \Delta t^2 \right),
\end{aligned} \tag{7.3}$$

where $\mathbf{a}$ and $\boldsymbol{\omega}$ denote the raw accelerometer and gyroscope measurements, which include gravity, bias, and noise effects.

AirIMU [61] is a learning-based model to estimate the corrections $[\boldsymbol{\sigma}^{\mathrm{acc}}, \boldsymbol{\sigma}^{\mathrm{gyro}}]$ and the uncertainty $[\boldsymbol{\eta}^{\mathrm{acc}}, \boldsymbol{\eta}^{\mathrm{gyro}}]$ of the IMU measurements, with the input as the raw IMU data $[\mathbf{a}, \boldsymbol{\omega}]$:

$$\begin{aligned}
(\boldsymbol{\sigma}^{\mathrm{acc}}, \boldsymbol{\sigma}^{\mathrm{gyro}}) = g_{\boldsymbol{\theta}}(\mathbf{a}, \boldsymbol{\omega}), \quad (\boldsymbol{\eta}^{\mathrm{acc}}, \boldsymbol{\eta}^{\mathrm{gyro}}) = \Sigma_{\boldsymbol{\theta}}(\mathbf{a}, \boldsymbol{\omega}), \\
\tilde{\mathbf{a}} = \mathbf{a} + \boldsymbol{\sigma}^{\mathrm{acc}} + \boldsymbol{\eta}^{\mathrm{acc}}, \quad \tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \boldsymbol{\sigma}^{\mathrm{gyro}} + \boldsymbol{\eta}^{\mathrm{gyro}},
\end{aligned} \tag{7.4}$$

where $\boldsymbol{\theta}$ is the parameter of the neural network.

The AirIMU jointly supervises the integrated IMU state and the propagated covariance to train the model. The IMU integration in AirIMU is similar to Eq. 7.2, while considering the state in the global world frame. The covariance propagation follows the formulation in [173]:

$$\begin{aligned}
\boldsymbol{\Sigma}_{ik+1} &= \mathbf{A} \boldsymbol{\Sigma}_{ik} \mathbf{A}^T + \mathbf{B} \mathrm{diag}(\boldsymbol{\Sigma}^{gyro}, \boldsymbol{\Sigma}^{acc}) \mathbf{B}^T \\
&= \mathbf{A} \boldsymbol{\Sigma}_{ik} \mathbf{A}^T + \mathbf{B}_g \boldsymbol{\Sigma}^{gyro} \mathbf{B}_g^T + \mathbf{B}_a \boldsymbol{\Sigma}^{acc} \mathbf{B}_a^T,
\end{aligned} \tag{7.5}$$

where $\boldsymbol{\Sigma}^{gyro}$ and $\boldsymbol{\Sigma}^{acc}$ are the measurement covariance, which is obtained with the uncertainty prediction of the network. The propagation starts from $\boldsymbol{\Sigma}_{ii} = \mathbf{0}$. The specific forms of $\mathbf{A}$ and $\mathbf{B}$ are provided in Sec. **??**.

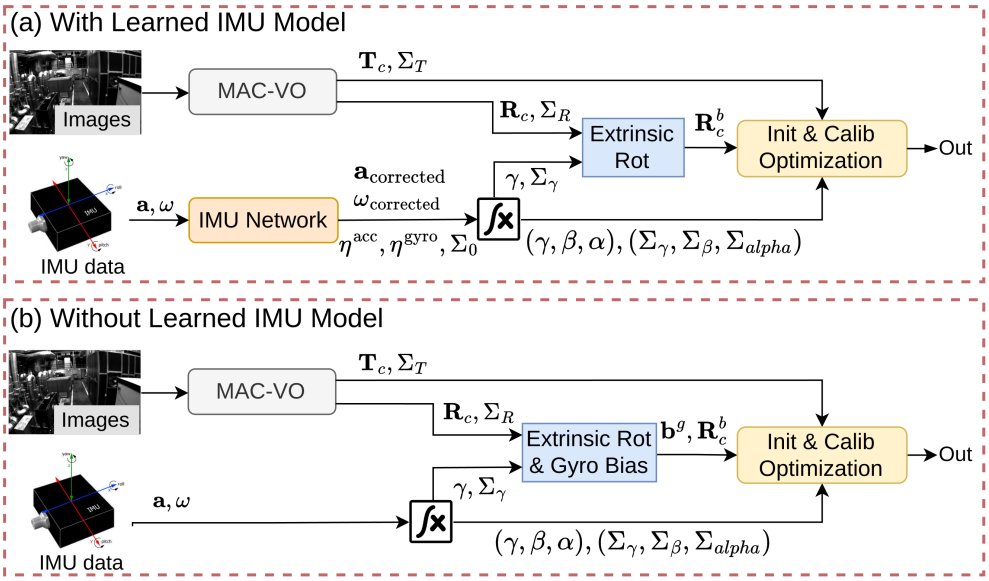

Figure 7.1: (**a**) System overview of the proposed visual-inertial initialization and calibration method with the learned IMU model. (**b**) As the learned model requires training and may not always be available, we also propose an alternative workflow that does not rely on the learned IMU model.

7.3.3 Learning-based Visual Odometry MAC-VO

MAC-VO [70] is a learning-based stereo visual odometry that learns a metrics-aware uncertainty model to select reliable keypoints and weight residuals in pose graph optimization, achieving robust and accurate visual pose estimation in challenging environments.

Specifically, MAC-VO estimates the camera poses by solving the following pose graph optimization (PGO) problem for consecutive frames c_{k-1} and c_k:

$$\begin{aligned} \mathbf{T}_{c_k}^{c_{k-1}\star} &= \underset{\mathbf{T}_{c_k}^{c_{k-1}}}{\arg\min} \sum_i \left\| \mathbf{p}_{k-1,i}^c - \mathbf{T}_{c_k}^{c_{k-1}} \mathbf{p}_{k,i}^c \right\|_{\mathbf{\Sigma}_i}^2, \\ \mathbf{\Sigma}_i &= \mathbf{\Sigma}_{i,k-1}^p + \mathbf{R}_{c_k}^{c_{k-1}} \mathbf{\Sigma}_{i,k}^p (\mathbf{R}_{c_k}^{c_{k-1}})^T, \end{aligned} \tag{7.6}$$

where (k, i) denotes the i-th feature in frame c_k, and $\|\cdot\|_{\mathbf{\Sigma}_i}$ represents the Mahalanobis distance with covariance matrix $\mathbf{\Sigma}_i$. $\mathbf{\Sigma}_{i,k}^p$ denotes the learned metrics-aware 3D covariance of the i-th feature in frame c_k.

7.4 Methodology

7.4.1 System Overview

The proposed VI initialization and calibration system is illustrated in Figure 7.1. The method consists of four components: (1) visual odometry with uncertainty estimation (MAC-VO), (2) IMU processing with learned correction and uncertainty prediction, (3) gyroscope bias and extrinsic rotation estimation, and (4) tuning-free VI initialization and calibration.

As the learned IMU model requires training data and may not always be available, we provide two workflows (Figure 7.1):

(a) With Learned IMU Model (Figure 7.1a): The learned IMU network processes raw IMU data ($\mathbf{a}$, $\boldsymbol{\omega}$) to output corrected preintegration terms and predicted uncertainties ($\boldsymbol{\eta}^{\text{gyro}}, \boldsymbol{\eta}^{\text{acc}}$). Since measurements are already corrected, IMU biases are not explicitly estimated in this workflow.

(b) Without Learned IMU Model (Figure 7.1b): Raw IMU data undergoes standard preintegration. Gyroscope bias $\mathbf{b}^g$ is explicitly estimated during initialization, and IMU uncertainty is pre-calibrated or manually tuned.

In both workflows, MAC-VO provides visual pose estimates $\mathbf{T}_c$ with uncertainties $\boldsymbol{\Sigma}_{\mathbf{T}}$.

7.4.2 Camera Motion Estimation and Pose Covariance

Building upon MAC-VO, we model the covariance of estimated visual poses using information matrices derived from the learned 3D feature uncertainty. Notably, despite being trained solely on synthetic data, our covariance model produces metric-aware pose uncertainties on real-world datasets.

The covariance of the estimated pose $\boldsymbol{\Sigma}_{\mathbf{T}_{c_k}^{c_{k-1}}}$ can be obtained via the Jacobian computed at convergence of the PGO problem in Eq. 7.6. We consider the left perturbation model $\mathbf{r}_{i,k}(\delta\xi) = \mathbf{p}_{k-1,i}^c - \text{Exp}(\boldsymbol{\delta\xi})\mathbf{T}_{c_k}^{c_{k-1}}\mathbf{p}_{k,i}^c$, which yields:

$$\begin{aligned} \mathbf{J}_{i,k} &= \frac{\partial \mathbf{r}_{i,k}(\boldsymbol{\delta\xi})}{\partial \boldsymbol{\delta\xi}} = \begin{bmatrix} -\mathbf{I}_{3\times 3} & (\mathbf{T}_{c_k}^{c_{k-1}}\mathbf{p}_{k,i}^c)^\wedge \end{bmatrix}, \\ \mathbf{H}_k &\approx \sum_i (\mathbf{J}_{i,k})^T \boldsymbol{\Sigma}_i^{-1} \mathbf{J}_{i,k}, \quad \boldsymbol{\Sigma}_{\mathbf{T}_{c_k}^{c_{k-1}}} \approx \mathbf{H}_k^{-1}, \end{aligned} \tag{7.7}$$

where $\mathbf{J}_{i,k}$ denotes the left Jacobian of the residual with respect to a left-multiplicative perturbation $\boldsymbol{\delta\xi}$ in the tangent space of the estimated visual pose, and $\boldsymbol{\Sigma}_i$ denotes the covariance matrix defined in Eq. 7.6.

We validate that the estimated visual pose covariance is metrics-aware, as illustrated in Figure 7.2. Specifically, we group the pose estimations into bins according to their predicted uncertainty and compute the mean translation and rotation error within each bin. The results show that the estimated uncertainty closely follows the ideal relationship of $y = x$, indicating a strong correlation between predicted uncertainty and actual pose error, with low-error samples showing low uncertainty and high-error samples showing high uncertainty.

Notably, our model is trained entirely on the synthetic TartanAir [174] dataset without any fine-tuning on real-world data, demonstrating strong generalization capability.

7.4.3 Learning-based Metrics-Aware Covariance for IMU

Observation on IMU Integration Error. We find that, after bias correction, the IMU integration error exhibits a consistent temporal pattern: a relatively large error at the beginning of the integration window, followed by a much slower growth thereafter, as shown in Figure 7.3.

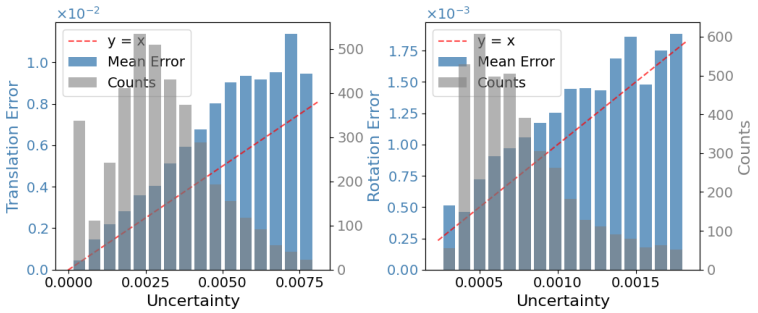

Figure 7.2: Translation and rotation pose errors versus predicted visual pose covariance on EuRoC. The estimated uncertainty exhibits metrics-aware behavior, closely following the ideal $y = x$ and generalizes from synthetic to real-world data.

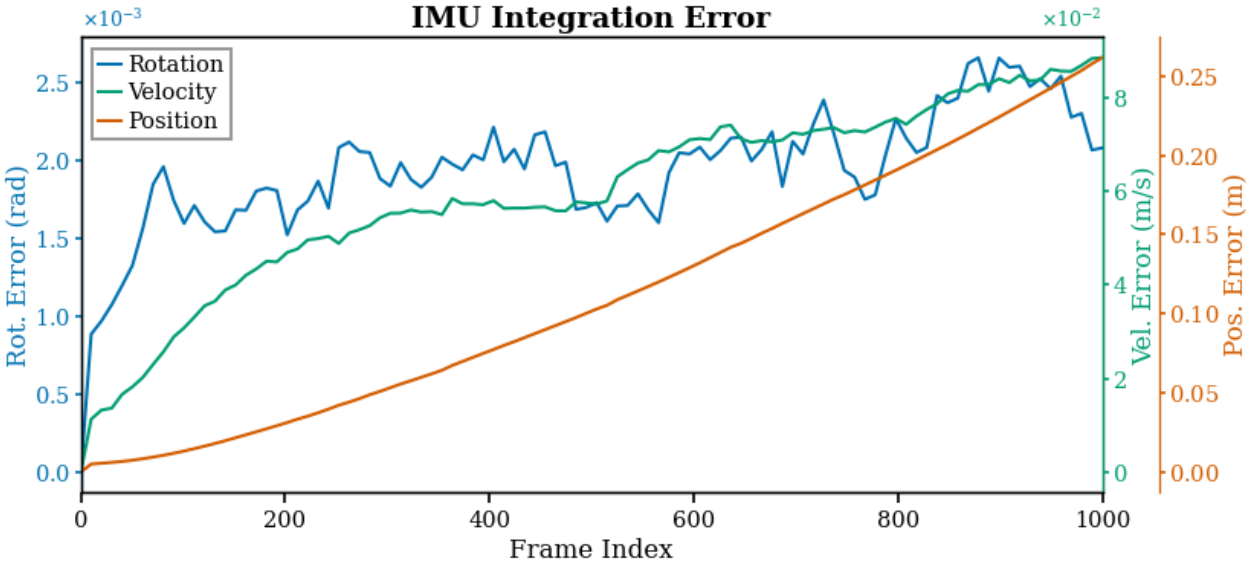

Figure 7.3: Average IMU integration error with ground-truth bias correction on the EuRoC MH_04_difficult sequence over multiple 1000-frame windows.

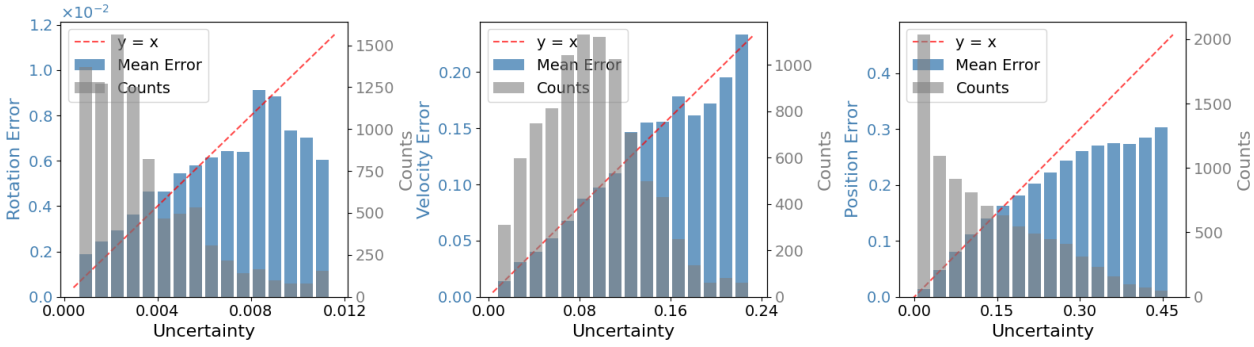

Figure 7.4: Metrics-aware IMU covariance estimation on unseen EuRoC sequences: `MH_02`, `MH_04`, `V1_01`, `V1_03`, and `V2_02`.

This pattern is particularly evident for rotational integration.

Learnable Initial Covariance. AirIMU implicitly learns to explain the observed integration error pattern by assigning significantly higher uncertainty to the initial frames with gated recurrent unit (GRU) encoding. However, when predicting uncertainty for a preintegration segment, the model must start from the first frame of that segment, preventing it from leveraging information from earlier measurements.

To address this issue, we introduce a learnable initial covariance $\mathbf{\Sigma}_0$, which serves as the starting covariance for the propagation process defined in Eq. 7.5. Moreover, we remove the GRU from the uncertainty prediction branch and instead employ only a CNN encoder shared with the correction prediction. This design enables the network to exploit the hidden relationship between IMU correction and uncertainty, while preventing it from relying on temporal encoding to reproduce the aforementioned predictions of AirIMU.

Covariance Fine-tuning. On unseen test sequences, the predicted covariance tends to be overconfident. To mitigate this issue, we split the training set into two subsets. The first subset is used to train the full network, while the second is used to fine-tune the learnable initial covariance and the uncertainty decoder, with all other network parameters frozen.

Additional Refinements. We further identify two additional factors that affect covariance accuracy. First, AirIMU directly uses the preintegration covariance propagated via Eq. 7.5 as the covariance of the residual; instead, we rigorously derive the residual loss covariance. Second, we observe that employing a higher-frequency encoder leads to improved correction and uncertainty prediction performance. The residual covariance and network architecture are provided in Sec. **??** and Sec. **??**.

Our final network achieves significant improvements in both IMU correction and uncertainty accuracy compared to AirIMU. In particular, the predicted uncertainties exhibit strong metrics-awareness on unseen test sets, as illustrated in Figure 7.4. The predicted metric-aware covariance is then used in subsequent VI initialization and calibration tasks.

7.4.4 Gyroscope Bias and Extrinsic Rotation Estimation

When the learned IMU model is unavailable, we explicitly estimate the gyroscope bias $\mathbf{b}^g$ at this stage.

In addition, when performing the calibration task, we also estimate the camera-IMU rotation $\mathbf{q}_b^c$. This can be achieved by minimizing the difference between the rotation from vision and that from IMU integration.

$$\begin{aligned} &\min_{\delta\mathbf{b}^g, \mathbf{q}_\mathbf{b}^\mathbf{c}} \sum_{k\in\mathcal{B}} \left\| \mathrm{Log}(\mathbf{q}_c^b \mathbf{q}_{c_k}^{c_{k+1}} (\mathbf{q}_c^b)^{-1} \hat{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k}) \right\|_{\mathbf{\Sigma}_{r_k}}^2 , \\ &\hat{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k} = \boldsymbol{\gamma}_{b_{k+1}}^{b_k} \mathrm{Exp}\left(\mathbf{J}_{\mathbf{b}^g}^{\gamma} \delta\mathbf{b}^g\right), \end{aligned} \tag{7.8}$$

where $\delta\mathbf{b}^g$ is the increment of the gyroscope bias. In this optimization, we set the initial gyroscope bias to be zero, therefore we have $\mathbf{b}^g = \delta\mathbf{b}^g$. $\mathcal{B}$ is the set of all the keyframes. $\boldsymbol{\gamma}_{b_{k+1}}^{b_k}$ is the rotation preintegration of the raw IMU measurements, $\hat{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k}$ is the first-order approximation of the rotation preintegration after gyroscope bias correction, referring to [173].

When a new keyframe is received, this estimation continues to run until the termination con-

dition is satisfied. The inverse of Hessian matrix naturally tells the uncertainty of the estimation. Following [175], we use the maximum singular value σ_{max} of the matrix as the convergence indicator and terminate the estimation process if $\sigma_{max} < \sigma_{th}$, where σ_{th} is a threshold. The estimates in this stage serve as the initial values for the following tightly-coupled optimization.

7.4.5 Tuning-free VI Initialization and Calibration

The final stage of the proposed VI initialization and calibration method performs an optimization that jointly estimates the state variables $\mathcal{X}$ defined in Eq. 7.1. We minimize a weighted sum of visual and inertial residuals:

$$\min_{\mathcal{X}} \sum_{k \in \mathcal{B}} \left\| \mathbf{r}_k^{\text{visual}} \right\|^2_{\mathbf{\Sigma}_k^{\text{visual}}} + \left\| \mathbf{r}_k^{\text{imu}} \right\|^2_{\mathbf{\Sigma}_k^{\text{imu}}}. \tag{7.9}$$

Visual Residual. Unlike methods that directly constrain raw visual features, our visual residual operates at the pose level. This preserves feature uncertainty information through derived pose covariance while simplifying the nonlinear optimization, making it particularly suitable for initialization. The visual residual $\mathbf{r}_k^{\text{visual}}$ requires consistency between relative camera motion from visual odometry and motion induced by the estimated inertial states:

$$\mathbf{r}_k^{\text{visual}} = \text{Log}(\mathbf{T}_{c_{k+1}}^{c_k} \mathbf{T}_b^c (\mathbf{T}_{b_{k+1}}^{c_0})^{-1} \mathbf{T}_{b_k}^{c_0} \mathbf{T}_c^b). \tag{7.10}$$

The covariance of the formulated visual residual is computed using the left Jacobians of SE(3):

$$\begin{aligned} \mathbf{\Sigma}_k^{\text{visual}} &= \frac{\partial \mathbf{r}_k^{\text{visual}}}{\partial \boldsymbol{\delta\xi}} \mathbf{\Sigma}_{\mathbf{T}_{c_k}^{c_{k-1}}} (\frac{\partial \mathbf{r}_k^{\text{visual}}}{\partial \boldsymbol{\delta\xi}})^T, \\ \frac{\partial \mathbf{r}_k^{\text{visual}}}{\partial \boldsymbol{\delta\xi}} &= \mathbf{J}_l^{-1}(\mathbf{r}_k^{\text{visual}}), \end{aligned} \tag{7.11}$$

where $\mathbf{\Sigma}_{\mathbf{T}_{c_k}^{c_{k-1}}}$ is the visual pose covariance derived in Eq. 7.7, and $\mathbf{J}_l^{-1}(\mathbf{r}_k^{\text{visual}})$ denotes the inverse left Jacobian of $SE(3)$ evaluated at $\mathbf{r}_k^{\text{visual}}$.

IMU Residual with Learned IMU Model. When the learned IMU model is available, explicit IMU bias estimation is unnecessary, as the network provides corrected measurements. The IMU residual $\mathbf{r}_k^{\text{imu}} = \left[\mathbf{r}_{\boldsymbol{\gamma}_k}^T, \mathbf{r}_{\boldsymbol{\beta}_k}^T, \mathbf{r}_{\boldsymbol{\alpha}_k}^T\right]^T$ is derived from Eq. 7.2:

$$\mathbf{r}_{\boldsymbol{\gamma}_k} = \text{Log}(\mathbf{R}_{c_0}^{b_{k+1}} \mathbf{R}_{b_k}^{c_0} \tilde{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k}), \tag{7.12}$$

$$\mathbf{r}_{\boldsymbol{\beta}_k} = \mathbf{R}_{c_0}^{b_k} \mathbf{R}_{b_{k+1}}^{c_0} \mathbf{v}_{b_{k+1}}^{b_{k+1}} - \mathbf{v}_{b_k}^{b_k} - \mathbf{R}_{c_0}^{b_k} \mathbf{g}^{c_0} \Delta t_{kk+1} - \tilde{\boldsymbol{\beta}}_{b_{k+1}}^{b_k}, \tag{7.13}$$

$$\begin{aligned} \mathbf{r}_{\boldsymbol{\alpha}_k} = {} & \mathbf{R}_{c_0}^{b_k} \mathbf{p}_{b_{k+1}}^{c_0} - \mathbf{R}_{c_0}^{b_k} \mathbf{p}_{b_k}^{c_0} - \mathbf{v}_{b_k}^{b_k} \Delta t_{kk+1} \\ & - \frac{1}{2} \mathbf{R}_{c_0}^{b_k} \mathbf{g}^{c_0} \Delta t_{kk+1}^2 - \tilde{\boldsymbol{\alpha}}_{b_{k+1}}^{b_k}, \end{aligned} \tag{7.14}$$

where $\tilde{\boldsymbol{\gamma}}_{b_{k+1}}^{b_k}$, $\tilde{\boldsymbol{\beta}}_{b_{k+1}}^{b_k}$, $\tilde{\boldsymbol{\alpha}}_{b_{k+1}}^{b_k}$ denote the corrected IMU preintegrations. The covariance of the IMU residuals $\mathbf{\Sigma}_k^{\text{imu}}$ is derived from the predicted preintegration covariance (see Sec. **??**).

IMU Residual without Learned IMU Model. Without a learned IMU model, we model the

IMU biases in the residue with first-order corrections:

$$\mathbf{r}_{\boldsymbol{\gamma}_k} = \mathrm{Log}(\mathbf{R}_{c_0}^{b_{k+1}} \mathbf{R}_{b_k}^{c_0} \boldsymbol{\gamma}_{b_{k+1}}^{b_k} \mathrm{Exp}(\mathbf{J}_{\mathbf{b}^g}^{\boldsymbol{\gamma}} \delta \mathbf{b}^g)), \tag{7.15}$$

$$\begin{aligned} \mathbf{r}_{\boldsymbol{\beta}_k} = & \mathbf{R}_{c_0}^{b_k} \mathbf{R}_{b_{k+1}}^{c_0} \mathbf{v}_{b_{k+1}}^{b_{k+1}} - \mathbf{v}_{b_k}^{b_k} - \mathbf{R}_{c_0}^{b_k} \mathbf{g}^{c_0} \Delta t_{kk+1} \\ & - (\boldsymbol{\beta}_{b_{k+1}}^{b_k} + \mathbf{J}_{\mathbf{b}^g}^{\boldsymbol{\beta}} \delta \mathbf{b}^g + \mathbf{J}_{\mathbf{b}^a}^{\boldsymbol{\beta}} \delta \mathbf{b}^a), \end{aligned} \tag{7.16}$$

$$\begin{aligned} \mathbf{r}_{\boldsymbol{\alpha}_k} = & \mathbf{R}_{c_0}^{b_k} \mathbf{p}_{b_{k+1}}^{c_0} - \mathbf{R}_{c_0}^{b_k} \mathbf{p}_{b_k}^{c_0} - \mathbf{v}_{b_k}^{b_k} \Delta t_{kk+1} - \frac{1}{2} \mathbf{R}_{c_0}^{b_k} \mathbf{g}^{c_0} \Delta t_{kk+1}^2 \\ & - (\boldsymbol{\alpha}_{b_{k+1}}^{b_k} + \mathbf{J}_{\mathbf{b}^g}^{\boldsymbol{\alpha}} \delta \mathbf{b}^g + \mathbf{J}_{\mathbf{b}^a}^{\boldsymbol{\alpha}} \delta \mathbf{b}^a). \end{aligned} \tag{7.17}$$

When performing initialization with known extrinsics, we exclude the accelerometer bias from state variables, as the short initialization window typically lacks sufficient excitation to decouple it from gravity. Analytical Jacobians for all residuals are provided in Sec. **??**.

7.5 Experiments

7.5.1 Experiment Setup

Table 7.1: Detailed initialization results for the 10KFs setting in EuRoC dataset. Only even-ordered trajectory is shown due to page limit. G.Dir Error $>$ 5 ° or Vel RMSE $>$ 0.5 m/s is treated as failure.

Trajectory	MH02		MH04		V101		V103		V202		Avg.		
	Vel(m/s)	G.Dir(°)	Vel(m/s)	G.Dir(°)	Vel(m/s)	G.Dir(°)	Vel(m/s)	G.Dir(°)	Vel(m/s)	G.Dir(°)	Vel(m/s)	G.Dir(°)	SR
[M] VINS-Mono	0.136	2.651	0.266	3.225	0.052	2.824	-	-	0.251	3.803	0.176	3.126	**0.36**
[M] DRT-l	0.049	0.927	0.106	0.944	0.047	1.011	0.095	1.304	0.054	0.896	0.070	1.016	0.74
[S] VINS-Fusion	0.120	2.734	0.217	2.960	0.017	2.874	0.183	2.867	0.231	3.844	0.154	3.056	**0.48**
[S] ORB-SLAM 3	0.146	2.188	0.229	2.406	0.087	2.111	0.290	2.554	0.325	3.000	0.215	2.452	0.61
[S] Stereo-NEC	0.148	2.203	0.226	2.439	0.090	2.158	0.273	2.462	0.330	2.839	0.213	2.420	**0.60**
[S] **MAC-I^2 w/o Learned IMU**	0.011	0.820	**0.022**	0.800	**0.010**	0.872	**0.025**	1.149	**0.020**	**0.804**	**0.018**	0.889	**1.00**
[S] **MAC-I^2 w/ Learned IMU**	**0.010**	**0.166**	0.023	**0.208**	**0.010**	**0.332**	**0.025**	**0.581**	0.024	**0.804**	**0.018**	**0.418**	**1.00**

[M] Monocular method. [S] Stereo method. - The initialization fails on all segments, or succeeds on too few segments to be statistically meaningful.

Table 7.2: Detailed initialization results for the 10KFs setting in TUM-VI dataset. G.Dir Error $>$ 5 ° is treated as failure.

Trajectory	Room2	Room4	Room6	Test Avg.	
	G.Dir(°)	G.Dir(°)	G.Dir(°)	G.Dir(°)	SR
[M] VINS-Mono	2.808	2.254	3.426	2.829	**0.08**
[M] DRT-l	3.210	2.734	2.449	2.798	**0.36**
[S] VINS-Fusion	3.297	3.519	3.518	3.445	**0.04**
[S] ORB-SLAM 3	1.994	2.787	2.521	2.434	0.48
[S] Stereo-NEC	2.078	2.646	2.441	2.388	0.41
[S] **MAC-I^2 w/o Learned IMU**	1.461	1.277	0.871	1.203	0.94
[S] **MAC-I^2 w/ Learned IMU**	**0.957**	**1.121**	**0.653**	**0.910**	**0.96**

Datasets. We evaluate the proposed VI initialization and calibration method using different datasets tailored to each task. For VI initialization, we conduct experiments on three public benchmarks: EuRoC [97], TUM-VI [133], and VBR [176]. EuRoC is a Micro Aerial Vehicle (MAV) dataset with diverse motion patterns. For TUM-VI, we use 6 motion-capture room

sequences, which provide ground-truth poses for the entire trajectory. The VBR dataset spans large-scale and dynamic scenarios, including multi-floor buildings, gardens, urban areas, and highways, and includes both handheld and vehicle-mounted data. We use the 8 sequences that provide ground-truth poses for evaluation. For camera-IMU extrinsic calibration, we evaluate on 4 IMU calibration sequences (imu-calib) from TUM-VI, which contain rapid and diverse motions that excite all six degrees of freedom.

Baselines. For initialization, we compare with several state-of-the-art geometric-based methods, including *VINS-Mono* and *VINS-Fusion* [6], two widely used monocular and stereo VI SLAM systems with distinct initialization strategies: *VINS-Fusion* directly performs tightly-coupled optimization after gyroscope bias estimation, whereas *VINS-Mono* adopts a loosely-coupled estimation of initial gravity and velocity. We also include *ORB-SLAM3*[72], which uses an inertial-only initialization, and *DRT-l*[157], a robust method that decouples rotation and translation, and estimates gyroscope bias using the Normal Epipolar Constraint (NEC). In addition, *Stereo-NEC*[158] is considered, which integrates NEC into the stereo *ORB-SLAM3* framework to improve initialization accuracy and robustness. For calibration, we compare with *VINS-Mono* as it is equipped with a complete online calibration solution, and use the calibration results computed by *Kalibr*[165] as ground truth.

Evaluation Metrics. For VI initialization, accurately estimating the initial system states and sensor parameters is important. Accordingly, we adopt the following evaluation metrics:

- Gravity direction error: $\arccos(\mathbf{g}^{c_0} \cdot \tilde{\mathbf{g}}^{c_0})$, where $\mathbf{g}^{c_0}$, $\tilde{\mathbf{g}}^{c_0}$ denote the estimated and ground truth gravity.
- Velocity RMSE: $\|\mathbf{v}_{b_k}^{b_k} - \tilde{\mathbf{v}}_{b_k}^{b_k}\|/N$, where $\mathbf{v}_{b_k}^{b_k}$, $\tilde{\mathbf{v}}_{b_k}^{b_k}$ denote the estimated and ground truth body velocity of frame k, N is the total number of keyframes.
- Gyroscope bias error: following [156, 157], let $\bar{\mathbf{b}}^g$ denote the mean of all biases along the ground-truth trajectory, and the error is defined as the relative deviation $\left|\|\mathbf{b}^g\| - \|\bar{\mathbf{b}}^g\|\right| / \|\bar{\mathbf{b}}^g\|$.
- Success rate (SR): the number of successfully initialized segments divided by the total number of segments.

For calibration, we also evaluate the estimated extrinsics with:

- Rotation error: $\|\mathrm{Log}((\mathbf{q}_c^b)^{-1} * \tilde{\mathbf{q}}_c^b)\| \cdot \frac{180}{\pi}$, where $\mathbf{q}_c^b$, $\tilde{\mathbf{q}}_c^b$ denote the estimated and ground truth extrinsic rotation.
- Translation error: $\|\mathbf{p}_c^b - \tilde{\mathbf{p}}_c^b\|$, where $\mathbf{p}_c^b$, $\tilde{\mathbf{p}}_c^b$ denote the estimated and ground truth extrinsic translation.

Training Details. For the visual features and uncertainty, the model is trained on the synthetic TartanAir dataset [174]. For the learning-based IMU model, different sequences are used for training and fine-tuning. On EuRoC, the entire network is trained using MH_01_easy, MH_05_difficult, V2_01_easy, and V2_03_difficult, followed by the uncertainty fine-tuning on MH_03_medium and V1_02_medium. For the TUM-VI room sequences, the full network is trained on dataset-room1_512_16 and dataset-room3_512_16, and the uncertainty is fine-tuned on dataset-room5_512_16.

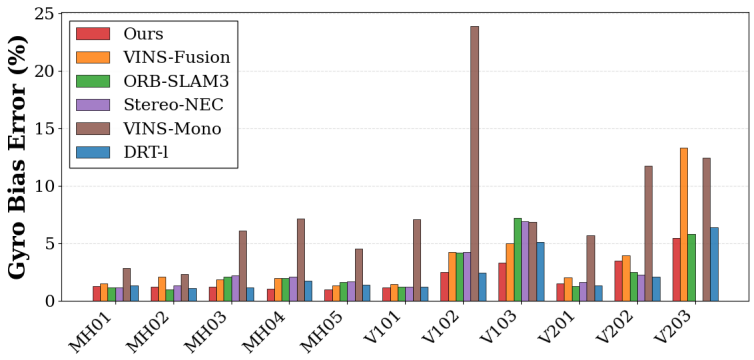

Figure 7.5: Comparison of gyroscope bias magnitude estimation on all the sequences of EuRoC dataset.

7.5.2 Initialization Accuracy and Robustness Evaluation

For the initialization experiments, each sequence is divided into multiple 5-second segments, and initialization is performed on each segment with 10 keyframes. For our method with the learned IMU model, evaluation is performed only on sequences that were not used for training. A segment is considered successfully initialized if the initialization variables are successfully solved and the resulting errors are within certain thresholds. For monocular methods, we set a scale error threshold following [157]. The other thresholds are reported in Tables 7.1, 7.2, and 7.3.

For EuRoC, the results are presented in Table 7.1. Our method demonstrates substantial improvements over state-of-the-art geometric-based approaches in both accuracy and robustness. Specifically, *MAC-I*^{2} *w/ Learned IMU* attains an average gravity direction error of only 0.418° and velocity RMSE of 0.018 m/s, representing improvements of approximately 59% and 74% over the best baseline *DRT-l* (1.016° and 0.070 m/s, respectively). Even without the learned IMU model, *MAC-I*^{2} *w/o Learned IMU* achieves competitive performance with 0.889° and 0.018 m/s, substantially outperforming other methods.

Notably, our method achieves a 1.00 success rate across all segments, significantly outperforming competing approaches. In comparison, DRT-l attains a 0.74 success rate, while VINS-Fusion and ORB-SLAM3 achieve 0.48 and 0.61, respectively. The performance gap is especially pronounced in challenging sequences. For example, on V103, DRT-l, VINS-Fusion, and ORB-SLAM3 achieve success rates of only 0.41, 0.11, and 0.25, respectively. These results demonstrate the superior robustness of our approach.

In addition, we evaluate gyroscope bias estimation, which has been shown to play a critical role in VI initialization [157]. The results are illustrated in Figure 7.5. Our method consistently estimates the gyroscope bias accurately on all the sequences.

Table 7.2 summarizes the initialization performance on TUM-VI across six room sequences. Since TUM-VI does not provide ground truth velocity, we focus our evaluation on initial gravity estimation. The results demonstrate the superior accuracy and robustness of the proposed method. *MAC-I*^{2} *w/ Learned IMU* attains an average gravity direction error of only 0.910° with an impressive 0.96 success rate, significantly outperforming all baseline methods. Even for *MAC-I*^{2} *w/o Learned IMU*, it still outperforms all the baseline methods on all the sequences.

For the VBR dataset, we observe that baseline methods struggle to achieve reliable and accurate initialization. Specifically, *VINS-Mono*, *DRT-l*, and *VINS-Fusion* attain success rates of

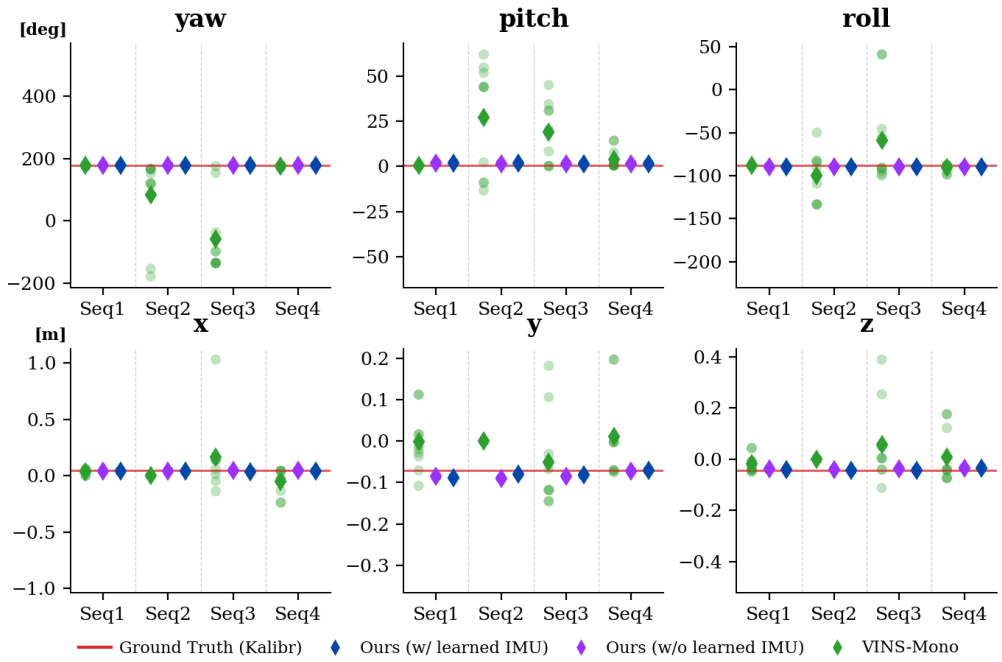

Figure 7.6: Comparison of camera-IMU extrinsic calibration on TUM-VI calib-imu1 sequence.

only 0.07, 0.06, and 0.10, respectively. Moreover, both *ORB-SLAM3* and *Stereo-NEC* fail on all evaluated segments. In contrast, as reported in Table 7.3, *MAC-I*^{2} *w/o Learned IMU* significantly outperforms the baseline methods, achieving a success rate of 0.71 with a gravity direction error of 2.69°. These results further demonstrate the robustness of our method in complex and challenging environments.

Table 7.3: Detailed initialization results for the 10KFs setting in VBR dataset. G.Dir Error > 5 ° is treated as failure.

Method	cam0		cam1		cia1		col0		dia0		pin0		spa0		Avg.	
	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR	G.Dir(°)	SR
VINS-Mono	2.30	0.09	1.87	0.11	3.35	0.05	2.72	0.25	2.35	0.03	3.57	0.04	4.58	0.02	2.96	**0.07**
DRT-l	2.71	0.07	1.00	0.02	4.07	0.03	1.69	0.17	1.19	0.08	4.11	0.11	-	-	2.46	**0.06**
VINS-Fusion	3.03	0.11	2.12	0.07	3.74	0.14	3.05	0.21	2.88	0.19	2.59	0.06	4.32	0.04	3.10	0.10
MAC-I^2 (w/o Learned IMU)	1.80	0.97	1.22	0.99	3.70	0.84	2.32	0.87	1.71	0.95	3.61	0.82	4.44	0.24	2.69	**0.71**

* ORB-SLAM3 and Stereo-NEC fail on all segments.

7.5.3 Calibration Results

We evaluate VI calibration on the IMU calibration sequences from TUM-VI, which include aggressive motions that excite all six degrees of freedom. We compare our method with *VINS-Mono* and use *Kalibr* [165] to provide ground-truth results. Each experiment is repeated 10 times per sequence.

The results are shown in Figure 7.6. Our method consistently produces accurate calibration estimates across all trials. In contrast, *VINS-Mono* fails to maintain tracking in most of the trials. This is because geometric feature-based methods are challenged by the highly aggressive motion patterns in the TUM-VI calibration sequences, which were originally designed for checkerboard-

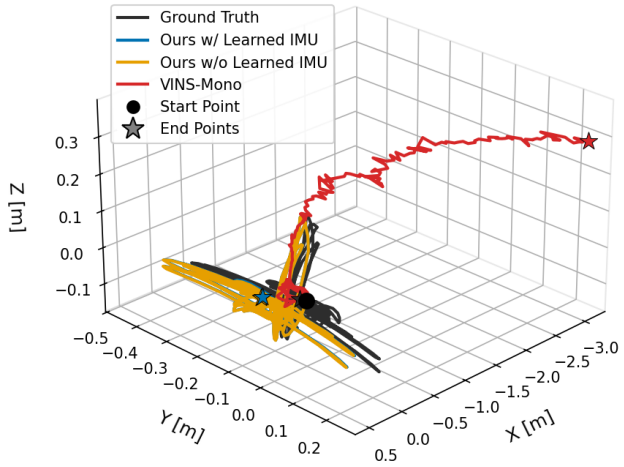

Figure 7.7: Trajectory comparison on the TUM-VI calib-imu1.

Table 7.4: Initialization errors on the EuRoC dataset under different covariance settings.

Metrics	Methods	MH02	MH04	V101	V103	V202	Avg
Velocity (m/s)	Id Vis+ Id IMU	0.013	0.031	**0.010**	0.026	**0.022**	0.021
	Id Vis+ Learned IMU	0.040	0.073	0.037	0.061	0.056	0.053
	Learned Vis+ Id IMU	0.011	**0.021**	0.011	0.027	0.029	0.020
	Learned Vis+ Learned IMU	**0.010**	0.023	**0.010**	**0.025**	0.024	**0.019**
Gravity Direction (deg)	Id Vis+ Id IMU	0.239	0.284	0.353	0.671	**0.766**	0.463
	Id Vis+ Learned IMU	0.284	0.459	0.385	0.655	0.865	0.530
	Learned Vis+ Id IMU	0.231	0.248	0.346	0.768	0.810	0.481
	Learned Vis+ Learned IMU	**0.166**	**0.208**	**0.332**	**0.581**	0.804	**0.418**
Body Rotation (deg)	Id Vis+ Id IMU	0.193	0.264	0.224	0.597	0.534	0.362
	Id Vis+ Learned IMU	0.107	0.106	0.192	0.365	0.428	0.240
	Learned Vis+ Id IMU	0.215	0.193	0.193	0.544	0.457	0.320
	Learned Vis+ Learned IMU	**0.084**	**0.099**	**0.161**	**0.362**	**0.368**	**0.215**
Body Translation (m)	Id Vis+ Id IMU	**0.009**	0.034	**0.008**	0.043	0.024	0.024
	Id Vis+ Learned IMU	0.024	0.048	0.026	0.055	0.042	0.039
	Learned Vis+ Id IMU	0.010	0.033	**0.008**	0.043	0.024	0.024
	Learned Vis+ Learned IMU	**0.009**	**0.031**	**0.008**	**0.037**	**0.022**	**0.021**

based calibration. Under such conditions, our learning-based approach remains robust and is able to deliver stable and accurate calibration results. Figure 7.7 visualizes the estimated trajectories during calibration, where *VINS-Mono* exhibits substantial drifts.

7.6 Ablation Study

7.6.1 Effectiveness of the Learned Covariance

To investigate the effectiveness of the learned visual and IMU covariance models, we conduct an ablation study with four configurations: (i) our full model with learned visual and IMU covariances (Learned Vis + Learned IMU), (ii) identity covariances for both visual and IMU measurements (Id Vis + Id IMU), (iii) learned visual covariance with identity IMU covariance (Learned Vis + Id IMU), and (iv) learned IMU covariance with identity visual covariance (Id Vis + Learned IMU). In addition to the estimation accuracy of initial gravity, velocity, and gyroscope bias, we also consider the accuracy of the estimated initial pose in this study. The quantitative results are summarized in Table 7.4.

The results show that incorporating learned covariance models consistently improves initializa-

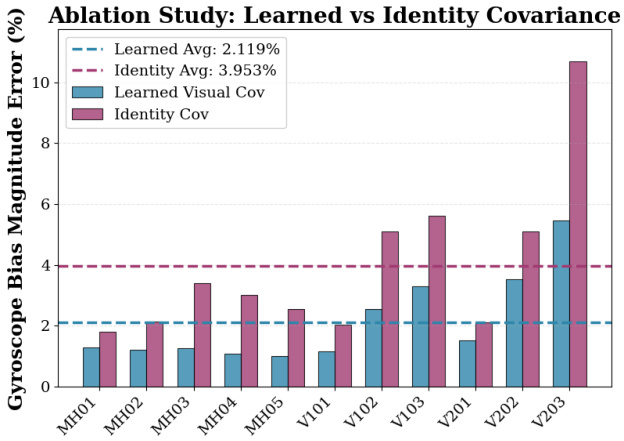

Figure 7.8: Gyroscope bias magnitude error using learned visual covariance versus identity covariance on EuRoC.

Table 7.5: Initialization errors on the EuRoC dataset under with and without learned IMU model.

Metrics	Methods	MH02	MH04	V101	V103	V202	Avg
Velocity (m/s)	w/o Learned IMU	0.011	**0.022**	**0.010**	**0.025**	**0.020**	**0.017**
	w/ Learned IMU	**0.010**	0.023	**0.010**	**0.025**	0.024	0.019
Gravity Direction (deg)	w/o Learned IMU	0.820	0.800	0.872	1.149	**0.804**	0.889
	w/ Learned IMU	**0.166**	**0.208**	**0.332**	**0.581**	**0.804**	**0.418**
Body Rotation (deg)	w/o Learned IMU	0.140	0.149	0.185	0.420	0.398	0.258
	w/ Learned IMU	**0.084**	**0.099**	**0.161**	**0.362**	**0.368**	**0.215**
Body Translation (m)	w/o Learned IMU	**0.009**	**0.031**	**0.008**	0.038	**0.021**	0.022
	w/ Learned IMU	**0.009**	**0.031**	**0.008**	**0.037**	0.022	**0.021**

tion accuracy across all metrics and sequences. Notably, the learned covariances show significant benefits for the initial pose estimation, with body rotation error decreasing from 0.362° to 0.215° (40.6% improvement) and body translation error reducing from 0.024 m to 0.021 m (12.5% improvement). Moreover, the best performance across all metrics is consistently achieved when both learned covariance models are used jointly.

We further analyze the impact of the learned visual covariance on gyroscope bias estimation, which is important when the learned IMU model is unavailable. Figure 7.8 compares the gyroscope bias magnitude error across all sequences between using learned visual covariance and an identity covariance baseline. The results show that incorporating learned visual covariance significantly improves gyroscope bias estimation, reducing the average error from 3.953% to 2.119%, corresponding to a 46.4% relative improvement.

7.6.2 With and Without Learned IMU Model

Table 7.5 presents an ablation study on the effect of the learned IMU model. Overall, incorporating the learned IMU model leads to consistent and significant improvements in orientation-related metrics, including gravity direction and body rotation errors, across all sequences. This

improvement is not only attributed to the learned IMU covariance modeling, but also closely related to the learned IMU model's ability to correct accelerometer measurements, which directly affects gravity alignment and long-term integration.

7.7 Conclusion and Discussion

This chapter presents MAC-I^2, a robust framework for visual-inertial initialization and online calibration that overcomes the limitations of traditional geometry-based methods in challenging environments. By leveraging learning-based visual features and metrics-aware uncertainty modeling, MAC-I^2 enables principled and tuning-free IMU initialization and calibration. The experimental results demonstrate that MAC-I^2 achieves a 1.00 success rate on EuRoC and substantially outperforms all baselines on TUM-VI and VBR, confirming the practical value of metrics-aware covariance modeling for VI system bootstrap. In future work, we will extend MAC-I^2 to multi-frame optimization and full visual-inertial odometry and SLAM systems, as well as broader multi-sensor fusion settings.

Chapter 8

MACVIO (Proposed): Learning-Based Visual-Inertial Odometry with Cross-Modal Uncertainty Fusion

8.1 Introduction

This chapter proposes MACVIO (Metrics-Aware Cross-Modal Visual-Inertial Odometry), which extends the completed single-sensor uncertainty modeling from MAC-VO and AirIMU into a unified visual-inertial estimator. MACVIO builds directly on three foundations established earlier in this dissertation: metrics-aware visual covariance from MAC-VO, learned inertial uncertainty from AirIMU and AirIO, and robust initialization and online calibration from MAC-I^2. The central idea is to fuse uncertainty information from both modalities and adapt fusion behavior online so that VIO remains accurate and stable under lighting degradation, aggressive motion, dynamic scenes, and sensor-quality variation.

Conventional visual-inertial systems rely on fixed fusion weights and manually tuned noise parameters. These settings are brittle under distribution shift because visual and inertial reliability vary with environment and motion context. MACVIO addresses this limitation by learning cross-modal uncertainty correlation and converting it into adaptive fusion weights during optimization.

The motivation for MACVIO is practical deployment. In real robotic systems, sensing quality changes rapidly: cameras degrade in low light or high-speed motion, while inertial signals degrade under vibration, bias drift, and unmodeled dynamics. A fixed-weight fusion strategy cannot respond to these shifts and often fails exactly when robustness is most needed. Therefore, this chapter proposes uncertainty-aware cross-modal control as the missing layer between modality-specific uncertainty prediction and full SLAM integration.

From a thesis-structure perspective, MACVIO is the bridge chapter in Part III. Parts I and II establish reliable uncertainty for individual modalities. MACVIO transforms these single-modal signals into a joint decision mechanism that determines how much each modality should influence state estimation at each time step. This proposed mechanism is expected to improve transfer across robots and environments while reducing manual retuning effort.

8.2 Proposed Method

8.2.1 Basic Idea

The basic idea of MACVIO is to treat visual and inertial uncertainty as complementary signals and fuse them with a learned reliability model. Given visual measurements $\mathcal{Z}^v_{1:T}$ and inertial measurements $\mathcal{Z}^i_{1:T}$, we estimate state trajectory $\mathcal{X}_{1:T}$ by minimizing:

$$\mathcal{X}^*_{1:T} = \arg\min_{\mathcal{X}_{1:T}} \sum_k w^v_k \, \|r^v_k(\mathcal{X}_{1:T})\|^2_{\Sigma^v_k} + \sum_k w^i_k \, \left\|r^i_k(\mathcal{X}_{1:T})\right\|^2_{\Sigma^i_k}, \tag{8.1}$$

where Σ^v_k is provided by the visual uncertainty model (MAC-VO), Σ^i_k is provided by the inertial uncertainty model (AirIMU/AirIO), and (w^v_k, w^i_k) are adaptive fusion weights.

8.2.2 Cross-Modal Reliability Learning

MACVIO learns a cross-modal reliability state from uncertainty and context:

$$\mathbf{s}_k = \phi\big(\Sigma^v_k, \Sigma^i_k, \mathbf{c}_k\big), \tag{8.2}$$

where $\mathbf{c}_k$ includes motion and feature-quality context (for example, image degradation and inertial excitation level). The reliability state is mapped to adaptive weights by:

$$\big(w^v_k, w^i_k\big) = g(\mathbf{s}_k), \quad w^v_k, w^i_k \in [0, 1]. \tag{8.3}$$

This mechanism increases trust in the currently reliable modality and suppresses degraded measurements.

8.2.3 How Previous Chapters Are Fused

MACVIO fuses outputs from prior chapters in a single VIO loop:

- **From MAC-VO:** metrics-aware visual covariance for correspondence filtering and residual weighting.
- **From AirIMU and AirIO:** learned inertial covariance and observability-aware inertial representation for high-dynamic motion.
- **From MAC-I^2:** robust visual-inertial initialization and online calibration priors.

By combining these modules, MACVIO is expected to provide better accuracy than fixed-weight fusion, especially in scenes where one modality temporarily degrades.

8.2.4 Proposed Training and Deployment

The proposed training strategy has two stages. First, pretrained visual and inertial uncertainty modules are loaded from MAC-VO and AirIMU to provide stable covariance priors. Second, the

cross-modal fusion controller is trained to predict adaptive weights that minimize trajectory error and consistency loss. During deployment, MACVIO updates weights online without per-platform retuning, aligning with the thesis goal of minimal additional human engineering.

Chapter 9

MAC-IO (Proposed): Self-Supervised Cross-Platform Adaptation for Inertial Odometry

9.1 Introduction

This chapter proposes MAC-IO, a self-supervised adaptation framework for learned inertial odometry. The starting point is the completed AirIO model in Chapter 6, which provides strong inertial odometry performance but can overfit to its training-domain motion and sensor statistics. When deployed on new platforms, mounting conditions, and unseen trajectories, accuracy can degrade due to distribution shift.

MAC-IO addresses this problem by leveraging the metrics-aware inertial uncertainty and preintegration formulation established in AirIMU (Chapter 5) as additional supervision signals. The key idea is to adapt the AirIO velocity network on new data without requiring external ground truth. To achieve this goal, MAC-IO proposes two self-supervision pathways that are complementary in efficiency and supervision strength.

The motivation is that collecting ground-truth motion labels for each new platform is expensive and often impractical. This limitation is especially severe for aerial, off-road, and safety-critical deployments where data collection conditions are difficult to control. As a result, many learning-based inertial odometry systems are accurate in-domain but fragile out-of-domain. MAC-IO targets this bottleneck by converting estimator structure itself into supervision, so adaptation can occur directly on unlabeled target-domain data.

The chapter is designed as an adaptation layer on top of prior completed work. AirIMU provides metrics-aware inertial uncertainty and preintegration consistency. AirIO provides a strong inertial velocity estimator and EKF fusion loop. MAC-IO combines these ingredients to create a practical transfer mechanism that favors fast large-scale adaptation when compute is limited and stronger iterative refinement when filter feedback is available.

9.2 Proposed Method

9.2.1 Problem Setup

Let a pretrained inertial odometry network predict velocity $\hat{\mathbf{v}}_t$ from IMU windows. Given target-domain IMU data without labels, MAC-IO adapts network parameters θ so that inertial predictions remain consistent with physically grounded estimation signals. The adaptation objective combines two self-supervised losses:

$$\mathcal{L}_{\text{MAC-IO}} = \lambda_{\Delta v}\,\mathcal{L}_{\Delta v} + \lambda_{\text{EKF}}\,\mathcal{L}_{\text{EKF}}. \tag{9.1}$$

9.2.2 Method 1: IMU Preintegration Delta-V Supervision

The first method uses IMU preintegration as a fast self-supervision signal. From raw inertial measurements, we compute the preintegrated velocity increment $\Delta\mathbf{v}^{\text{imu}}_{t\to t+1}$. From network outputs, we form the predicted increment $\Delta\hat{\mathbf{v}}_{t\to t+1} = \hat{\mathbf{v}}_{t+1} - \hat{\mathbf{v}}_t$. The adaptation loss is:

$$\mathcal{L}_{\Delta v} = \sum_t \left\| \Delta\hat{\mathbf{v}}_{t\to t+1} - \Delta\mathbf{v}^{\text{imu}}_{t\to t+1} \right\|^2_{\Sigma_t^{\Delta v}}, \tag{9.2}$$

where $\Sigma_t^{\Delta v}$ is derived from metrics-aware inertial uncertainty.

This method is computationally light, easy to scale, and suitable for large target-domain datasets. Its limitation is that it supervises velocity *changes* rather than absolute velocity, so bias in global velocity can persist.

9.2.3 Method 2: EKF-Filtered Velocity Supervision

The second method uses the filtered state from an uncertainty-aware EKF as a stronger supervisory target. After fusing AirIO network outputs with IMU dynamics in the EKF, we obtain filtered states $(\mathbf{p}_t^f, \mathbf{v}_t^f, \mathbf{R}_t^f)$. MAC-IO then supervises network velocity outputs directly with filtered velocity:

$$\mathcal{L}_{\text{EKF}} = \sum_t \left\| \hat{\mathbf{v}}_t - \mathbf{v}_t^f \right\|^2_{\Sigma_t^f}, \tag{9.3}$$

where Σ_t^f is the EKF state covariance used as confidence weighting.

This method follows a bilevel-style adaptation intuition: the network proposal is fused into the filter, and the improved filtered state is fed back as supervision for iterative refinement. Compared with Method 1, this pathway can supervise absolute velocity and typically yields stronger adaptation, at the cost of higher computation and tighter coupling with the filter loop.

9.2.4 Expected Adaptation Behavior

Together, the two methods provide a practical adaptation hierarchy:

- **Method 1** for fast large-scale adaptation with lightweight supervision.

- **Method 2** for stronger iterative refinement using filter-improved state targets.

The expected outcome is improved inertial odometry performance on unseen platforms and trajectories without manual retuning or external ground-truth labels, directly supporting the thesis objective of minimal additional human engineering.

Chapter 10

MACSLAM (Proposed): Unified Uncertainty-Aware Multi-Frame Visual-Inertial SLAM

10.1 Introduction

This chapter proposes MACSLAM, a unified SLAM framework that combines multi-frame optimization, visual-inertial fusion, and loop closure under a shared uncertainty-aware formulation. It extends the completed modules in earlier chapters: metrics-aware visual covariance from MAC-VO, inertial uncertainty modeling from AirIMU and AirIO, and adaptive visual-inertial fusion from MACVIO.

The key design choice is a unified visual foundation front-end based on DINO features. The same visual representation is used for local geometric matching, global visual place recognition, and semantic-language alignment through lightweight adapters. This design aims to reduce front-end fragmentation and make local odometry, loop closure, and semantic mapping mutually consistent.

The motivation for MACSLAM is that robust autonomy requires more than local odometry. A deployable system must maintain global consistency over long trajectories, recover from drift through loop closure, and support higher-level map understanding for planning and interaction. Existing pipelines often treat these capabilities as loosely connected modules with separate feature spaces, which introduces inconsistency and brittle behavior under domain shift.

MACSLAM addresses this fragmentation through two unification principles. First, visual and inertial information are fused in one uncertainty-aware multi-frame optimizer rather than isolated local modules. Second, one foundation visual representation supports local matching, global place recognition, and semantic-language mapping. This shared representation is intended to improve both robustness and scalability while opening a path toward language-aware semantic SLAM in real deployments.

10.2 Proposed Method

10.2.1 Unified Optimization Objective

Let $\mathcal{X}_{1:T}$ be the trajectory states, $\mathcal{M}$ the map variables, and $\mathcal{L}$ loop-closure constraints. MACSLAM solves a single factor-graph objective:

$$(\mathcal{X}^*_{1:T}, \mathcal{M}^*) = \arg\min_{\mathcal{X}_{1:T}, \mathcal{M}} \sum_{k \in \mathcal{V}} \|r^v_k\|^2_{\Sigma^v_k} + \sum_{k \in \mathcal{I}} \left\|r^i_k\right\|^2_{\Sigma^i_k} + \sum_{k \in \mathcal{L}} \left\|r^{lc}_k\right\|^2_{\Sigma^{lc}_k}, \tag{10.1}$$

where visual, inertial, and loop-closure residuals are jointly optimized with learned, metrics-aware covariance. Unlike conventional pipelines with fixed noise assumptions, MACSLAM propagates data-conditioned uncertainty through all factor types.

10.2.2 DINO-Based Unified Visual Front-End

MACSLAM uses DINO features as a shared visual backbone with three heads:

- **Geometric head:** dense feature matching and correspondence confidence for local visual odometry factors.
- **Global retrieval head:** place descriptors for loop-closure candidate retrieval and verification.
- **Semantic-language head:** adapter from DINO tokens to semantic and language-aligned embeddings for semantic mapping.

The core hypothesis is that one foundation representation can support both local geometry and global semantics more consistently than separate task-specific front-ends. This allows loop closure and semantic mapping to directly reuse features already optimized for odometry.

10.2.3 Visual-Inertial Fusion in Multi-Frame SLAM

Visual and inertial factors are fused under uncertainty-aware weighting. Visual covariance priors come from the metrics-aware modeling of MAC-VO, while inertial covariance priors come from AirIMU/AirIO and are adapted online via MACVIO-style reliability control. Fusion is not implemented as static modality weighting; instead, confidence is adjusted online according to degradation context (for example, motion blur, low texture, aggressive maneuvers, and IMU disturbance).

10.2.4 Loop Closure and Semantic Mapping

Loop closure is modeled as a two-stage process: DINO-based global retrieval proposes candidates, and geometric verification adds loop factors with uncertainty-aware covariance. Candidates with high uncertainty are down-weighted to reduce false closures. In parallel, semantic-language adapters project visual tokens into language-aware map attributes, enabling semantic mapping that can support text-conditioned retrieval and higher-level scene reasoning.

10.2.5 Expected Outcomes

MACSLAM is expected to provide:

- improved trajectory consistency through unified multi-frame visual-inertial optimization,
- more reliable loop closure under domain shift via uncertainty-aware global matching,
- reduced system complexity by using one DINO-based front-end for geometry, retrieval, and semantics,
- semantic mapping with language-aligned features on top of the same SLAM backbone.

These outcomes support the thesis goal of a full spatial perception system that is accurate, robust, and deployable with minimal additional human engineering.

Bibliography

[1] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. 8, 13, 34, 36, 39, 44, 45, 47

[2] Ivan Cisneros, Peng Yin, Ji Zhang, Howie Choset, and Sebastian Scherer. Alto: A large-scale dataset for uav visual place recognition and localization. *arXiv preprint arXiv:2207.12317*, 2022. 9, 52, 61, 72, 73

[3] Sachini Herath, Hang Yan, and Yasutaka Furukawa. Ronin: Robust neural inertial navigation in the wild: Benchmark, evaluations, & new methods. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3146–3152. IEEE, 2020. 13, 21, 51, 52, 53, 65, 66, 67, 77, 78, 79, 86

[4] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic robotics*. MIT press, 2005.

[5] Simon J Julier and Jeffrey K Uhlmann. Unscented filtering and nonlinear estimation. volume 92, pages 401–422. IEEE, 2004.

[6] Tong Qin, Peiliang Li, and Shaojie Shen. Vins-mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4):1004–1020, 2018. 21, 24, 50, 51, 52, 69, 78, 92, 93, 94, 102

[7] Stefan Leutenegger, Simon Lynen, Michael Bosse, Roland Siegwart, and Paul Furgale. Keyframe-based visual–inertial odometry using nonlinear optimization. *The International Journal of Robotics Research*, 34(3):314–334, 2015. 92

[8] Frank Dellaert and Michael Kaess. *Factor graphs for robot perception*, volume 6. Now Publishers, 2017. 19, 21

[9] Ronald Clark, Sen Wang, Hongkai Wen, Andrew Markham, and Niki Trigoni. Vinet: Visual-inertial odometry as a sequence-to-sequence learning problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[10] Berta Bescos, José M Fácil, Javier Civera, and José Neira. Dynaslam: Tracking, mapping, and inpainting in dynamic scenes. *IEEE Robotics and Automation Letters*, 3(4):4076–4083, 2018.

[11] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. *arXiv preprint arXiv:2312.14132*, 2024.

[12] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. *Advances in neural information processing systems*, 34:16558–16569, 2021. 21, 22, 23, 24, 28, 36

[13] Tong Qin, Peiliang Li, and Shaojie Shen. Vins-mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4):1004–1020, 2018.

[14] Stefan Leutenegger. Okvis2: Realtime scalable visual-inertial slam with loop closure. *arXiv preprint arXiv:2202.09199*, 2022.

[15] Tixiao Shan, Brendan Englot, Carlo Ratti, and Daniela Rus. Lvi-sam: Tightly-coupled lidar-visual-inertial odometry via smoothing and mapping. *IEEE International Conference on Robotics and Automation (ICRA)*, pages 5692–5698, 2021. 21

[16] Chunge Bai, Tao Xiao, Yajie Chen, Haoqian Wang, Fang Zhang, and Xiang Gao. Faster-lio: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels. *IEEE Robotics and Automation Letters*, 7(2):4861–4868, 2022.

[17] Jiarong Lin and Fu Zhang. R3live: A robust, real-time, rgb-colored, lidar-inertial-visual tightly-coupled state estimation and mapping package. *IEEE International Conference on Robotics and Automation (ICRA)*, pages 10672–10678, 2022. 21

[18] Chunran Zheng, Qingyan Zhu, Wei Xu, Xiyuan Liu, Qizhi Guo, and Fu Zhang. Fast-livo: Fast and tightly-coupled sparse-direct lidar-inertial-visual odometry. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4003–4009, 2022.

[19] Julian Nubert, Igor Walczak, Shehryar Khattak, Cesar Cadena, and Marco Hutter. Holistic fusion: Task and setup agnostic lidar-visual-inertial place recognition and localization. *arXiv preprint arXiv:2501.00296*, 2025.

[20] Raul Mur-Artal, Jose Maria Martinez Montiel, and Juan D Tardos. Orb-slam: a versatile and accurate monocular slam system. *IEEE transactions on robotics*, 31(5):1147–1163, 2015. 21, 22, 24, 34, 36

[21] Wenshan Wang, Yaoyu Hu, and Sebastian Scherer. Tartanvo: A generalizable learning-based vo. 2020. 24, 28

[22] Zachary Teed, Lahav Lipson, and Jia Deng. Deep patch visual odometry. *Advances in Neural Information Processing Systems*, 36, 2024. 22, 25, 26, 36

[23] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981. 22

[24] Shibo Zhao, Hengrui Zhang, Peng Wang, Lucas Nogueira, and Sebastian Scherer. Super odometry: Imu-centric lidar-visual-inertial estimator for challenging environments. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8729–8736. IEEE, 2021. 21, 22, 24, 77, 78

[25] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, June 2018. doi: 10.1109/cvprw.2018.00060. URL http://dx.doi.org/10.1109/CVPRW.2018.00060. 22, 24, 36

[26] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. Orb: An efficient alternative to sift or surf. In *2011 International Conference on Computer Vision*, pages 2564–2571, 2011. doi: 10.1109/ICCV.2011.6126544. 24, 26, 36

[27] David G. Lowe. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vision*, 60(2):91–110, nov 2004. ISSN 0920-5691. doi: 10.1023/B:VISI.0000029664.99615.94. URL https://doi.org/10.1023/B:VISI.0000029664.99615.94. 22, 24

[28] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. *arXiv preprint arXiv:2312.14132*, 2023. 21, 22, 24, 36

[29] Zhaoyang Huang, Xiaoyu Shi, Chao Zhang, Qiang Wang, Ka Chun Cheung, Hongwei Qin, Jifeng Dai, and Hongsheng Li. Flowformer: A transformer architecture for optical flow. In *European conference on computer vision*, pages 668–685. Springer, 2022. 23, 25

[30] Shihao Jiang, Dylan Campbell, Yao Lu, Hongdong Li, and Richard Hartley. Learning to estimate hidden motions with global motion aggregation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9772–9781, 2021. 23

[31] Georg Klein and David Murray. Parallel tracking and mapping for small ar workspaces. In *2007 6th IEEE and ACM international symposium on mixed and augmented reality*, pages 225–234. IEEE, 2007. 24

[32] Rui Wang, Martin Schworer, and Daniel Cremers. Stereo dso: Large-scale direct sparse visual odometry with stereo cameras. In *Proceedings of the IEEE international conference on computer vision*, pages 3903–3911, 2017. 24

[33] Jakob Engel, Thomas Schöps, and Daniel Cremers. Lsd-slam: Large-scale direct monocular slam. In *European conference on computer vision*, pages 834–849. Springer, 2014. 21, 36

[34] X. Gao, R. Wang, N. Demmel, and D. Cremers. Ldso: Direct sparse odometry with loop closure. In *International Conference on Intelligent Robots and Systems (IROS)*, October 2018. 24

[35] Javier Civera, Andrew J. Davison, and J. M. MartÍnez Montiel. Inverse depth parametrization for monocular slam. *IEEE Transactions on Robotics*, 24(5):932–945, 2008. doi: 10.1109/TRO.2008.2003276. 24, 36

[36] José M. M. Montiel, Javier Civera, and Andrew J. Davison. Unified inverse depth parametrization for monocular slam. In *Robotics: Science and Systems*, 2006. URL https://api.semanticscholar.org/CorpusID:18457284. 24

[37] Shichao Yang and Sebastian Scherer. Cubeslam: Monocular 3-d object slam. *IEEE Transactions on Robotics*, 35(4):925–938, 2019. 24

[38] Yuheng Qiu, Chen Wang, Wenshan Wang, Mina Henein, and Sebastian Scherer. Airdos: Dynamic slam benefits from articulated objects. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 8047–8053. IEEE, 2022. 24, 77

[39] Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II 16*, pages 402–419. Springer, 2020. 24, 36

[40] Chethan M Parameshwara, Gokul Hari, Cornelia Fermüller, Nitin J Sanket, and Yiannis Aloimonos. Diffposenet: Direct differentiable camera pose estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6845–6854, 2022. 24, 36

[41] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4938–4947, 2020. 21, 24, 36

[42] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G Lowe. Unsupervised learning of depth and ego-motion from video. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1851–1858, 2017. 24, 36

[43] Ruihao Li, Sen Wang, Zhiqiang Long, and Dongbing Gu. Undeepvo: Monocular visual odometry through unsupervised deep learning. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 7286–7291. IEEE, 2018. 24, 34, 36

[44] Keisuke Tateno, Federico Tombari, Iro Laina, and Nassir Navab. Cnn-slam: Real-time dense monocular slam with learned depth prediction. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6243–6252, 2017. 24

[45] Gabriele Costante, Michele Mancini, Paolo Valigi, and Thomas A Ciarfuglia. Exploring representation learning with cnns for frame-to-frame ego-motion estimation. *IEEE robotics and automation letters*, 1(1):18–25, 2015. 24

[46] Eric Dexheimer and Andrew J. Davison. Learning a depth covariance function. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, page 13122–13131. IEEE, June 2023. doi: 10.1109/cvpr52729.2023.01261. URL http://dx.doi.org/10.1109/CVPR52729.2023.01261. 24, 36

[47] Xinyu Nie, Dianxi Shi, Ruihao Li, Zhe Liu, and Xucan Chen. Uncertainty-aware self-improving framework for depth estimation. *IEEE Robotics and Automation Letters*, 7(1): 41–48, 2022. doi: 10.1109/LRA.2021.3116293. 24

[48] Anne S. Wannenwetsch, Margret Keuper, and Stefan Roth. Probflow: Joint optical flow and uncertainty estimation. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017. 24, 36

[49] Nan Yang, Lukas von Stumberg, Rui Wang, and Daniel Cremers. D3vo: Deep depth, deep pose and deep uncertainty for monocular visual odometry. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1281–1292, 2020. 24, 25, 36

[50] Taimeng Fu, Shaoshu Su, Yiren Lu, and Chen Wang. islam: Imperative slam. *IEEE Robotics and Automation Letters*, 2024.

[51] Anurag Ranjan, Varun Jampani, Lukas Balles, Kihwan Kim, Deqing Sun, Jonas Wulff, and Michael J Black. Competitive collaboration: Joint unsupervised learning of depth, camera motion, optical flow and motion segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12240–12249, 2019.

[52] Paul-Edouard Sarlin, Ajaykumar Unagar, Mans Larsson, Hugo Germain, Carl Toft, Viktor Larsson, Marc Pollefeys, Vincent Lepetit, Lars Hammarstrand, Fredrik Kahl, and Torsten Sattler. Back to the feature: Learning robust camera localization from pixels to pose. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2021. doi: 10.1109/cvpr46437.2021.00326. URL http://dx.doi.org/10.1109/CVPR46437.2021.00326.

[53] Gabriele Costante and Michele Mancini. Uncertainty estimation for data-driven visual odometry. *IEEE Transactions on Robotics*, 36(6):1738–1757, 2020. 24

[54] D Muhle, L Koestler, KM Jatavallabhula, and D Cremers. Learning correspondence uncertainty via differentiable nonlinear least squares. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13102–13112, 2023. 24, 36

[55] Nimet Kaygusuz, Oscar Mendez, and Richard Bowden. Mdn-vo: Estimating visual odometry with confidence. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, September 2021. doi: 10.1109/iros51168.2021.9636827. URL http://dx.doi.org/10.1109/IROS51168.2021.9636827.

[56] Sadegh Rabiee and Joydeep Biswas. Iv-slam: Introspective vision for simultaneous localization and mapping, 2020.

[57] Riku Murai, Eric Dexheimer, and Andrew J Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. *arXiv preprint arXiv:2412.12392*, 2024. 24

[58] Philipp Lindenberger, Paul-Edouard Sarlin, Viktor Larsson, and Marc Pollefeys. Pixel-perfect structure-from-motion with featuremetric refinement. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, October 2021. doi: 10.1109/iccv48922.2021.00593. URL http://dx.doi.org/10.1109/ICCV48922.2021.00593. 24

[59] Rebecca L Russell and Christopher Reale. Multivariate uncertainty in deep learning. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12):7937–7943, 2021. 25, 54, 57

[60] Anastasios N. Angelopoulos and Stephen Bates. *Conformal Prediction: A Gentle Introduction*. Now Foundations and Trends, 2023.

[61] Yuheng Qiu, Chen Wang, Can Xu, Yutian Chen, Xunfei Zhou, Youjie Xia, and Sebastian Scherer. Airimu: Learning uncertainty propagation for inertial odometry. 2023. 25, 83, 84, 85, 86, 93, 94, 95

[62] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. 2020. 26, 28

[63] Chen Wang, Dasong Gao, Kuan Xu, Junyi Geng, Yaoyu Hu, Yuheng Qiu, Bowen Li, Fan Yang, Brady Moon, Abhinav Pandey, et al. Pypose: A library for robot learning with physics-based optimization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22024–22034, 2023. 28, 52, 62

[64] Zitong Zhan, Xiangfu Li, Qihang Li, Haonan He, Abhinav Pandey, Haitao Xiao, Yangmengfei Xu, Xiangyu Chen, Kuan Xu, Kun Cao, et al. Pypose v0. 6: The imperative programming interface for robotics. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) Workshop*, 2023. 28

[65] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W Achtelik, and Roland Siegwart. The euroc micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016. doi: 10.1177/0278364915620033. URL https://doi.org/10.1177/0278364915620033. 28, 29, 40, 44, 45

[66] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012. 28, 30, 40, 44, 45

[67] Sen Wang, Ronald Clark, Hongkai Wen, and Niki Trigoni. Deepvo: Towards end-to-end visual odometry with deep recurrent convolutional neural networks. In *2017 IEEE international conference on robotics and automation (ICRA)*, pages 2043–2050. IEEE, 2017. 34

[68] Wenshan Wang, Yaoyu Hu, and Sebastian Scherer. Tartanvo: A generalizable learning-based vo. In *Conference on Robot Learning*, pages 1761–1772. PMLR, 2021.

[69] Zachary Teed, Lahav Lipson, and Jia Deng. Deep patch visual odometry. *Advances in Neural Information Processing Systems*, 36:39033–39051, 2023.

[70] Yuheng Qiu, Yutian Chen, Zihao Zhang, Wenshan Wang, and Sebastian Scherer. Mac-vo: Metrics-aware covariance for learning-based stereo visual odometry. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3814. IEEE, 2025. 34, 35, 37, 44, 47, 93, 96

[71] Raul Mur-Artal and Juan D Tardós. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE transactions on robotics*, 33(5):1255–1262, 2017. 34

[72] Carlos Campos, Richard Elvira, Juan J Gomez Rodriguez, Jose MM Montiel, and Juan D Tardos. Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021. 21, 34, 93, 102

[73] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021. 34, 38

[74] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. 34, 38

[75] Sifan Zhou, Liang Li, Xinyu Zhang, Bo Zhang, Shipeng Bai, Miao Sun, Ziyu Zhao, Xiaobo Lu, and Xiangxiang Chu. Lidar-ptq: Post-training quantization for point cloud 3d object detection. In *The Twelfth International Conference on Learning Representations*, 2024. 34, 40

[76] Zhihang Yuan, Chenhao Xue, Yiqi Chen, Qiang Wu, and Guangyu Sun. Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization. In *European Conference on Computer Vision*, 2022.

[77] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. 34, 36

[78] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37: 100213–100240, 2024. 34, 36, 38, 44, 45, 47

[79] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4909–4916. IEEE, 2020. 35, 40, 41, 44

[80] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023. 36

[81] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024. 36

[82] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821*, 2023. 36

[83] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. Omniquant: Omnidirectionally calibrated quantization for large language models. *CoRR*, abs/2308.13137, 2023. 36

[84] Xingxing Zuo, Nathaniel Merrill, Wei Li, Yong Liu, Marc Pollefeys, and Guoquan Huang. Codevio: Visual-inertial odometry with learned optimizable dense depth. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14382–14388. IEEE, 2021. 36

[85] Yingye Xin, Xingxing Zuo, Dongyue Lu, and Stefan Leutenegger. Simplemapping: Real-time visual-inertial dense mapping with deep multi-view stereo. *arXiv preprint arXiv:2306.08648*, 2023. 36

[86] Yuxiang Peng, Chuchu Chen, and Guoquan Huang. Quantized visual-inertial odometry. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 17954–17960. IEEE, 2024. 36

[87] Yuxiang Peng, Chuchu Chen, and Guoquan Huang. Qvio2: Quantized map-based visual-inertial odometry. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1053–1059. IEEE, 2025. 36

[88] Niraj Pudasaini, Muhammad Abdullah Hanif, and Muhammad Shafique. Spaq-dl-slam: Towards optimizing deep learning-based slam for resource-constrained embedded platforms. In *2024 18th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, pages 972–978. IEEE, 2024. 36

[89] Ron Banner, Yury Nahshan, and Daniel Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. *Advances in Neural Information Processing Systems*, 32, 2019. 38, 40

[90] JiangYong Yu, Sifan Zhou, Dawei Yang, Shuo Wang, Shuoyu Li, Xing Hu, Chen Xu, Zukang Xu, Changyong Shu, and Zhihang Yuan. Mquant: Unleashing the inference potential of multimodal large language models via full static quantization. In *Proceedings of the 33rd ACM international conference on multimedia (MM'25)*, 2025. 38

[91] Tianchen Zhao, Xuefei Ning, Tongcheng Fang, Enshu Liu, Guyue Huang, Zinan Lin, Shengen Yan, Guohao Dai, and Yu Wang. Mixdq: Memory-efficient few-step text-to-image diffusion models with metric-decoupled mixed precision quantization. In *European Conference on Computer Vision*, pages 285–302. Springer, 2024. 39

[92] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In *2011 31st international conference on distributed computing systems workshops*, pages 166–171. IEEE, 2011. 39

[93] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. Brecq: Pushing the limit of post-training quantization by block reconstruction. In *International Conference on Learning Representations*. 40

[94] Junjie Bai, Fang Lu, Ke Zhang, et al. Onnx: Open neural network exchange. https://github.com/onnx/onnx, 2019. 47

[95] NVIDIA Corporation. *NVIDIA TensorRT*, 2024. URL https://developer.nvidia.com/tensorrt. 47

[96] Shibo Zhao, Damanpreet Singh, Haoxiang Sun, Rushan Jiang, YuanJun Gao, Tianhao Wu, Jay Karhade, Chuck Whittaker, Ian Higgins, Jiahe Xu, et al. Subt-mrs: A subterranean, multi-robot, multi-spectral and multi-degraded dataset for robust slam. *arXiv preprint arXiv:2307.07607*, 2023. 50, 52, 61, 70

[97] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W Achtelik, and Roland Siegwart. The euroc micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016. 61, 63, 85, 101

[98] Andreas Geiger, P Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics: the kitti dataset. *The International Journal of Robotics Research*, 32:1231–1237, 09 2013. doi: 10.1177/0278364913491297. 50, 61, 66

[99] Honghui Qi and John B Moore. Direct kalman filtering approach for gps/ins integration. *IEEE Transactions on Aerospace and Electronic Systems*, 38(2):687–693, 2002. 50

[100] Juan David Hincapié-Ramos, Kasim Ozacar, Pourang P Irani, and Yoshifumi Kitamura. Gyrowand: Imu-based raycasting for augmented reality head-mounted displays. In *Proceedings of the 3rd ACM Symposium on Spatial User Interaction*, pages 89–98, 2015. 50

[101] Matteo Menolotto, Dimitrios-Sokratis Komaris, Salvatore Tedesco, Brendan O'Flynn, and Michael Walsh. Motion capture technology in industrial applications: A systematic review. *Sensors*, 20(19):5687, 2020. 50

[102] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. Imu preintegration on manifold for efficient visual-inertial maximum-a-posteriori estimation. 2015. 20, 21, 50, 52, 54, 56, 62, 77, 78, 86

[103] Changhao Chen, Xiaoxuan Lu, Andrew Markham, and Niki Trigoni. Ionet: Learning to cure the curse of drift in inertial odometry. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. 21, 50, 52, 78

[104] Billur Barshan and Hugh F Durrant-Whyte. Inertial navigation systems for mobile robots. *IEEE transactions on robotics and automation*, 11(3):328–342, 1995. 50, 52

[105] Joern Rehder, Janosch Nikolic, Thomas Schneider, Timo Hinzmann, and Roland Siegwart. Extending kalibr: Calibrating the extrinsics of multiple imus and of individual axes. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4304–4311. IEEE, 2016. 50, 51, 52, 63

[106] Ruihao Li, Chunlian Fu, Wei Yi, and Xiaodong Yi. Calib-net: Calibrating the low-cost imu via deep convolutional neural network. *Frontiers in Robotics and AI*, 8:772583, 2022. 51

[107] Chi-Shih Jao, Danmeng Wang, and Andrei M Shkel. Prio-imu: Prioritizable imu array for enhancing foot-mounted inertial navigation accuracy. In *2023 IEEE International Symposium on Inertial Sensors and Systems (INERTIAL)*, pages 1–4. IEEE, 2023. 51

[108] Domenico Capriglione, Marco Carratù, Marcantonio Catelani, Lorenzo Ciani, Gabriele Patrizi, Antonio Pietrosanto, and Paolo Sommella. Experimental analysis of filtering algorithms for imu-based applications under vibrations. *IEEE Transactions on Instrumentation and Measurement*, 70:1–10, 2020. 52

[109] Mingyang Li and Anastasios I Mourikis. Online temporal calibration for camera–imu systems: Theory and algorithms. *The International Journal of Robotics Research*, 33(7): 947–964, 2014. 51

[110] David Tedaldi, Alberto Pretto, and Emanuele Menegatti. A robust and easy to implement method for imu calibration without external equipments. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3042–3049. IEEE, 2014. 21, 51, 52

[111] Paul Furgale, Joern Rehder, and Roland Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1280–1286. IEEE, 2013. 92, 94

[112] Christian Krebs. Generic imu-camera calibration algorithm: Influence of imu-axis on each other. Tech. rep., Autonomous Systems Lab, ETH Zurich, 2012. [Online]. Available: `http://students.asl.ethz.ch/upl_pdf/396-report.pdf`. 51, 52

[113] Martin Brossard, Axel Barrau, and Silvère Bonnabel. Ai-imu dead-reckoning. *IEEE Transactions on Intelligent Vehicles*, 5(4):585–595, 2020. 51, 54

[114] Russell Buchanan, Varun Agrawal, Marco Camurri, Frank Dellaert, and Maurice Fallon. Deep imu bias inference for robust visual-inertial odometry with factor graphs. *IEEE Robotics and Automation Letters*, 8(1):41–48, 2022. 51, 53, 77

[115] Wenxin Liu, David Caruso, Eddy Ilg, Jing Dong, Anastasios I Mourikis, Kostas Daniilidis, Vijay Kumar, and Jakob Engel. Tlio: Tight learned inertial odometry. *IEEE Robotics and Automation Letters*, 5(4):5653–5660, 2020. 21, 51, 52, 53, 54, 65, 66, 77, 78, 83, 84, 86

[116] Scott Sun, Dennis Melamed, and Kris Kitani. Idol: Inertial deep orientation-estimation and localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6128–6137, 2021. 51, 52, 53, 78

[117] Russell Buchanan, Marco Camurri, Frank Dellaert, and Maurice Fallon. Learning inertial odometry for dynamic legged robot state estimation. In *Conference on robot learning*, pages 1575–1584. PMLR, 2022. 51, 78

[118] Ming Zhang, Mingming Zhang, Yiming Chen, and Mingyang Li. Imu data processing for inertial aided navigation: A recurrent neural network based approach. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3992–3998. IEEE, 2021. 51, 53, 63, 64

[119] Garrett Hemann, Sanjiv Singh, and Michael Kaess. Long-range gps-denied aerial inertial navigation with lidar localization. In *2016 IEEE/RSJ International conference on intelligent robots and systems (IROS)*, pages 1659–1666. IEEE, 2016. 52, 61

[120] Joern Rehder and Roland Siegwart. Camera/imu calibration revisited. *IEEE Sensors Journal*, 17(11):3257–3268, 2017. 52

[121] JC Hung, JS Hunter, WW Stripling, and HV White. Size effect on navigation using a strapdown imu. *US Army Missile Research and Development Command, Guidance and Control Directorate Technology Laboratory, Tech. Rep*, 1979. 52

[122] Xiaoji Niu, You Li, Hongping Zhang, Qingjiang Wang, and Yalong Ban. Fast thermal calibration of low-grade inertial sensors and inertial measurement units. *Sensors*, 13(9): 12192–12217, 2013. 52

[123] Martin Brossard, Axel Barrau, Paul Chauchat, and Silvère Bonnabel. Associating uncertainty to extended poses for on lie group imu preintegration with rotating earth. *IEEE Transactions on Robotics*, 38(2):998–1015, 2021. 52

[124] Umar Qureshi and Farid Golnaraghi. An algorithm for the in-field calibration of a mems imu. *IEEE Sensors Journal*, 17(22):7479–7486, 2017. 52

[125] Yulin Yang, Patrick Geneva, Xingxing Zuo, and Guoquan Huang. Online self-calibration for visual-inertial navigation: Models, analysis, and degeneracy. *IEEE Transactions on Robotics*, 2023. 52

[126] Jiajun Lv, Jinhong Xu, Kewei Hu, Yong Liu, and Xingxing Zuo. Targetless calibration of lidar-imu system based on continuous-time batch estimation. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9968–9975. IEEE, 2020. 52

[127] Hang Yan, Qi Shan, and Yasutaka Furukawa. Ridi: Robust imu double integration. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 621–636, 2018. 52, 77, 78

[128] Xiya Cao, Caifa Zhou, Dandan Zeng, and Yongliang Wang. Rio: Rotation-equivariance supervised learning of robust inertial odometry. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6614–6623, 2022. 52, 53, 79

[129] Martin Brossard, Silvere Bonnabel, and Axel Barrau. Denoising imu gyroscopes with deep learning for open-loop attitude estimation. *IEEE Robotics and Automation Letters*, 5(3): 4796–4803, 2020. 52, 53, 59, 63, 65

[130] Fernando Nobre and Christoffer Heckman. Learning to calibrate: Reinforcement learning for guided calibration of visual–inertial rigs. *The International Journal of Robotics Research*, 38(12-13):1388–1402, 2019. 53

[131] Naser El-Sheimy, Haiying Hou, and Xiaoji Niu. Analysis and modeling of inertial sensors using allan variance. *IEEE Transactions on instrumentation and measurement*, 57(1):140–149, 2007. 54

[132] Dan Solodar and Itzik Klein. Vio-dualpronet: Visual-inertial odometry with learning based process noise covariance. *arXiv preprint arXiv:2308.11228*, 2023. 54

[133] David Schubert, Thore Goll, Nikolaus Demmel, Vladyslav Usenko, Jörg Stückler, and Daniel Cremers. The tum vi benchmark for evaluating visual-inertial odometry. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1680–1687. IEEE, 2018. 61, 63, 101

[134] Patrick Geneva, Kevin Eckenhoff, Woosik Lee, Yulin Yang, and Guoquan Huang. Openvins: A research platform for visual-inertial estimation. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4666–4672. IEEE, 2020. 21, 63, 92, 94

[135] Yafei Hu, Junyi Geng, Chen Wang, John Keller, and Sebastian Scherer. Off-policy evaluation with online adaptation for robot exploration in challenging environments. *IEEE Robotics and Automation Letters (RA-L)*, 8(6):3780–3787, 2023. 77

[136] Xiaofeng Guo, Guanqi He, Jiahe Xu, Mohammadreza Mousaei, Junyi Geng, Sebastian Scherer, and Guanya Shi. Flying calligrapher: Contact-aware motion and force planning and control for aerial manipulation. *IEEE Robotics and Automation Letters*, 9(12):11194–11201, 2024. doi: 10.1109/LRA.2024.3486236. 77

[137] Shibo Zhao, Yuanjun Gao, Tianhao Wu, Damanpreet Singh, Rushan Jiang, Haoxiang Sun, Mansi Sarawata, Yuheng Qiu, Warren Whittaker, Ian Higgins, Yi Du, Shaoshu Su, Can Xu, John Keller, Jay Karhade, Lucas Nogueira, Sourojit Saha, Ji Zhang, Wenshan Wang, Chen Wang, and Sebastian Scherer. Subt-mrs dataset: Pushing slam towards all-weather environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22647–22657, June 2024. 77

[138] Giovanni Cioffi, Leonard Bauersfeld, Elia Kaufmann, and Davide Scaramuzza. Learned inertial odometry for autonomous drone racing. In *IEEE Robotics and Automation Letters (RA-L)*, 2023. 77, 78, 79, 84, 86, 90

[139] Shuo Yang, Zixin Zhang, Benjamin Bokser, and Zachary Manchester. Multi-imu proprioceptive odometry for legged robots. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 774–779. IEEE, 2023. 77

[140] Xiaoqiang Teng, Pengfei Xu, Deke Guo, Yulan Guo, Runbo Hu, Hua Chai, and Didi Chuxing. Arpdr: An accurate and robust pedestrian dead reckoning system for indoor localization on handheld smartphones. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10888–10893. IEEE, 2020. 77

[141] Kunyi Zhang, Chenxing Jiang, Jinghang Li, Sheng Yang, Teng Ma, Chao Xu, and Fei Gao. Dido: Deep inertial quadrotor dynamical odometry. *IEEE Robotics and Automation Letters*, 7(4):9083–9090, 2022. 77, 79

[142] James Svacha, James Paulos, Giuseppe Loianno, and Vijay Kumar. Imu-based inertia estimation for a quadrotor using newton-euler dynamics. *IEEE Robotics and Automation Letters*, 5(3):3861–3867, 2020. 78

[143] Sachini Herath, David Caruso, Chen Liu, Yufan Chen, and Yasutaka Furukawa. Neural inertial localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6604–6613, 2022. 78

[144] Yingying Wang, Hu Cheng, and Max Q-H Meng. A2dio: Attention-driven deep inertial odometry for pedestrian localization based on 6d imu. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 819–825. IEEE, 2022. 78

[145] Xiangyu Deng, Shenyue Wang, Chunxiang Shan, Jinjie Lu, Ke Jin, Jijunnan Li, and Yandong Guo. Data-driven based cascading orientation and translation estimation for inertial navigation. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3381–3388. IEEE, 2023. 78

[146] Martin Brossard, Axel Barrau, and Silvere Bonnabel. Rins-w: Robust inertial navigation system on wheels. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2068–2075. IEEE, 2019. 79

[147] Jan Steinbrener, Christian Brommer, Thomas Jantos, Alessandro Fornasier, and Stephan Weiss. Improved state propagation through ai-based pre-processing and down-sampling of high-speed inertial data. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 6084–6090. IEEE, 2022. 79

[148] Angad Bajwa, Charles Champagne Cossette, Mohammed Ayman Shalaby, and James Richard Forbes. Dive: Deep inertial-only velocity aided estimation for quadrotors. *IEEE Robotics and Automation Letters*, 2024. 79, 89

[149] Amado Antonini, Winter Guerra, Varun Murali, Thomas Sayre-McCord, and Sertac Karaman. The blackbird dataset: A large-scale dataset for uav perception in aggressive flight. In *International Symposium on Experimental Robotics*, pages 130–139. Springer, 2018. 81, 85

[150] Marcelo Jacinto, João Pinto, Jay Patrikar, John Keller, Rita Cunha, Sebastian Scherer, and António Pascoal. Pegasus simulator: An isaac sim framework for multiple aerial vehicles simulation. In *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 917–922, 2024. doi: 10.1109/ICUAS60882.2024.10556959. 81, 86

[151] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008. 82

[152] Weihan Li, Shengyu Zhao, Kun Qian, Yibin Wang, and Shoudong Li. Stereo visual-inertial odometry with online initialization and extrinsic self-calibration. *IEEE Transactions on Instrumentation and Measurement*, 72:1–14, 2023. 92

[153] Agostino Martinelli. Closed-form solution of visual-inertial structure from motion. *International Journal of Computer Vision*, 106(2):138–152, 2013. 93

[154] James Kaiser, Agostino Martinelli, Flavio Fontana, and Davide Scaramuzza. A robust and modular multi-sensor fusion approach applied to mav navigation. *IEEE Robotics and Automation Letters*, 2(1):18–25, 2017.

[155] David Zuniga-Noel, Jose-Raul Ruiz-Sarmiento, and Javier Gonzalez-Jimenez. An efficient solution to non-minimal case essential matrix estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(11):2707–2719, 2020. 93

[156] Changshi Mu, Qingxuan Zhang, Lihua Xie, and Hesheng Zhang. A robust and efficient visual-inertial initialization with probabilistic normal epipolar constraint. *IEEE Robotics and Automation Letters*, 10(2):1234–1241, 2025. 93, 102

[157] Yijia He, Bo Xu, Zhanpeng Ouyang, and Hongdong Li. A rotation-translation-decoupled solution for robust and efficient visual-inertial initialization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 787–796, 2023. doi: 10.1109/CVPR52729.2023.00078. 93, 102, 103

[158] Weihan Wang, Kun Li, Yue Chen, and Ming Zhang. Stereo-nec: Enhancing stereo visual-inertial slam initialization with normal epipolar constraints. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3456–3463. IEEE, 2024. 93, 102

[159] Yancheng Liu, Hao Wang, and Jie Chen. Eta-init: Enhancing the translation accuracy for stereo visual-inertial slam initialization. *arXiv preprint arXiv:2404.12345*, 2024. 93

[160] Wei Zhang, Xiaoyu Li, Chen Wang, and Shoudong Huang. Ple-slam: A visual-inertial slam based on point-line features and efficient imu initialization. *arXiv preprint arXiv:2404.56789*, 2024. 93

[161] Jaehoon Kim, Sungho Park, and Kyungdon Lee. Edi: Eskf-based disjoint initialization for visual-inertial slam systems. *IEEE Robotics and Automation Letters*, 8(7):4321–4328, 2023. 94

[162] Mingyang Zhou, Xiaoli Chen, Bin Wang, and Shaojie Li. A robust parallel initialization method for monocular visual-inertial slam. *IEEE Sensors Journal*, 22(18):17845–17854, 2022. 94

[163] Raul Mur-Artal and Juan D Tardos. Visual-inertial monocular slam with map reuse. *IEEE Robotics and Automation Letters*, 2(2):796–803, 2017. 94

[164] Xiaoming Chen, Yue Liu, Bin Zhang, and Sheng Wang. Fast and robust initialization for visual-inertial slam. *IEEE Transactions on Robotics*, 35(4):1123–1135, 2019. 94

[165] Paul Furgale, Joern Rehder, and Roland Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1280–1286, 2013. doi: 10.1109/IROS.2013.6696514. 94, 102, 104

[166] Jianjun Huang and Ming Liu. Online initialization and automatic camera-imu extrinsic calibration for monocular visual-inertial slam. *arXiv preprint arXiv:1808.03498*, 2018. 21, 94

[167] Jingwei Wang, Qingjie Zhang, and Yang Liu. A dynamic and static combined camera-imu extrinsic calibration method. *Measurement*, 235:114920, 2025. 94

[168] Chunxiao Qiao, Tao Pu, Hao Zhang, Jin Xie, and Maojun Yao. Viw-fusion: Extrinsic calibration and pose estimation for visual-imu-wheel encoder system. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8714–8721. IEEE, 2023. 94

[169] Bruno Petit, Alice Lesecq, and Eric Marchand. Time shifted imu preintegration for temporal calibration. In *2020 International Conference on 3D Vision (3DV)*, pages 909–918. IEEE, 2020. 94

[170] Seunghyun Lee, Younghun Kim, and Frank Park. Lpl-vio: monocular visual-inertial odometry with deep learning-based point and line features. *arXiv preprint arXiv:2403.12720*, 2024. 94

[171] Jongmin Lee, Byungjin Huh, and Minsu Cho. Learning rotation-equivariant features for visual correspondence. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21887–21896, 2023. 94

[172] David B. Adrian, Martin Humenberger, Gabriela Csurka, and Torsten Sattler. Cycle-correspondence loss: Learning dense view-invariant visual features. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5544–5550. IEEE, 2024. 94

[173] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. On-manifold preintegration for real-time visual–inertial odometry. *IEEE Transactions on Robotics*, 33(1): 1–21, 2017. doi: 10.1109/TRO.2016.2597321. 95, 99

[174] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4909–4916. IEEE, 2020. 97, 102

[175] Zhenfei Yang and Shaojie Shen. Monocular visual–inertial state estimation with online initialization and camera–imu extrinsic calibration. *IEEE Transactions on Automation Science and Engineering*, 14(1):39–51, 2017. doi: 10.1109/TASE.2016.2550621. 100

[176] Leonardo Brizi, Emanuele Giacomini, Luca Di Giammarino, Simone Ferrari, Omar Salem, Lorenzo De Rebotti, and Giorgio Grisetti. Vbr: A vision benchmark in rome. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15868–15874. IEEE, 2024. 101